

Review article

Synthesis of model predictive control and reinforcement learning: Survey and classification[☆]

Rudolf Reiter^{a,*,1}, Jasper Hoffmann^{b,1}, Dirk Reinhardt^d, Florian Messerer^a,
Katrín Baumgärtner^a, Shambhuraj Sawant^d, Joschka Bödecker^a, Moritz Diehl^{a,c},
Sebastien Gros^d

^a Department of Microsystems Engineering, University of Freiburg, Freiburg im Breisgau, 79110, Baden-Württemberg, Germany

^b Department of Computer Science, University of Freiburg, Freiburg im Breisgau, 79110, Baden-Württemberg, Germany

^c Department of Mathematics, University of Freiburg, Freiburg im Breisgau, 79104, Baden-Württemberg, Germany

^d Department of Engineering Cybernetics, Norwegian University of Science and Technology, Trondheim, 7034, Norway

ARTICLE INFO

Keywords:

Model predictive control

Reinforcement learning

Approximate dynamic programming

ABSTRACT

Model predictive control (MPC) and reinforcement learning (RL) are two successful control techniques for Markov decision processes. Both approaches are derived from similar fundamental principles, and both are widely used in practical applications, including robotics, process control, energy systems, and autonomous driving. Despite their similarities, MPC and RL follow distinct paradigms that emerged from diverse communities and different requirements. Various technical discrepancies, particularly the role of an environment model as part of the algorithm, lead to methodologies with nearly complementary advantages. Due to their orthogonal benefits, research interest in combination methods has recently increased significantly, resulting in a large and growing set of complex ideas that leverage MPC and RL. This work illuminates the differences, similarities, and fundamentals that enable various combination algorithms and categorizes existing work accordingly. Particularly, we focus on the versatile actor–critic RL approach as a basis for our categorization and examine how the online optimization approach of MPC can be used to improve the overall closed-loop performance of a policy.

Contents

1. Introduction	3
1.1. Scope	3
1.2. Related work	3
1.3. Notation	4
1.4. Overview	4
2. Problem setting	4
3. Reinforcement learning	5
3.1. Theoretical background	5
3.1.1. Dynamic programming	6
3.1.2. Temporal difference (TD) methods	6
3.1.3. Policy gradient methods	7
3.2. Deep reinforcement learning methods	7
3.2.1. Deep Q-networks	7

[☆] This research was supported by DFG, Germany, via projects 504452366 (SPP2364) and 525018088, by BMWK, via 03EI4057A and 03EN3054B, by the EU via ELO-X 953348, and by NFR, Norway through the project Safe Reinforcement Learning using Model Predictive Control (SARLEM, grant number 300172).

* Correspondence to: University of Freiburg, Georges-Köhler-Allee 102, Room 00-075, 79110, Freiburg im Breisgau, Germany.

E-mail addresses: rreiter@ifi.uzh.ch (R. Reiter), hoffmaja@informatik.uni-freiburg.de (J. Hoffmann), dirk.p.reinhardt@ntnu.no (D. Reinhardt), florian.messerer@imtek.uni-freiburg.de (F. Messerer), katrin.baumgaertner@imtek.uni-freiburg.de (K. Baumgärtner), shambhuraj.sawant@ntnu.no (S. Sawant), jboeddeck@informatik.uni-freiburg.de (J. Bödecker), moritz.diehl@imtek.uni-freiburg.de (M. Diehl), sebastien.gros@ntnu.no (S. Gros).

¹ Authors contributed equally.

3.2.2.	Deep deterministic policy gradient.....	7
3.2.3.	Proximal-policy optimization.....	7
3.2.4.	Soft actor-critic.....	8
4.	Model predictive control.....	8
4.1.	MPC problem formulations.....	8
4.1.1.	Dynamics model.....	9
4.1.2.	Objective.....	9
4.1.3.	Constraints.....	9
4.1.4.	Uncertainty-aware MPC.....	9
4.1.5.	Optimization problem classification.....	10
4.1.6.	Suboptimality and control theory.....	10
4.2.	Numerical methods for MPC problems.....	10
4.2.1.	Sampling-based methods.....	10
4.2.2.	Derivative-based numerical optimization.....	11
5.	Comparison of MPC and RL.....	12
5.1.	State space.....	12
5.2.	Model and application.....	12
5.3.	Intrinsic stochasticity.....	13
5.4.	Model mismatch.....	13
5.5.	Stability.....	14
5.6.	Constraints.....	14
5.7.	Online computation time.....	14
5.8.	Offline computation, engineering, and maintenance.....	14
5.9.	Generalization.....	14
5.10.	Performance in practice.....	14
6.	Combination approaches.....	15
6.1.	Architectures of parameterized MPC.....	15
6.1.1.	Integrated architecture.....	16
6.1.2.	Algorithmic architecture.....	16
6.1.3.	Hierarchical architecture.....	16
6.1.4.	Parallel architecture.....	16
6.1.5.	Parameterized architecture.....	16
7.	MPC as an expert actor.....	17
7.1.	Imitation learning from MPC.....	17
7.2.	Guided policy search using MPC.....	18
8.	MPC within the Deployed Policy.....	18
8.1.	Aligned learning.....	18
8.1.1.	Learning the model.....	18
8.1.2.	Learning the terminal value function.....	19
8.2.	Closed-loop learning.....	19
8.2.1.	Differentiable MPC.....	19
8.2.2.	MPC as part of the environment.....	21
8.2.3.	Exploration.....	21
8.3.	MPC for pre- and postprocessing.....	21
8.3.1.	Preprocessing: MPC as a reference generator.....	21
8.3.2.	MPC for postprocessing.....	21
8.4.	MPC optimization guided by RL.....	22
9.	MPC as a critic.....	22
9.1.	MPC as an expert critic.....	23
9.2.	MPC as a learnable critic.....	24
9.3.	MPC as a learnable actor-critic.....	24
10.	Theoretical considerations for combining MPC and RL.....	24
10.1.	Extensions of MPC-MDP equivalence.....	25
10.2.	Approximating discounted MDP with undiscounted MPC.....	25
10.3.	State-transition model for closed-loop optimality.....	26
11.	Software tools and implementation aspects.....	27
11.1.	Integrating software from machine learning and numerical optimization.....	27
11.2.	Implementation aspects.....	27
12.	Conclusion and discussion.....	27
12.1.	Future research directions.....	28
	Declaration of competing interest.....	29
	Appendix A. Differentiating through a parameterized MPC layer.....	29
	Appendix B. Gradients in the hierarchical architecture.....	29
	B.1. Parameter critic.....	29
	B.2. Control critic.....	29
	Data availability.....	29
	References.....	30

1. Introduction

Real-world sequential decision-making problems are naturally modeled as Markov decision processes (MDPs), which provide a rigorous framework for optimal control under uncertainty. Solving an MDP entails finding a policy that minimizes the expected cumulative costs for an agent interacting with a stochastic environment.

Two particularly influential and complementary paradigms to efficiently solve MDPs are model predictive control (MPC) and reinforcement learning (RL). Within the MPC framework, an optimization problem that approximates the MDP is solved online. The related MPC algorithms involve simulating an internal prediction model to evaluate and optimize future controls. These optimized controls are applied to the real-world environment in a closed loop at each time step. Historically, the success of MPC has been driven by the development of powerful numerical optimization algorithms, such as linear programming (Dantzig, 1956). Due to the limitations of the involved numerical optimization, most algorithms still require differentiable models of an optimization-friendly structure, often obtained from physical first principles.

In contrast, within the RL framework, a policy is expressed as a parameterized function of the state and is iteratively improved by interacting with the environment, adapting its parameters based on the observed cost. Given sufficient interaction, the obtained policy can become closed-loop optimal, without requiring the composition or the predictive simulation of an environment model at each step. These algorithms are referred to as model-free RL. Some RL algorithms utilize models, either in the context of online planning (Bertsekas, 2022a) or by building models of observations emerging from interactions with the environment during learning (Sutton, 1991). Under this classification (Moerland et al., 2023), the proposed combinations of MPC and RL can be viewed as novel approaches that expand existing model-based RL algorithms. In the context of this survey, RL mainly refers to the fundamental concepts of policy learning as proposed, for instance, in Sutton and Barto (2018).

Remarkably, both MPC and RL have shown outstanding performance in classical real-world control problems (Schwenzer et al., 2021). However, the applications differ depending on the specific requirements and system properties. For instance, if measurement data is scarce and expensive, and the environment can be described by optimization-friendly models, MPC is often favored (Görges, 2017) due to its sample efficiency. On the contrary, RL is successfully implemented in settings where lots of training data can be generated (Mnih et al., 2015; Wurman et al., 2022).

Besides their discrepancies in sampling efficiency, several further advantages and disadvantages of both methods are nearly orthogonal (Görges, 2017), i.e., weaknesses of either approach are strengths of the other. For instance, RL struggles with safety issues (Brunke et al., 2022), whereas MPC can guarantee constraint satisfaction related to a particular environment model (Rawlings et al., 2017). The orthogonal strengths of both approaches motivated many authors to combine the advantages and synthesize novel algorithms that build on both methods. These combinations led to safety (Gros, Zanon, & Bemporad, 2020; Reiter et al., 2023; Wabersich & Zeilinger, 2018; Yang et al., 2020) and stability (Didier et al., 2024) guarantees, improved sample efficiency (Hansen et al., 2022; Karnchanachari et al., 2020; Romero et al., 2024; Yang et al., 2020), reduced training time (Hoeller et al., 2020; Karnchanachari et al., 2020), adaptability (Yang et al., 2020), and generalization (Romero et al., 2024) when compared to RL. Moreover, when compared to MPC, combinations achieved faster online computation time (Brito et al., 2022, 2021; Reiter et al., 2025; Rickenbach et al., 2025), and the possibility to optimize closed-loop performance without advanced models (Gros & Zanon, 2020a).

Given the success of existing literature in combining MPC and RL, this work aims to contribute the following. We first analyze the properties and orthogonal strengths of MPC and RL in a unified language.

Secondly, we propose a novel systematic classification of how MPC can be combined with RL, considering key distinctions such as the role of MPC in an RL framework, architectures of how highly nonlinear NNs are used within the optimization framework, and how existing theoretical work connects to the proposed categories. Finally, we provide the first comprehensive literature review focusing specifically on RL and MPC combinations, guided by the proposed classification. This survey reviews both practical and theoretical work, concluding with a section on available open-source software.

1.1. Scope

The scope of the survey and classification is centered on formulations that use MPC as part of RL algorithms. We separate the concept of MPC, i.e., using a model and an objective to optimize a predicted trajectory, from the actual algorithm to solve the optimization problem, e.g., second-order, first-order, or sampling-based variants. There are multiple definitions for RL and vanishing boundaries between different adjacent areas, such as approximate dynamic programming. In this work, we view RL as a collection of algorithms that adapt a policy through environment interactions towards closed-loop optimality, according to concepts from Bertsekas and Tsitsiklis (1996), Sutton and Barto (2018).

1.2. Related work

Several works shed light on various aspects of the substantial research fields of control systems, machine learning, and optimization. The author of Görges (2017) presents relations between MPC and RL at a conceptual level and provides a detailed discussion of discrete-time linear time-invariant constrained systems with quadratic costs. RL is contextualized from the perspective of control systems in Recht (2019) where MPC is mentioned briefly as one particular control technique. The authors of Brunke et al. (2022) survey methods for safe learning in robotics and also consider the role of MPC. In particular, the authors demonstrate how parameterized MPC formulations can be utilized to guarantee safety during learning and how safety can be integrated into RL in general. However, these surveys focus on comparisons rather than on combining MPC and RL.

The survey of Bengio et al. (2021) provides an extensive overview of how machine learning is combined with combinatorial optimization. In combinatorial optimization, determining the relevant discrete decision variables is a pivotal challenge addressed by machine learning techniques. However, similarities can be observed in how NNs are integrated into an optimization problem.

The author in Bertsekas (2019) compares RL and MPC from the perspective of approximate dynamic programming and proposes a unified mathematical framework. In Bertsekas (2022a, 2022b), the RL-based program AlphaZero (Silver et al., 2017), which plays games such as Chess, Shogi, or Go, is cast within an MPC framework. While a classification of MPC and RL combinations is the main scope of the authors, the results still reveal rigorous insights into their fundamental relation.

The high-level survey in Hewing, Wabersich, et al. (2020) focuses on how general machine learning concepts are applied within the MPC framework to guarantee safety. The authors of Mesbah et al. (2022) focus on investigating general machine learning used as part of MPC and provide a categorization into (i) learning of the model and uncertainty, (ii) using closed-loop data for tuning, (iii) learning a terminal value function, and (iv) approximations of the optimization algorithm. While the categories do not contradict our categorization, we propose a broader, deeper classification specifically for RL and MPC synthesis, with the elements of Mesbah et al. (2022) as subcategories. Both surveys (Hewing, Wabersich, et al., 2020) and Mesbah et al. (2022) focus on a broader class of machine learning concepts, and RL is a minor topic.

The authors of Moerland et al. (2023) classify model-based RL algorithms, which cover general planning but not specifically MPC formulations. Since MPC can be seen as a specific variant of planning with a model, several concepts of Moerland et al. (2023) apply to MPC as well. However, since MPC is more specific than general planning, our survey proposes an alternative categorization tailored for combinations of MPC with RL. The related categorizations in adjacent fields are discussed in more detail in Section 12.

Further work compares MPC and RL for specific applications. Similar to Mesbah et al. (2022), the authors in Norouzi et al. (2023) compare the combination of MPC and machine learning specifically for automotive applications. Finally, the authors in Zhang et al. (2022b) compare RL and MPC applied to energy management in buildings with a focus on data-driven MPC methods.

With the proposed categorization, our framework enables the categorization of a large corpus of specific RL literature that was previously impossible, providing in-depth insights into algorithms, implementations, and available software. Our proposed framework can be seen as a complementary work to Bertsekas (2019), Moerland et al. (2023) and Mesbah et al. (2022).

1.3. Notation

The MPC and RL literature uses different symbols and notations for identical objects, as shown in the comparison (Bertsekas, 2019). RL notation is used predominantly, with exceptions to maintain established terminology of the control literature, as shown in Table 1. For instance, we primarily use “environment” instead of “plant” or “system” but may occasionally write “system” when it is more common in the context. Note that we exclusively use the term MDP that also refers to the conceptually equivalent term discrete-time stochastic optimal control.

Within the field of RL notation, ambiguities of the notation exist. In this survey, we primarily use RL to refer to model-free RL algorithms and include imitation learning (IL) due to their conceptual overlap. MPC that learns a model online is often seen as a variant of model-based RL (Wang et al., 2019). Since the proposed synthesis approaches are not limited to model learning, and, moreover, model-based RL encompasses approaches that learn a model without the intention of being used in an optimization problem, we omit a detailed discussion on model-based RL.

For the concatenation $z = [x^\top, y^\top]^\top$ of column vectors x and y , we write $z = (x, y)$. Given set $\mathcal{A}$ and set $\mathcal{B}$, we denote with $\mathcal{B}^{\mathcal{A}}$ the set of all functions that map from $\mathcal{A}$ to $\mathcal{B}$. Given a set $\mathcal{A}$, we denote with $\text{Dist}(\mathcal{A})$ the space of all probability distributions or probability measures over $\mathcal{A}$. Let p be a probability density of a probability distribution over $\mathcal{A}$, then the support is defined as $\text{supp}(p) := \{a \in \mathcal{A} \mid p(a) > 0\}$.

1.4. Overview

On a high level, this paper is structured into (1) an introductory part, (2) a comparison, and (3) a classification scheme and survey of synthesis approaches. An overview of the paper structure is provided in Fig. 1.

Introduction and background. The introductory part comprises Section 2, 3 and 4. Section 2 states the general problem setting, focusing on MDPs and optimal control. Sections 3 and 4 describe the main concepts behind RL and MPC and discuss how they aim at solving the general problem of Section 2. Both sections are divided into conceptual and algorithmic parts, providing the foundation for the remainder of this work.

Comparison. After explaining the basics of MPC and RL and their relation to the root problem of solving MDPs, we transition to the combination approaches by first highlighting the individual strengths of each approach in Section 5. The comparison highlights practical differences between the two approaches and shows their orthogonal benefits. Experts in the field of MPC and RL may skip Section 3, 4, and potentially the comparison in Section 5.

Table 1

Symbols used within this paper.

Name	Symbol
state	s
sampled state	S
planned states within MPC	x
action (control)	a
sampled action	A
planned actions within MPC	u
stochastic model	P
deterministic model approximation within MPC	f^{MPC}
stochastic policy (controller)	π
deterministic policy	μ
parameterized policy	π_θ, μ_θ
cost function	l
cost function approximation within MPC	l^{MPC}
optimal stochastic policy	π^*
value function under policy π	V^π
value function under policy π parameterized by θ	J^π
optimal value function	V^*
terminal value function approximation under policy π	$\bar{V}^\pi$
value function of MPC	V^{MPC}
value function of MPC parameterized by θ	V_θ^{MPC}
terminal value function within MPC	$\bar{V}^{\text{MPC}}$
terminal value function within MPC parameterized by θ	$\bar{V}_\theta^{\text{MPC}}$
action-value function under policy π	Q^π
optimal action-value function	Q^*
action-value function of MPC	Q^{MPC}
action-value function of MPC parameterized by θ	Q_θ^{MPC}
fixed sampled dataset	$\mathcal{D}$
data sampled under policy π	$\mathcal{D}^\pi$
data sampled under ϵ -greedy policy implicitly defined by Q	$\mathcal{D}^{\epsilon_Q}$
replay buffer with transitions stored during training	$\mathcal{D}^{\text{buffer}}$

Combination approaches. The remaining paramount sections are devoted to the classification and review of combinations of both approaches. In Section 6, essential concepts are introduced to categorize existing combination variants based on the actor-critic framework of RL. Following the proposed categorization, literature that uses MPC as an expert for training an RL agent is summarized in Section 7. The most widespread variants using MPC within the policy are outlined in Section 8, and variants that use the MPC to determine the value function at a particular state are surveyed in Section 9. The theory Section 10 focuses on significant theoretical results from the field of MPC and RL, aligned with the previous categorization. For practitioners, current open-source software frameworks are presented in Section 11. In the final Section 12, the work is concluded and discussed.

2. Problem setting

This section describes the Markov decision process (MDP) framework — a central concept for both RL and MPC.

MDPs provide a framework for modeling a discrete-time stochastic decision process. An MDP is defined over a state space S , an action space $\mathcal{A}$, and a stochastic transition model

$$P : S \times \mathcal{A} \rightarrow \text{Dist}(S) \quad (1)$$

describing a probability distribution over the next states given a current state and action. A stage cost $l(s, a)$ with $l : S \times \mathcal{A} \rightarrow \mathbb{R} \cup \{\infty\}$ defines the

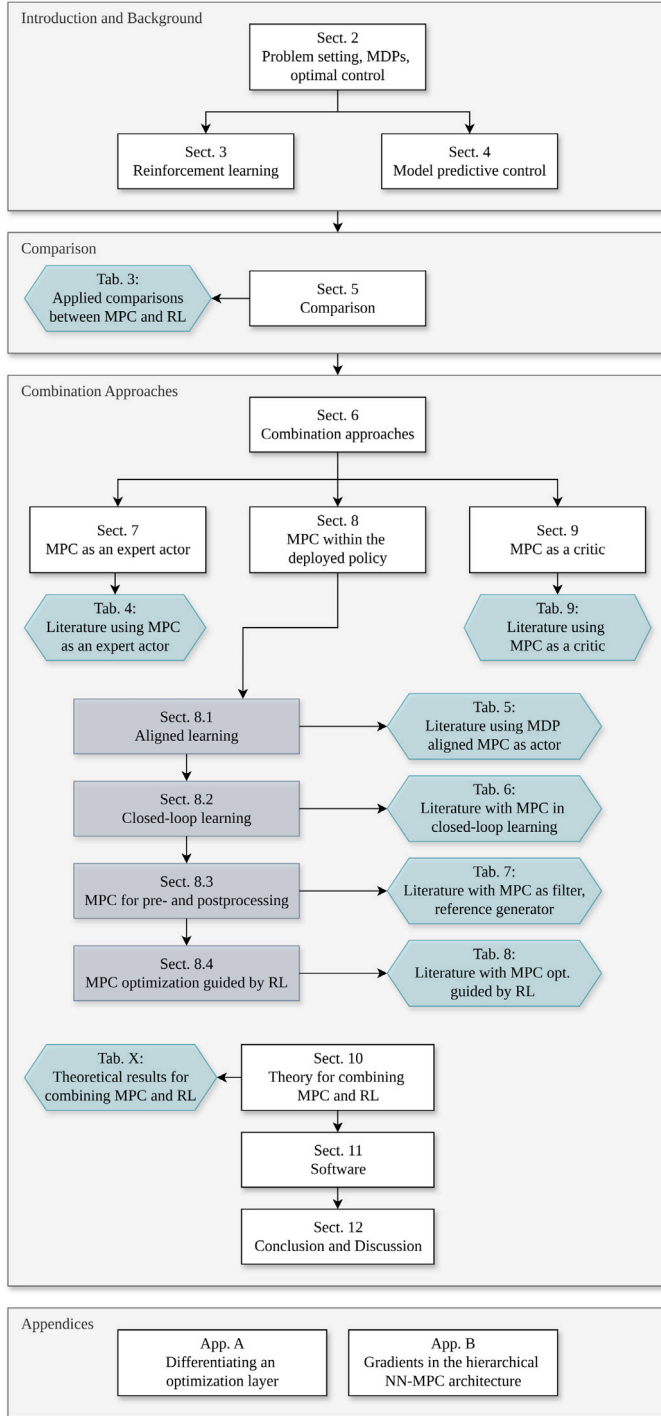


Fig. 1. Paper structure. The main sections are highlighted in white, subchapters in grey, and tables are green.

cost of each state–action pair, typically discounted by a factor $\gamma \in (0, 1]$. The MDP (Puterman, 2005) is then defined by the 5-tuple

$$\mathcal{M} := (S, \mathcal{A}, P, l, \gamma). \quad (2)$$

Solving the MDP refers to obtaining actions that minimize the discounted stage cost, which is elaborated in the following.

For solving MDPs, we introduce stochastic policies $\pi : S \rightarrow \text{Dist}(\mathcal{A})$ that map a state to a probability distribution over possible actions. The value function $V^\pi : S \rightarrow \mathbb{R} \cup \{\infty\}$ is the discounted expected future cost

starting from state s and following a policy π defined by

$$V^\pi(s) := \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k l(S_k, A_k) \right], \quad (3)$$

$$A_k \sim \pi(\cdot | S_k), \quad S_{k+1} \sim P(\cdot | S_k, A_k), \quad S_0 = s.$$

The state S_k and action A_k describe a sequence of random variables generated by applying the policy π on the MDP. Note that within this framework, a wide variety of model classes can be used as function approximators for the transition model, policy, and stage cost, including, e.g., linear and nonlinear models, Gaussian processes, decision trees, or NNs.

The aim of solving the MDP is to obtain an optimal policy π^* that minimizes the expected cost for all states via

$$\pi^* \in \bigcap_{s \in S} \arg \min_{\pi} V^\pi(s). \quad (4)$$

Solving this equation is often also shortly referred to as solving the MDP. Note that the intersection in (4) is never empty, and there also always exists an optimal deterministic policy μ^* (Sutton & Barto, 2018) satisfying (4). All optimal policies π^* share the same optimal value function $V^* := V^{\pi^*}$.

The action-value function $Q^\pi : S \times \mathcal{A} \rightarrow \mathbb{R} \cup \{\infty\}$, associated with a policy π , is the expected discounted future cost of applying the action a at the current state s and following the policy π afterwards, i.e.,

$$Q^\pi(s, a) := \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k l(S_k, A_k) \right], \quad (5)$$

$$A_k \sim \pi(\cdot | S_k), \quad S_{k+1} \sim P(\cdot | S_k, A_k), \quad S_0 = s, \quad A_0 = a.$$

We define the optimal value and action-value functions as $V^* := V^{\pi^*}$ and $Q^* := Q^{\pi^*}$. The identities

$$V^*(s) = \min_a Q^*(s, a), \quad (6a)$$

$$\text{supp}(\pi^*(\cdot | s)) \subseteq \arg \min_{a^*} Q^*(s, a^*), \quad (6b)$$

are relating the optimal value function and the optimal policy to the optimal action-value function. In cases with multiple optimal actions for a state s , an optimal policy π^* can be stochastic and randomly select an action in the set of optimal actions with any probability.

Remark 2.1. The optimal policy for a finite-horizon problem is generally time-varying unless the terminal cost is V^* . Time-varying environments can be described in terms of (1) by augmenting the state with an additional clock state.

In the following, we introduce MPC and RL, the two pivotal frameworks of this survey for approximately solving MDPs (4).

3. Reinforcement learning

RL is a powerful approach for solving MDPs using concepts from dynamic programming, Monte Carlo simulation, and stochastic approximation. RL typically involves an agent modeled by a policy that iteratively collects data by interacting with an environment, which could be a simulation model or the real world. The collected data, consisting of state transitions, applied actions, and costs, is then used to update the policy iteratively. In most RL methods, an optimal policy is approximated via estimating the optimal action-value function using temporal difference (TD) learning or by iteratively updating the policy using policy gradient (PG) methods (Sutton & Barto, 2018). In contrast, in actor–critic methods, both forms are combined.

3.1. Theoretical background

In this section, a short theoretical overview of RL is provided, including dynamic programming, temporal difference methods, and policy gradient methods.

3.1.1. Dynamic programming

Introduced in [Bellman \(1966\)](#), dynamic programming (DP) provides the theoretical foundation for many RL algorithms. DP solves MDPs with known transition models by breaking them into subproblems and using stored solutions to avoid recomputation. DP systematically updates value functions given complete knowledge of the environments MDP. A general DP approach to find the action-value function Q^π of a given policy π can be described by the Bellman operator $T^\pi : \mathbb{R}^{S \times A} \rightarrow \mathbb{R}^{S \times A}$,

$$T^\pi Q(s, a) := l(s, a) + \gamma \mathbb{E}_{\substack{S^+ \sim P(\cdot | s, a), \\ A^+ \sim \pi(\cdot | S^+)}} [Q(S^+, A^+)], \quad (7)$$

where S^+ is the next sampled state and A^+ is a sampled action. In the space of value functions, the Bellman operator T^π is a contraction mapping for $\gamma < 1$ with respect to the state supremum norm ([Bertsekas & Tsitsiklis, 1996](#)), and Q^π is the unique fixed point. In other words, one can start with an arbitrary action-value function $Q \in \mathbb{R}^{S \times A}$ and iteratively apply T^π to converge to the action-value function Q^π . Determining V^π and Q^π for a fixed policy π is referred to as policy evaluation.

Similar to the Bellman operator T^π , the Bellman optimality operator $T : \mathbb{R}^{S \times A} \rightarrow \mathbb{R}^{S \times A}$ is defined as

$$TQ(s, a) := l(s, a) + \gamma \min_{a^+} \mathbb{E}_{S^+ \sim P(\cdot | s, a)} [Q(S^+, a^+)]. \quad (8)$$

Equivalently to T^π , T is a contraction mapping for $\gamma < 1$. Iteratively applying T on an arbitrary action-value function $Q \in \mathbb{R}^{S \times A}$ converges to the optimal value function Q^* . The resulting method is called value iteration (VI) and, unlike policy evaluation, tries to solve MDPs of (4) implicitly by finding the optimal value function Q^* .

The main drawback of DP methods is the unfavorable scaling to high-dimensional or continuous state and action spaces, which is often referred to as the curse of dimensionality ([Bellman, 1966](#)). DP methods require complete knowledge of the environment model P , which is a fundamental limitation compared to more generic model-free RL algorithms. approximate DP (ADP) ([Powell, 2007](#)) addresses the curse of dimensionality by using different approximation strategies to extend classical DP, as discussed in the following in the context of RL.

3.1.2. TD methods

To avoid the scaling issues of DP, temporal difference (TD) methods ([Sutton & Barto, 2018](#)) introduce two extensions: Firstly, they learn the value functions V^π or Q^π for a policy π and their optimal versions V^* and Q^* with only transition samples utilizing stochastic approximation and without explicitly requiring a transition model P . Secondly, they use an adaptive exploration-exploitation strategy to determine which states are favorable for updating the value function.

In the following, different TD learning algorithms are presented. The simplest policy evaluation method is called TD(0), which estimates the value function V^π for a given policy π . Given a current value function V , an update for a given state S for V is defined by

TD(0) ($V \approx V^\pi$)

$$V(S) \leftarrow V(S) + \alpha \delta, \quad (9a)$$

$$\delta := l(S, A) + \gamma V(S^+) - V(S), \quad (9b)$$

$$\text{with } S \sim \mathcal{D}^\pi, A \sim \pi(\cdot | S), S^+ \sim P(\cdot | S, A), \quad (9c)$$

where $\leftarrow$ denotes overwriting the function V at S , $\alpha > 0$ denotes the learning rate, $A \sim \pi(\cdot | S)$ a sampled action from the policy π at state S and $S^+ \sim P(\cdot | S, A)$ the next state sampled from the stochastic model (1). Furthermore, δ is called the temporal difference, measuring the stochastic difference between the sampled target $l(S, A) + \gamma V(S^+)$ and the current estimate $V(S)$. Finally, the distribution $\mathcal{D}^\pi$ is a sampling or exploration strategy where the fixed policy π sequentially generates new states by interacting with the environment. For a more technical discussion, see [Remark 3.1](#).

Remark 3.1. The notations $S \sim \mathcal{D}^\pi$ or $(S, A) \sim \mathcal{D}^\pi$ are mathematically not well defined as distributions. Without further elaboration, given an initial state s , an episode is simply run iteratively by applying a policy π :

$$S_0 = s, A_0 \sim \pi(\cdot | S_0), S_1 \sim P(\cdot | S_0, A_0), A_1 \sim \pi(\cdot | S_1), \dots \quad (10)$$

After each time step, an update can then be performed by a TD method. We still use this notation for instructional purposes, especially to highlight which policy was used to generate states and actions.

Similar to DP, under some technical assumptions from stochastic approximation theory, the update scheme converges to the unique fixpoint V^π ([Bertsekas & Tsitsiklis, 1996](#)). For example, one assumption is that the learning rate α must decrease to zero over time. The update scheme of (9) can be extended to also learn Q^π .

Besides estimating the value function V^π or Q^π for given policies π , a primary target of RL algorithms is learning the optimal value functions V^* and Q^* . For instance, the SARSA algorithm ([Sutton & Barto, 2018](#)) is an RL method for approximating the optimal value function Q^* and given by the update rule

SARSA ($Q \approx Q^*$)

$$Q(S, A) \leftarrow Q(S, A) + \alpha \delta, \quad (11a)$$

$$\delta := l(S, A) + \gamma Q(S^+, A^+) - Q(S, A), \quad (11b)$$

$$\text{with } (S, A) \sim \mathcal{D}^{\pi_Q}, S^+ \sim P(\cdot | S, A), A^+ \sim \pi_Q^\epsilon(\cdot | S^+). \quad (11c)$$

The ϵ -greedy policy is implicitly defined by Q via

$$\pi_Q^\epsilon(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{if } a = \arg \min_{a^*} Q(s, a^*), \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{otherwise,} \end{cases} \quad (12)$$

where, for notational convenience, it is assumed that there is only one optimal action for a given state s . Note that π_Q^ϵ is used to sample the next action $A^+ \sim \pi_Q^\epsilon$ and to generate the state and action with $\mathcal{D}^{\pi_Q^\epsilon}$, where the update is performed.

Another variant for learning the optimal action-value function Q^* is the popular Q-learning method ([Watkins & Dayan, 1992](#)), which, instead of sampling an ϵ -greedy action like SARSA, takes the greedy action to build the temporal difference. More explicitly, the update rule is defined via

Q-learning ($Q \approx Q^*$)

$$Q(S, A) \leftarrow Q(S, A) + \alpha \delta, \quad (13a)$$

$$\delta := l(S, A) + \gamma \min_{a^*} Q(S^+, a^*) - Q(S, A), \quad (13b)$$

$$\text{with } (S, A) \sim \mathcal{D}^{\pi_Q^\epsilon}, S^+ \sim P(\cdot | S, A). \quad (13c)$$

Note that Q-learning still uses the ϵ -greedy exploration policy to sample $(S, A) \sim \mathcal{D}^{\pi_Q^\epsilon}$ in (13c), whereas the policy that is used for the temporal difference is the greedy policy. Thus, Q-learning is an off-policy method, whereas SARSA is an on-policy method, as the same ϵ -greedy policy is used for both exploration and the temporal-difference update. For a more elaborate discussion, see [Sutton and Barto \(2018\)](#).

As discussed, a key motivation for TD methods is that learning a value function over the entire state space can be intractable, a major drawback of DP. Importantly, typically only a fraction of the state space is encountered under a given policy or an optimal policy. Thus, TD methods often incorporate a trade-off between exploring the state space and exploiting current knowledge using strategies like the ϵ -greedy policy π_Q^ϵ . Yet, for discrete state spaces, convergence to Q^* is only guaranteed if each state and action is seen infinitely often ([Bertsekas & Tsitsiklis, 1996](#)).

3.1.3. Policy gradient methods

Different from the previous methods, policy gradient (PG) methods directly optimize a parameterized policy π_θ that can be used for discrete and continuous action spaces $\mathcal{A}$. Given a parameterized policy $\pi_\theta : S \rightarrow \text{Dist}(\mathcal{A})$ and an initial state distribution $\rho_0 \in \text{Dist}(S)$, the goal of PG methods is to find the optimal parameters θ^* that minimize the expected return

$$J^\pi(\theta) := \mathbb{E}_{S \sim \rho_0} [V^{\pi_\theta}(S)] \quad (14a)$$

$$= \frac{1}{1 - \gamma} \mathbb{E}_{S \sim \rho^{\pi_\theta}(S), A \sim \pi_\theta(\cdot|S)} [l(S, A)], \quad (14b)$$

where ρ^π is the discounted visitation frequency defined as follows. Given a policy π_θ , the environment model P , and an initial state s , let $p(s \rightarrow s^+, k, \pi_\theta)$ be the probability of reaching state s^+ at time step k by starting from state s following policy π_θ . The normalized discounted visitation frequency is defined by $\rho^{\pi_\theta}(s^+) := (1 - \gamma) \mathbb{E}_{S \sim \rho_0} [\sum_{k=1}^{\infty} \gamma^{k-1} p(S \rightarrow s^+, k, \pi_\theta)]$. It is important to highlight that finding a policy that minimizes the objective of (14) is less restrictive than solving the MDP over the full state space as defined in (4). The reasoning is that some parts of the state space might not be reached by the policy, which could be omitted if the support of the initial state distribution ρ_0 is required to cover the whole state space.

To find an update direction in which the expected return (14) improves, its gradient $\nabla_\theta J^\pi(\theta)$ is required. Different reformulations of (14) exist that allow one to derive sample estimates of the gradient of the PG objective, $\nabla_\theta J^\pi(\theta)$. First, the stochastic policy gradient theorem reformulates the policy gradient by

$$\nabla_\theta J^\pi(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{S \sim \rho^{\pi_\theta}(S), A \sim \pi_\theta(\cdot|S)} [Q^{\pi_\theta}(S, A) \nabla_\theta \log \pi_\theta(A | S)], \quad (15)$$

building the theoretical foundation of the REINFORCE algorithm (Williams, 1992) or the first actor-critic methods (Sutton & Barto, 2018). A derivation is provided in Sutton and Barto (2018).

The second reformulation is called the deterministic policy gradient theorem. Let $\mu_\theta : S \rightarrow \mathcal{A}$ be a deterministic policy. By differentiating through the expected state-action value function Q^{μ_θ} the deterministic PG (DPG) is obtained by

$$\nabla_\theta J^\pi(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{S \sim \rho^{\pi_\theta}(S)} \left[\nabla_\theta \mu_\theta(S) \nabla_a Q^{\mu_\theta}(S, a) \Big|_{a=\mu_\theta(S)} \right]. \quad (16)$$

A derivation is provided in Silver et al. (2014). The advantage of the DPG is particularly pronounced in high-dimensional action spaces, since in stochastic PG actions A are sampled to estimate the gradient (Silver et al., 2014), potentially leading to higher gradient variance. Thus, the DPG formulation is used in many of the state-of-the-art methods like twin-delayed actor-critic (TD3) (Fujimoto et al., 2018) and soft actor-critic (SAC) (Haarnoja et al., 2018). A concrete algorithm for an actor-critic algorithm using the DPG is provided in Section 3.2.2.

An important distinction between the stochastic PG and the DPG is the ability to handle discrete action spaces. With the stochastic PG, discrete and continuous action spaces can be optimized directly, whereas the DPG requires a differentiable parameterized policy. To circumvent this problem, some work extends the DPG to stochastic policies using relaxation techniques to differentiate through the sampling process of the discrete actions (Tilbury et al., 2023).

3.2. Deep reinforcement learning methods

In the following, an overview of four influential deep RL algorithms, deep Q-networks (DQN), deep deterministic PG (DDPG), proximal policy optimization (PPO), and soft actor-critic (SAC), is provided.

3.2.1. Deep Q-networks

Function approximators like NNs approximate the action-value function to extend Q-learning to continuous state spaces. One prominent implementation of this is DQN (Mnih et al., 2015), a combination of deep learning and RL. DQN collects transition samples in

a buffer $\mathcal{D}^{\text{buffer}}$ and minimizes the mean squared error between the current value function Q_w and the sampled target value by

$$\mathcal{L}_{\text{DQN}}^Q(w) := \mathbb{E}_{(S, A, S^+) \sim \mathcal{D}^{\text{buffer}}} \left[\left(l(S, A) + \gamma \min_{a^*} Q_{\bar{w}}(S^+, a^*) - Q_w(S, A) \right)^2 \right]. \quad (17)$$

For the sample target, a fixed copy $\bar{w}$ of the parameter w is used that is only periodically updated. This stabilizes the training (Mnih et al., 2015). For an overview of different function approximation methods and their potential instabilities, see Sutton and Barto (2018).

Given a parameterized Q-function Q_w , the resulting update scheme from DQN is given by

DQN ($Q_w \approx Q^*$)

$$w \leftarrow w + \frac{\alpha_w}{B} \sum_{i=1}^B \delta_i \nabla_w Q_w(S_i, A_i), \quad (18a)$$

$$\delta_i := l(S_i, A_i) + \gamma \min_{a^*} Q_{\bar{w}}(S_i^+, a^*) - Q_w(S_i, A_i), \quad (18b)$$

$$\text{with } (S_i, A_i, S_i^+) \sim \mathcal{D}^{\text{buffer}}. \quad (18c)$$

The update rule in (18a) provides a sample-based estimate of the gradient in (17), where B represents the batch size used for the update and α_w is a learning rate. Averaging over multiple samples, often referred to as mini-batch training, leads to improved performance and accelerated convergence when training NNs (LeCun et al., 1998). Exploration in DQN is handled again by the ϵ -greedy policy π_ϵ^Q . Different from the update scheme of Q-learning (13), the buffer $\mathcal{D}^{\text{buffer}}$ stores state and actions that were generated from previous policies derived from Q_w earlier in the training. Like Q-learning, DQN is an off-policy method.

3.2.2. Deep deterministic policy gradient

Extending DQN, deep deterministic PG (DDPG) (Lillicrap et al., 2016) is an off-policy actor-critic method that learns a deterministic policy μ_θ and a value function Q_w in parallel. Both the actor and the critic are NNs. The update rule of DDPG is given by

DDPG ($\mu_\theta \approx \pi^*$)

$$w \stackrel{Q}{\leftarrow} w + \frac{\alpha_w}{B} \sum_{i=1}^B \delta_i \nabla_w Q_w(S_i, A_i), \quad (19a)$$

$$\theta \stackrel{\mu}{\leftarrow} \theta + \frac{\alpha_\theta}{B} \sum_{i=1}^B \nabla_\theta \mu_\theta(S_i) \nabla_a Q_w(S_i, a) \Big|_{a=\mu_\theta(S_i)}, \quad (19b)$$

$$\delta_i := l(S_i, A_i) + \gamma Q_{\bar{w}}(S_i^+, A_i^+) - Q_w(S_i, A_i), \quad (19c)$$

$$\text{with } (S_i, A_i, S_i^+) \sim \mathcal{D}^{\text{buffer}}, A_i^+ = \mu_\theta(S_i^+), \quad (19d)$$

where $\stackrel{Q}{\leftarrow}$ denotes the update for Q and $\stackrel{\mu}{\leftarrow}$ denotes the update for μ . As can be seen from the update scheme, the critic Q_w is used to update the actor μ_θ in (19b), in that sense “criticizing” the actor. Unlike the update rule of DQN (18a), where the greedy action is considered to build the temporal difference, in DDPG, a single evaluation update with respect to the current policy μ_θ is performed. The buffer $\mathcal{D}^{\text{buffer}}$ is filled up over time by using the policy $\mu_\theta + \xi$, where ξ is either an Ornstein–Uhlenbeck process for temporally correlated noise or a Gaussian (Lillicrap et al., 2016).

3.2.3. Proximal-policy optimization

A popular on-policy actor-critic method suitable for both discrete and continuous action spaces is proximal policy optimization (PPO) (Schulman et al., 2017). Before each policy and critic update, data is collected in the form of multiple episodes. Since PPO is an on-policy method, transitions generated earlier in the training by outdated

policies are discarded. In practice, PPO is often used in high-speed simulation environments, where generating new samples incurs low computational costs. A primary advantage of PPO is its ability to prevent drastic parameter updates that could potentially destabilize the training. Similar to trust region methods (Schulman et al., 2015), this is achieved by restricting the policy update.

During training, a stochastic policy π_θ – often a parameterized Gaussian – is used. Assuming a given initial state s_0 , a trajectory is drawn by the forward simulation $S_{k+1} \sim P(\cdot | S_k, A_k)$ and $A_k \sim \pi_\theta(\cdot | S_k)$ until a maximum roll-out length M . Given multiple roll-outs, an estimate $\hat{A}(S_k, A_k)$ of the advantage function defined by $A^\pi(S_k, A_k) := Q^\pi(S_k, A_k) - V^\pi(S_k)$ can be derived, see Schulman et al. (2016). Additionally, with the probability ratio $R_k(\theta) := \pi_\theta(A_k | S_k) / \pi_{\bar{\theta}}(A_k | S_k)$, which measures how much the new policy π_θ changes with respect to the current policy $\pi_{\bar{\theta}}$, the PPO clipping objective is defined by

$$J_{\text{CLIP}}^\pi(\theta) := \mathbb{E} \left[\sum_{k=0}^{M-1} \max \left\{ R_k(\theta) \hat{A}(S_k, A_k), \right. \right. \\ \left. \left. \text{clip}(R_k(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}(S_k, A_k) \right\} \right],$$

where the clip function projects the ratio R_k to an interval from $1 - \epsilon$ to $1 + \epsilon$.

One of the primary advantages of PPO is its simplicity and ease of implementation (Schulman et al., 2017), compared to previous methods based on trust-region optimization (Schulman et al., 2015). A proof of convergence of PPO to the optimal policy for discrete MDPs is given in Grudzien et al. (2022).

3.2.4. Soft actor–critic

The SAC algorithm (Haarnoja et al., 2018, 2019) is a widely used off-policy actor–critic method that incorporates an entropy bonus for stochastic policies with higher entropy. Using entropy regularization is considered in the framework of maximum-entropy RL (Ziebart et al., 2008). Like DQN and DDPG, SAC optimizes the policy in an off-policy manner, collecting transitions encountered during training in a replay buffer, resulting in improved sample efficiency compared to PPO.

SAC extends the objective (14) by introducing an entropy regularization term, leading to a “soft” policy gradient objective

$$J_{\text{soft}}^\pi(\theta) := \frac{1}{1 - \gamma} \mathbb{E}_{S \sim \rho^\pi, A \sim \pi_\theta(\cdot | S)} [l(S, A) + \lambda_H H(\pi_\theta(\cdot | S))],$$

where H denotes the entropy of the parameterized policy $\pi_\theta(\cdot | S)$ at the sampled state S . Given a distribution X and its density p_X , the entropy is defined by $H(X) = \mathbb{E}[-\log(p_X(X))]$. The entropy regularization, scaled by the parameter λ_H , encourages exploration, stabilizes policy training (Ball & Roberts, 2021), and can lead to more robust policies (Eysenbach & Levine, 2022). Additionally, it has been shown in Müller and Cayci (2024) that for discrete MDPs, the error introduced by the entropy regularization decreases exponentially to the inverse regularization strength, $1/\lambda_H$.

In TD3, twin Q-networks were introduced to address the overestimation bias in Q-value estimation (Fujimoto et al., 2018). Whereas in Haarnoja et al. (2018), the fundamentals of SAC are described, Haarnoja et al. (2019) introduced an improved version using twin Q-networks and automatic tuning of the entropy regularization with λ_H . The latter builds the basis for most implementations in current RL software frameworks.

4. Model predictive control

This section introduces MPC, a commonly used framework to obtain policies for continuous MDPs. MPC utilizes a – typically deterministic – model that approximates the true stochastic environment (Grüne & Pannek, 2017; Rawlings et al., 2017). Starting from the current environment state, MPC uses the internal model to predict how different

choices of the planned control trajectory would affect the state trajectory and evaluates the cost associated with this prediction. Usually, evaluating the MPC policy involves solving an optimization problem online to obtain the control input. In this section, we first provide an overview of MPC problem formulations and relevant considerations, followed by a discussion of the algorithms used to solve them.

4.1. MPC problem formulations

Solving the MDP optimization problem (4) is, in general, intractable due to several reasons, including the infinite horizon, the optimization over the space of policy functions, and the expectation over nonlinear transformations of stochastic variables. MPC leverages several approximations of (4) in order to derive a computationally tractable optimization problem.

As a first step, the optimal policy is computed only for the current state s and the infinite horizon in (3) and (4) is approximated by a finite horizon, resulting in the optimization problem

$$\min_{\pi} \mathbb{E} \left[\gamma^N \bar{V}^\pi(S_N) + \sum_{k=0}^{N-1} \gamma^k l(S_k, A_k) \right], \quad (20)$$

with $S_0 = s$, $S_{k+1} \sim P(\cdot | S_k, A_k)$, $A_k \sim \pi(\cdot | S_k)$,

where the terminal cost function $\bar{V}^\pi$ is an approximation of the exact value (or cost-to-go) function V^π . In the second step, the true stochastic state transition $S_{k+1} \sim P(S_k, A_k)$ is approximated by a simplified model. The most commonly used formulation is nominal MPC, in which a deterministic model is used, i.e., $x_{k+1} = f^{\text{MPC}}(x_k, u_k)$. Hence, uncertainty is not explicitly accounted for. Here, we introduced $x_k \in S$ to denote predictions of the state within the MPC problem. Since a deterministic model does not capture the possibility of deviations from the prediction, the planned action trajectory is a trajectory of fixed actions $(u_0, \dots, u_{N-1})$, $u_k \in \mathcal{A}$, as opposed to a policy function π . The resulting deterministic optimal control problem is given by

$$\min_{u_0, \dots, u_{N-1}} \gamma^N \bar{V}^*(x_N) + \sum_{k=0}^{N-1} \gamma^k l(x_k, u_k), \text{ with } x_0 = s, x_{k+1} = f^{\text{MPC}}(x_k, u_k), \quad (21)$$

where $\bar{V}^*$ is an approximation of the optimal terminal value function.

In the MDP framework (2), the stage cost $l(x_k, u_k)$ may assign some regions of the state space an infinite cost to prohibit them. Similarly, due to actuator limitations, the action space $\mathcal{A}$ is often a compact subset of $\mathbb{R}^{n_u}$. In numerical optimization, this is typically handled by explicitly considering constraints $h^{\text{MPC}}(x_k, u_k)$ and the terminal safe set $h_N^{\text{MPC}}(x_k, u_k)$ as part of the problem formulation. This results in a constrained nonlinear program (NLP), associated with the MPC value function $V^{\text{MPC}}(s)$ and the terminal value function $\bar{V}^{\text{MPC}}(s)$ within the NLP formulation, and can be stated as

$$V^{\text{MPC}}(s) = \min_z \bar{V}^{\text{MPC}}(x_N) + \sum_{k=0}^{N-1} l^{\text{MPC}}(x_k, u_k) \quad (22a)$$

$$\text{s.t. } x_0 = s, \quad (22b)$$

$$x_{k+1} = f^{\text{MPC}}(x_k, u_k), 0 \leq k < N, \quad (22c)$$

$$0 \leq h^{\text{MPC}}(x_k, u_k), 1 \leq k < N, \quad (22d)$$

$$0 \leq h_N^{\text{MPC}}(x_N), \quad (22e)$$

using the vector of decision variables $z = (x_0, \dots, x_N, u_0, \dots, u_{N-1}) \in \mathbb{R}^{n_z}$. In the above formulation, we introduced the state trajectory $(x_0, \dots, x_N)$ as additional decision variables, which are constrained to start at the given value of the current state (22b), and to follow the system dynamics (22c). This is in contrast to formulation (21), where the state trajectory is considered as an explicit function of the action trajectory. Both approaches are equivalent in terms of the solutions they admit. However, the iterations of numerical optimization

algorithms may differ depending on the formulation. The formulation in (21) is referred to as a single shooting or sequential formulation, whereas (22) is a multiple shooting or simultaneous formulation. For nonlinear unstable systems, the latter is typically preferable.

When deploying MPC (22), the NLP is solved online at every discrete time instant, based on a specified value of the current state s . This yields an optimal trajectory of actions, $(u_0^*, \dots, u_{N-1}^*)$, of which only the first one, u_0^* , is applied to the environment. The resulting new state is then used as the initial state for the following optimization problem, and a new input trajectory is computed. Hence MPC (22) defines a policy.

Similarly to the definition of the value function in (22), we can define the corresponding Q-function by additionally fixing the initial action vector u_0 to the given value,

$$Q^{\text{MPC}}(s, a) = \min_z \sum_{k=0}^{N-1} l^{\text{MPC}}(x_k, u_k) + \bar{V}^{\text{MPC}}(x_N) \quad (23a)$$

$$\text{s.t.} \quad x_0 = s, \quad (23b)$$

$$u_0 = a, \quad (23c)$$

$$x_{k+1} = f^{\text{MPC}}(x_k, u_k), \quad 0 \leq k < N, \quad (23d)$$

$$0 \leq h^{\text{MPC}}(x_k, u_k), \quad 1 \leq k < N, \quad (23e)$$

$$0 \leq h_N^{\text{MPC}}(x_N). \quad (23f)$$

Based on this Q-function and assuming a unique minimizer, the MPC policy is

$$\mu^{\text{MPC}}(s) = \arg \min_a Q^{\text{MPC}}(s, a), \quad (24)$$

where μ instead of π is used to denote that the MPC policy is deterministic.

We now take a closer look at the components of the MPC problem (22), followed by various other relevant considerations.

4.1.1. Dynamics model

The central component is the dynamics constraint (22c), in which $f^{\text{MPC}}(x, u)$ is a model of the environment (1). This model can be derived from first principles or identified from data. In the case of nominal MPC, it is deterministic and does not consider the stochasticity of (1).

4.1.2. Objective

The stage cost $l^{\text{MPC}}(x, u)$ in the MPC objective (22a) commonly corresponds to the stage cost of the MDP (2). Note that the stage cost may be implicitly time-varying by including an augmented clock state. Thus, it may implicitly include a discounting factor γ . Crucially, the MPC optimization problem must be numerically tractable, and the ultimate goal is to provide a policy that achieves sufficient closed-loop performance when applied in the real environment. Therefore, the stage cost may also be approximated by a function with favorable numerical properties. Often, the stage cost imposes a convex penalty on the deviation of the action and state trajectories from a reference point or trajectory. This is referred to as regulatory MPC or tracking MPC. The focus in regulatory MPC problems is primarily on stabilizing systems. Historically, the ability of MPC to incorporate inequality constraints and to handle multiple inputs simultaneously justified the additional computational complexity. In contrast, economic MPC uses a cost that directly expresses the quantity of interest to be optimized, e.g., time, energy use, financial cost, or yield of a production process. Therefore, economic MPC is closely related to solving MDPs beyond stabilization.

The terminal cost $\bar{V}^{\text{MPC}}(x)$ should ideally capture the cost-to-go at the end of the MPC horizon, cf. (20). The choice of horizon length and terminal cost is often crucial for the performance of the resulting MPC policy. Indeed, a close-to-optimal cost-to-go approximation via the terminal cost allows for shorter prediction horizons. The approximation quality becomes less important as the horizon length grows, and $\bar{V}^{\text{MPC}}(x_N) \equiv 0$ is a widely used choice (Grüne, 2013). However,

the NLP (22) becomes computationally more expensive for longer horizons. Thus, the horizon length is typically limited by the available computational resources. In practice, the terminal cost function is often chosen heuristically or based on stability considerations. Still, there is also research on how to explicitly select it as an approximation of the infinite-horizon cost (with respect to the MPC cost), e.g., by simulating forward a pre-selected simple feedback law (Diehl et al., 2003, 2004; Nicolao et al., 1996a, 1998). When stabilizing a system at a steady state using a locally smooth convex stage cost, a straightforward choice is the infinite horizon linear quadratic regulator (LQR) (Anderson & Moore, 1990) cost computed for the system resulting from the linearization of the model (22c) at that steady state.

4.1.3. Constraints

The final components are the stage and terminal constraints. Stage constraints (22d) can be used to avoid prohibited regions of the state space and to take into account actuator constraints. Terminal constraints (22e) can be used to ensure stability and recursive feasibility of the resulting MPC policy. For terminal constraints, considerations similar to those for the terminal cost apply. In principle, they should capture the system's future behavior over the infinite horizon and ensure that the MPC plan is not overly short-sighted regarding the stability and recursive feasibility of the resulting MPC policy.

Constraints can lead to situations in which (22) is infeasible for the current initial state value s , i.e., there exists no value for the decision variables such that all constraints are satisfied. Consequently, the solver would not be able to return a solution, and the MPC policy evaluation would fail. Thus, for practical MPC implementations, it can often be helpful not to enforce the constraints strictly but to penalize their violation. For exact penalties with sufficiently high penalty weight, constraint satisfaction is guaranteed if the original problem is feasible (Kerrigan, 2000; Nocedal & Wright, 2006). Otherwise, a solution that minimizes the constraint violation is returned.

4.1.4. Uncertainty-aware MPC

In contrast to the deterministic model in (22c), stochastic and robust MPC formulations (Kouvaritakis & Cannon, 2015; Mesbah, 2016; Rawlings & Risbeck, 2017) explicitly take into account the uncertainty of the model prediction. The former models uncertainty as a probability distribution, whereas the latter predicts bounded sets of all possible realizations (with respective tractable outer approximations). Both can be separated into scenario (Calafiore & Campi, 2006; Shapiro et al., 2009) and tube (Langson et al., 2004; Mayne et al., 2011) approaches. Scenario approaches consider discrete distributions, which may be obtained by sampling from a continuous distribution. This can take the form of sampling several disturbance trajectories separately (Nagabandi et al., 2018) and planning the corresponding state trajectories in parallel or of constructing a tree of scenarios that are branched at every time step (Kouzoipis et al., 2019). Tube approaches predict parameterized approximations of the state distribution with respect to uncertainty set trajectories. Typical parametrizations are, e.g., normal distributions, ellipsoids (Houska, 2011; Kurzhanski & Valyi, 1997; Villanueva et al., 2017) or polytopes (Raković, 2019; Rakovic et al., 2012; Villanueva et al., 2024). This allows for a finite-dimensional representation of the uncertainty, yielding a tractable NLP.

Both scenario and tube approaches can also be classified into open-loop and closed-loop formulations. Open-loop formulations plan only one fixed trajectory of actions. This can quickly lead to unrealistically conservative uncertainty predictions because they do not account for the fact that noise will be counteracted in the real environment by feedback (Ben-Tal et al., 2004). Closed-loop predictions account for future feedback, leading to more accurate and realistic predictions. Using scenario trees, this can be achieved by planning a distinct action for every tree node. This implicitly corresponds to planning over policies in the discretized disturbance space, since the actions depend on past disturbances. Here, nonanticipativity with respect to the

causality of the policy should be carefully considered. Closed-loop tube approaches usually explicitly consider parameterized simple feedback laws, e.g., linear feedback, that react to state deviations from the tube center. These feedback laws can be precomputed (Mayne et al., 2011; Parisini & Zoppoli, 1995) or optimized (Messerer & Diehl, 2021; Nagy & Braatz, 2004; Villanueva et al., 2024). While the optimization of state feedback gains is highly nonconvex even for linear systems, the optimization over affine disturbance feedback yields equivalent convex, but also higher-dimensional, optimization problems (Ben-Tal et al., 2004; Goulart et al., 2006). In the context of polyhedral sets, it is, under some assumptions, sufficient to consider only their vertices, which results in tree-structured formulations (Diehl, 2007; Scokaert & Mayne, 1998).

4.1.5. Optimization problem classification

Depending on the mathematical form of the functions in (22), the optimization problem can be classified differently. This is relevant because it informs both the choice of solution algorithm and the theory of the resulting policy. In linear MPC (LMPC), the model is linear, the constraint functions are affine, and the cost functions are convex quadratic, resulting in a quadratic program (QP). LMPC problems can be solved reliably and efficiently. This is often used in contrast to nonlinear model predictive control (NMPC), where the model is typically nonlinear, and the resulting optimization problem is an NLP (22). When the action space is discrete, this corresponds to an additional restriction of the action variables to the space of integers, resulting in a mixed-integer NLP (MINLP) or mixed-integer QP (MIQP) (Reiter et al., 2021). If the dynamics contain nonsmooth or discontinuous events, such as contact physics, this leads to mathematical programs with complementarity constraints (MPCCs) (Nurkanović, 2023).

4.1.6. Suboptimality and control theory

As discussed, the MPC problem (22) leverages several forms of approximations of the MDP (4). This raises the question of how the resulting MPC policy behaves with respect to the MDP or when applied to a real system. Since it solves the MDP only approximately, it can be considered a form of suboptimal control. An overview of several sources of suboptimality can be found in Bertsekas (2005). The suboptimality from the finite horizon approximation is analyzed in Grüne and Rantzer (2008), Karapetyan et al. (2023), Li et al. (2023). The consequences of approximating the expected value via sampling are addressed in the stochastic programming literature in the context of the sample average approximation (Shapiro et al., 2009). In Hadjiyiannis et al. (2011), the authors analyze the suboptimality resulting from affine feedback parameterization in a robust problem formulation. The suboptimality of nominal MPC in a stochastic environment is analyzed in Fleming (1971), Messerer et al. (2024), and regret bounds in Lin et al. (2022).

The control theory literature often asks a different, though closely related, question, see, e.g., Grüne and Pannek (2017), Rawlings et al. (2017). It investigates under which conditions the MPC policy exhibits desirable behavior. This includes system theoretical properties of the resulting closed-loop system, such as stability (Chen & Allgöwer, 1998; Limon et al., 2009; Mayne et al., 2000), and properties like recursive feasibility (Grüne, 2012; Mayne, 2013), i.e., the controller should not maneuver itself into a state in which the MPC problem (22) becomes infeasible. MPC should also be able to work under model-plant mismatch and reject disturbances. This is referred to as inherent robustness, and the theory covers nominal MPC, which uses no uncertainty model (Magni & Sepulchre, 1997; McAllister & Rawlings, 2022; Nicolao et al., 1996b; Scokaert et al., 1997), but can also be extended to stochastic MPC (McAllister & Rawlings, 2022), even if it is designed with respect to a wrong disturbance model (McAllister & Rawlings, 2024). Additionally, fast-paced applications may only allow for a suboptimal solution to (22) in the assigned computational budget. Stability (Diehl et al., 2005; Scokaert et al., 1999) and inherent robustness results (Allan et al., 2017; Pannocchia et al., 2011) also exist for this case. Further, while stability results are usually derived for continuous action spaces, they can also be generalized to discrete actuators (Rawlings & Risbeck, 2017).

4.2. Numerical methods for MPC problems

We distinguish two common and fundamentally different approaches to – possibly approximately – solving finite-horizon optimal control problem (OCP) formulations: Sampling-based methods and methods leveraging derivative-based numerical optimization. In the following, we briefly introduce and discuss both approaches, focusing on algorithms that address the nominal OCP formulation.

4.2.1. Sampling-based methods

As a first class of methods, we consider sampling-based approaches that aim at finding an approximate solution to the stochastic open-loop or nominal OCP by sampling control trajectories and evaluating the associated cost via forward simulation.

The simplest sampling-based method, also known as random shooting and used, e.g., in Nagabandi et al. (2018), Piovesan and Tanner (2009), considers a finite number of independently sampled action sequences and the corresponding state trajectories, which are obtained via forward simulation. The algorithm then chooses the open-loop action trajectory associated with the lowest cost as an approximate solution.

A second, more sophisticated, sampling-based approach is the cross-entropy method (CEM) (Rubinstein, 1997), where the probability distribution generating the open-loop action trajectory samples is iteratively refined based on previously sampled actions yielding low costs. In particular, CEM samples a finite number of action sequences and evaluates their associated costs via forward simulation. The action sequences yielding the lowest cost trajectories are used to adapt the probability distribution from which new action samples are generated. Typically, a Gaussian distribution or a Gaussian mixture model is used, in which the mean and covariance are adapted at each step. The method has been successfully implemented for MPC in Chua et al. (2018), Kobilarov (2012).

As a third sampling approach, we consider model predictive path integral control (MPPI), which provides a framework (Kappen, 2005; Theodorou, 2015; Theodorou & Todorov, 2012; Williams et al., 2017) for solving MPC problems via trajectory sampling. In particular, the approach directly addresses the stochastic formulation without resorting to the nominal problem (22). As the method is derived based on a continuous-time model, we refer the interested reader to Williams et al. (2017) for an in-depth derivation. Crucially for this survey, the method poses some strong restrictions on the structure of the model and cost: First, the stochasticity of the dynamics needs to enter via the actions, i.e., $f_{\text{MPPI}}(s_k, a_k + w_k)$, where w_k is a random variable in the dimension of the actions. Secondly, the cost J^{MPC} is assumed to be separable in states and actions, and quadratic in actions, with a weighting matrix that is inversely proportional to the noise variance. Furthermore, the main theoretical results and optimality (in the limit of infinite samples) only hold if the dynamics are affine in controls and noise (Theodorou, 2015). Similar to CEM, MPPI adaptively updates the probability distribution from which action trajectories are sampled. MPPI uses a weighted average to update the mean of this distribution, where the weights are based on the associated costs. Since the underlying algorithm of MPPI requires limited implementation efforts, many papers use custom implementations (Goldfain et al., 2019). However, recently an efficient implementation as part of TorchRL (Bou et al., 2023) and a CUDA-based parallel computation framework was published (Vlahov et al., 2024).

Sampling-based approaches naturally allow for stochastic models (Chua et al., 2018; Kantas et al., 2009) and can thus directly tackle the stochastic open-loop OCP. Furthermore, sampling-based methods can be applied to problems with highly nonlinear, or even non-smooth, costs or dynamics as long as a simulator is available. While sampling-based optimization methods are typically straightforward to implement and benefit from parallelization, they scale poorly with the dimension of both the planning horizon and the dimension of the action space,

i.e., they severely suffer from the curse of dimensionality. Furthermore, state constraints can be addressed only via penalty reformulations or the rejection of infeasible samples. For highly constrained systems, the rejection method further increases the sampling complexity.

4.2.2. Derivative-based numerical optimization

As a second class of methods, we discuss derivative-based numerical optimization. On this account, approaches derived from a continuous-time OCP formulation, such as indirect methods, collocation, and pseudospectral methods, as well as numerical simulation methods, fall into this class but are outside the scope of this survey. We refer the interested reader to [Binder et al. \(2001\)](#), [Rao \(2009\)](#) for a survey on numerical methods starting from a continuous-time formulation and to [Rawlings et al. \(2017, Chap. 8\)](#) for a textbook overview of direct methods.

Assuming that all problem functions in (22) are sufficiently smooth, OCP (22), which is typically referred to as the multiple shooting formulation ([Bock & Plitt, 1984](#)), can directly be addressed with standard numerical methods for constrained nonlinear optimization. While the multiple shooting formulation (22) keeps both states and actions as optimization variables, one may alternatively eliminate the states from (22) via the equality constraints, yielding the single shooting formulation as introduced at the beginning of this survey and given in (21). The single shooting formulation typically results in dense subproblems with few optimization variables and can thus be efficiently solved by general-purpose nonlinear solvers. In the following, we focus on the multiple shooting problem given in (22) and on tailored numerical methods that address the particular problem structure arising from this formulation.

First, we distinguish between numerical methods that use first-order derivative information versus approaches that use second-order derivative information. First-order methods are less computationally complex but may require many more iterations. Second-order methods require fewer but computationally more complex iterations. Due to their low complexity, first-order methods have been considered for solving OCPs, particularly in the context of embedded applications ([Englert et al., 2019](#); [Jerez et al., 2014](#); [Kouvaritakis & Cannon, 2015](#); [Patrinos & Bemporad, 2013](#)). Furthermore, a tailored method for the scenario-based OCP formulations based on alternating direction method of multipliers (ADMM) has been developed in [Kang et al. \(2015\)](#).

Second-order methods include nonlinear interior point (IP) methods and sequential quadratic programming (SQP), two widely used classes of methods for numerical optimization. We will briefly discuss both of them in the following. For a more detailed overview of second-order numerical methods for optimal control, we refer to [Diehl et al. \(2009\)](#).

Nonlinear interior point methods tackle the non-smooth Karush-Kuhn-Tucker (KKT) conditions associated with the constrained nonlinear optimization problem by formulating an approximate, but smooth, root-finding problem parameterized by a homotopy parameter, which is iteratively lowered towards zero — in the limit, recovering the non-smooth optimality conditions. The intermediate root-finding problems are solved via Newton-type iterations. Nonlinear interior point methods tailored to the OCP structure are discussed in [Frey et al. \(2020\)](#), [Frison and Diehl \(2020\)](#), [Rao et al. \(1998\)](#), [Vanroye et al. \(2023\)](#).

Within the SQP framework, a sequence of quadratic approximations of the nonlinear OCP is solved. The quadratic subproblems arising in SQP might, in turn, be solved using an interior point method or an active-set solver. SQP methods can typically be warm-started, rendering them particularly attractive for MPC applications where the solutions to subsequent problem instances are expected to be similar. For further implementation details tailored to SQP methods for MPC, such as full and partial condensing, we refer to the survey in [Kouzoupis et al. \(2018\)](#). SQP-type solvers tailored to OCP are implemented in [Houska et al. \(2011\)](#), [Verschuere et al. \(2022\)](#). A widely used approximate approach closely tied to SQP is the real time iteration (RTI) ([Bock et al.,](#)

[2007](#); [Diehl, 2002](#); [Gros et al., 2016](#)). Within the RTI framework, a single iteration of an SQP method is performed: a single QP approximation to the nonlinear OCP is solved to obtain an approximate solution that drastically reduces the computation time per iteration.

Both IP and SQP methods require the computation of (an approximation of) the Hessian of the Lagrangian associated with the OCP. While the exact Hessian yields locally quadratic convergence to the solution ([Nocedal & Wright, 2006](#)), its computation is typically costly. This motivates the use of Hessian approximations, e.g., the Gauss–Newton Hessian ([Bock, 1983](#); [Gauss, 1809](#)), which are cheaper to compute and yield convex subproblems.

Independent of the particular choice of Hessian approximation, the subproblems encountered in both IP and SQP methods applied to the multiple shooting formulation (22) exhibit a particular sparsity pattern, which tailored solvers typically exploit.

Another second-order method, which cannot be directly interpreted within the Newton-type framework, is differential dynamic programming (DDP), originally proposed in [Mayne \(1966\)](#). DDP is also based on solving QP subproblems via a Riccati recursion, followed by a nonlinear forward sweep of the system, which employs the linear feedback law returned by the Riccati recursion. The DDP variant using a Gauss–Newton Hessian approximation, which is more commonly referred to as iterative linear quadratic regulator (iLQR), especially within the robotics community ([Tassa et al., 2014](#)), has been introduced in [Li and Todorov \(2004\)](#), [Todorov and Li \(2005\)](#). In their standard form, DDP and iLQR cannot directly handle additional constraints, but extensions to OCPs with input bounds have been proposed, e.g., in [Marti-Saumell et al. \(2020\)](#), [Tassa et al. \(2014\)](#). sequential linear quadratic programming (SLQ) ([Sideris & Bodrow, 2005](#)) is often mentioned in the context of DDP. However, it can more precisely be classified as an SQP method with a Gauss–Newton Hessian approximation applied to the single shooting OCP (21) for which each of the QP subproblems is solved in a sparsity exploiting manner, i.e., by a Riccati recursion, cf. also [Baumgärtner et al. \(2023\)](#).

Crucially, all discussed methods will generally converge to a local optimum and thus require a sufficiently good initialization. The local rate of convergence is determined by the accuracy of the Jacobian and Hessian approximation ([Nocedal & Wright, 2006](#)).

Only in the particular case of a convex NLP, such as for LMPCs, convergence to a global optimum can be guaranteed. Under these assumptions, the solution map is continuous and piecewise affine. This fact is leveraged in explicit MPC, where the optimal feedback law is precomputed offline to minimize online computation ([Bemporad et al., 2000](#)). Explicit MPC is usually limited to small state dimensions, few inequality constraints, or short horizons. Explicit NMPC was investigated, e.g., in [Grancharova and Johansen \(2012\)](#).

In addition to algorithms and software focusing on nominal OCP formulations, numerical methods tailored to tree-structured problems arising in open-loop as well as closed-loop stochastic formulations have been developed ([Frison et al., 2017](#); [Klintberg et al., 2016](#); [Kouzoupis, 2019](#); [Kouzoupis et al., 2018](#); [Steinbach, 2002](#)). Numerical methods addressing the tube-based open-loop and closed-loop stochastic OCP formulation are presented in [Feng et al. \(2020\)](#), [Frey et al. \(2024\)](#), [Patil et al. \(2015\)](#), [Zanelli et al. \(2021\)](#) and [Goulart et al. \(2008\)](#), [Leeman et al. \(2024\)](#), [Messerer and Diehl \(2021\)](#), respectively.

In contrast to the exponential scaling of sampling-based approaches, e.g., shown for linear unstable systems in [Yoon et al. \(2022\)](#), solving the nonlinear OCP via numerical optimization alleviates the curse of dimensionality, as the computational complexity of the nominal problem typically scales linearly with the horizon and polynomially with the state and control dimension. On the other hand, the sub-problems need to be optimization-friendly and require access to derivative information. In comparison, sampling-based methods only require (fast) forward simulation.

Table 2

Comparison of practical properties between MPC and RL. The evaluation is simplified and conceptual. Exemptions may exist.

Property	MPC	RL
state-space	model specific ✗	(quite) arbitrary ✓
model requirements	differentiability/ online simulation ✗	offline simulation ✓
uncertainty	guarantees with known uncertainty ✓	probabilistic guarantees ✓
stability	strong theory ✓	minor theory ✗
constraint handling	inherent ✓	hard to achieve ✗
online computation time	high ✓	low ✗
offline computation time	low ✗	high ✓
adaptability	inherent ✓	needs retraining ✗
generalization	inherent ✓	poor when out-of-distribution ✗

5. Comparison of MPC and RL

Starting from the general problem of solving MDPs, we have shown how RL and MPC are different techniques that aim at deriving optimal policies, cf., Section 3 and Section 4, respectively. MPC, for instance, emerged from the problem of solving multivariable constrained control problems, initially with the goal of setpoint stabilization (Morari & Lee, 1999). In contrast, RL methods aim at maximizing closed-loop performance without necessarily relying on a model of the real environment. Their discrepancy is unsurprising since both approaches were developed in parallel communities with different focuses. This is further substantiated in the nearly orthogonal properties of MPC and RL, which were reviewed and emphasized in a case study for a specific linear system in Görges (2017). The more recent adaptations towards economic MPC, e.g., Angeli et al. (2012) fit the paradigm of solving MDPs and, hence, bring the goals of both communities closer together.

Considering the practicalities of MPC and RL of solving MDPs, further particularities appear, which we compare in the following. The comparison is concluded in Table 2, which provides an overview of relevant properties, similar to Görges (2017) and Table 3, from work that explicitly compares MPC and RL in practical applications. We compare requirements on the state-space representation and the properties of the mathematical model, such as smoothness or continuity required by MPC. We refer to Bertsekas (2005, 2019) for further theoretical comparisons.

The following presents the main practical and conceptual differences between MPC and RL.

5.1. State space

Real environments can only be approximately described by a state, which is usually not always directly measurable, leading to the concept of partially observable MDPs (POMDPs). So far, we have omitted a discussion of POMDPs and have only mentioned some points required for a high-level debate on the state space. Considering an environment where the state s is unknown and only observations $O_k \in \mathbb{R}^{n_o}$ are made at step k , the most general concept of a state space would involve the collection of all observations $O_0, O_1, \dots$ and applied actions $A_0, A_1, \dots$ up to the current time step. Even this very general concept may not allow to fully describe the real environment due to partial observability. However, often these observations, or even the M most recent observations $\mathcal{O}_M = (O_{k-M}, A_{k-M}, \dots, A_{k-1}, O_k)$ at step k are sufficient to estimate the relevant states of an environment (Bakker, 2001).

In MPC and RL algorithms, the state space is interpreted conceptually differently. In derivative-based MPC, the state space is usually constructed by relating it to an optimization-friendly model, usually with a physical interpretation based on differential equations. Yet, alternative approaches also consider a sequence of measurements as state (Coulson et al., 2019a), cf. Section 4.2. States used within derivative-based MPC do not necessarily correspond to the measured sensor outputs and,

thus, are often estimated by a state observer that converts a series of observations $\mathcal{O}_M$ into a state estimate.

RL methods and sampling-based MPC methods may use states based on a corresponding physics-based model and the related estimators (Brandi et al., 2022; Ceusters et al., 2021; Mamedov et al., 2024) or a state observer. However, the state may also be kept as a recent history of raw sensor data, which could be based on a variety of different input modalities, such as images or text. Often, end-to-end learning is used, where raw sensor inputs, such as images, are fed directly to NNs. The NN outputs subsequently the controls, e.g., as in Kaufmann et al. (2023). It is common to make the learned dynamics model or policy dependent on a window of the most recent history of observations and actions $\mathcal{O}_M$, cf., Puterman (2005), which is particularly useful for POMDPs. While an NN architecture does not explicitly model states, the stacking of layers, the related transformations, the exploitation of equi/invariances, and the condensing of information can be interpreted as modeling of hidden states, cf. Lambrechts et al. (2022). Subsequent layers can be interpreted as the policy based on these hidden states. Remarkably, this structure is not enforced explicitly. Also, recent approaches of the more classical observer/controller architecture propose to tune MPC and the state estimator together (Simpson et al., 2023).

5.2. Model and application

Highly related to the state space are the characteristics of a potential model. As explained in Section 4, MPC requires a model to simulate the dynamics. Whereas sampling-based MPC only requires forward simulation of a model, derivative-based MPC involves computing gradients through the model. Therefore, derivative-based MPC requires an optimization-friendly model, which imposes strict limitations on its structure, since the model is part of the optimization problem. The MPC optimization problem becomes particularly challenging if the model is non-smooth (Nurkanović, 2023), stochastic, or contains integer variables (Ceusters et al., 2021).

A physically motivated prediction model, which is often used in MPC, has the advantage of better understanding and explaining the environment behavior, usually referred to as explainability. The ability to predict interpretable model states enables straightforward constraint definition. For example, a particular velocity must not be exceeded in a vehicle control problem. This constraint can be readily formulated by a model that accurately predicts the system's velocity. Besides physically motivated models, more general models such as NNs or nonparametric models such as Gaussian processes (Hewing, Kabzan, & Zeilinger, 2020) can be used. These models still have the advantage of predicting the environment. However, the explainability of the states can be lost.

Not requiring the explicit modeling of the environment is a major claim of RL. However, this statement needs some additional framing. In fact, many successful model-free RL applications use models, at least for training the policy (Singh et al., 2022). One fundamental difference between RL and MPC models is that RL models are often used

Table 3

Literature that compares MPC with RL on practical applications. The table lists whether real-world experiments (RWE) were performed and which RL and MPC algorithms were used for the experiments.

Ref.	Application	RWE	MPC Formulation (Algorithm, Solver)	RL Algorithm
Ernst et al. (2009)	power system	No	NMPC (IP, N/A)	fitted Q iteration ^a
Ceusters et al. (2021)	multi-energy system	No	MILP-MPC (cplex ^b)	TD3, PPO
Hasankhani et al. (2021)	ocean current turbine	Yes	N/A	DQN
Lin et al. (2021)	cruise control	No	NMPC (IP, IPOPT ^c)	DDPG
Brandi et al. (2022)	thermal energy system	Yes	MILP (N/A)	SAC
Di Natale et al. (2022)	building temperature control	Yes	GP-MPC ^d , Bilevel DeePC ^e (custom)	TD3
Dobriborsci et al. (2022)	mobile robot	Yes	NMPC (SLSQP ^f)	DQN
Byravan et al. (2022)	MuJoCo locomotion	No	Sequential Monte Carlo ^g , CEM	MPO ^h
Wang et al. (2023)	building temperature control	No	NMPC (IP, IPOPT ⁱ)	SAC, DDPG, DDQN
Song et al. (2023)	drone racing	Yes	NMPC (SQP, acados)	PPO
Shi et al. (2023)	vehicle parking	No	NMPC (N/A)	PPO
Imran et al. (2023)	traffic control	No	MIQP-MPC (Gurobi)	DQN
Reiter et al. (2023)	autonomous racing	No	NMPC (SQP, acados)	SAC
Morcego et al. (2023)	greenhouse climate control	No	NMPC (IP, IPOPT)	DDPG
Mamedov et al. (2024)	flexible robot manipulation	No	NMPC (SQP, acados)	PPO, SAC
Hoffmann, Gorelik, and Ivanov (2024)	vehicle lane change	No	N/A	PPO, Q-learning
Krinner et al. (2024)	drone racing	Yes	NMPC (SQP, acados)	PPO
Oh (2024)	chemical and biological processes	No	LMPC (N/A), NMPC (IP, IPOPT)	SAC, DDPG, TD3

Notes: ^aErnst et al. (2003); ^bCplex (2009); ^cWächter and Biegler (2006); ^dHewing, Kabzan, and Zeilinger (2020); ^eCoulson et al. (2019b); ^fSciPy v1.15.0 Manual (2026); ^gPiche et al. (2018); ^hAbdolmaleki et al. (2018); ⁱGurobi Optimization, LLC (2023).

for offline simulation but not during deployment of the final policy. This allows for a large number of complex computations involved in the simulation, which could include logical statements or high-fidelity models. Often, these models are part of a dedicated simulation framework for RL (Song, Naji, et al., 2020; Todorov et al., 2012). Even though most RL methods still require simulation models, real-world RL is an active research field (Dulac-Arnold et al., 2021), making significant progress (Smith et al., 2023).

The low inference time of NNs makes it appealing to increase the sampling frequency of RL algorithms for faster feedback loops. However, a too-small discretization time step in RL can result in the function approximation error exceeding the value difference of actions, rendering deep RL methods that rely on function approximation useless (Tallec et al., 2019). Standard offline system identification techniques used to identify the MPC model also suffer from excessively high sampling frequencies, e.g., decreased signal-to-noise ratio and an ill-conditioned model (Ljung, 1999). However, in derivative-based MPC, the choice of the sampling frequency is often limited by the solution time of the underlying optimization problem.

5.3. Intrinsic stochasticity

As introduced in (1), the environment is typically assumed to be intrinsically stochastic. Thus, even if the environment is perfectly known, it is not possible to precisely predict future state trajectories. Both robust and stochastic MPC explicitly account for this uncertainty, which is typically referred to as noise or disturbance in this context. Robust MPC is, in a sense, agnostic to the question of whether the uncertainty is intrinsic or due to a systematic modeling error: it considers all trajectories possible under the given assumptions. For stochastic MPC, on the other hand, it is essential to be aware of whether prediction errors correlate over time, as this affects the predicted state distribution. Intrinsically stochastic noise is often assumed to be independent, i.e., noncorrelating.

As stochasticity is inherently part of the MDP framework, RL algorithms naturally consider stochastic environments. The policies can be trained directly in the real world to account for real-world uncertainty, cf. Section 3. A form of stochasticity particularly challenging for RL algorithms are rare events, i.e., large but rare deviations from the average obtained cost. Learning value estimates with rare events is particularly difficult (Frank et al., 2008).

5.4. Model mismatch

In practice, the environment for which a policy is designed or trained will often differ from the one on which it is deployed. Thus, it is essential to ensure the policy will perform well during deployment. Standard RL algorithms can be biased towards the specific model used during training. Different strategies, such as domain randomization, robust RL (Moos et al., 2022; Morimoto & Doya, 2000), and meta RL, tackle model uncertainty. In all of these approaches, the agent is trained not only on a single model but also on a potentially adaptive distribution of models, which can improve the generalization to new models (Finn et al., 2017). The underlying assumption is that the real-world environment lies in the distribution of the training models (Finn et al., 2017). Optimization-based meta RL (Finn et al., 2017) learns weights that can quickly adapt to new models. In contrast, in-context meta RL uses history-dependent policies to infer the current dynamics of the environment (Brown et al., 2020).

In the case of intrinsic stochasticity, both stochastic and robust MPC can explicitly take model mismatch into account. For stochastic MPC, it is crucial to consider that predictive errors due to model uncertainty are typically strongly correlated across time. Note that the model used in a stochastic MPC formulation is typically a simplification or approximation of the environment, leading to a mismatch of the stochastic MPC model with respect to the environment. The field of distributionally robust MPC aims to robustify against mismatches in the distribution model (Van Parys et al., 2016). Even without explicitly

accounting for model mismatch, MPC can perform remarkably well. This is the property of inherent robustness of MPC, as explained in more detail in Section 4.

5.5. Stability

Stability theory is usually not a significant concern for RL algorithms, since the main objective is closed-loop performance rather than practical stability. However, as denoted in Section 4, asymptotic stability and constraint satisfaction are historically the primary focus of MPC algorithms and applications. A widely used tool for analyzing the stability of control problems involves the construction of Lyapunov functions. If a Lyapunov function exists for a controlled deterministic MDP, it is said to be asymptotically stable, i.e., trajectories converge. If the MDP is stochastic, the concept of input-to-state-stability can be applied, which requires the trajectories of the controlled system to converge to a region around the origin whose magnitude depends on the maximum norm of the noise (Rawlings et al., 2017).

Input-to-state stability and asymptotic stability may not be applied to general MDPs since they require specific properties of the cost function, such as a feasible origin. Therefore, the authors in Gros, Zanon (2022a) propose a stability concept named D-stability that applies to MDPs and generalizes the dissipativity theory of economic MPC.

5.6. Constraints

The ability to account for constraint satisfaction and stability is one of the main reasons MPC is superior to other control techniques. In recent years, the RL community has also increasingly focused on providing safety guarantees (Brunke et al., 2022). A widely adopted framework in RL involves modeling constraints using constrained MDPs (Altman, 1999). Following Brunke et al. (2022), these include: Hard constraints, the constraints are always fulfilled, chance constraints, the constraints are fulfilled with high probability, and constraint violations are transformed into accumulated costs. The latter can be formulated either so that the accumulated costs do not exceed a safety budget or as a penalty integrated into the task objective (Brunke et al., 2022). The choice of formulation dictates the strategies employed to address these constraints. Common approaches include deriving safe action sets (Kalweit et al., 2020), optimizing a Lagrange formulation with dual gradient descent (Ray et al., 2019), using trust-region optimization (Achiam et al., 2017) or control-barrier functions (Cheng et al., 2019). Additionally, MPC (Wabersich & Zeilinger, 2021) was proposed to provide safety guarantees for a nominal model. Further work suggests augmenting the RL state with Lagrange multipliers (Calvo-Fullana et al., 2024) or with the currently used safety budget (Sootla et al., 2022). Still, a common problem is that safe RL policies can be either too conservative or violate constraints (Zhang et al., 2024). Thus, combining MPC and RL is highly desirable for constraint satisfaction.

5.7. Online computation time

A major concern in embedded applications is the maximum online computation time of an algorithm on embedded hardware. The maximum inference time of many NN architectures, which are often explicit functions, can usually be tightly bounded. However, the computation time of optimization solvers for NMPC problems is often unbounded. In fact, it cannot be guaranteed in general that a meaningful solution, i.e., at least a feasible solution, is returned by the optimization algorithm outside of particular optimization problem classes such as convex QPs (Arnström, 2023; Richter et al., 2011). For NMPC, the primal and, possibly, dual variable initialization of the optimization algorithm is essential for fast online computations. If the variables are sufficiently close to the optimal solution, the local convergence rate is fast.

5.8. Offline computation, engineering, and maintenance

In contrast to their fast inference time, RL algorithms usually require a tremendous number of training samples and therefore take a long time to train, even for small-sized MDPs. The training may be performed without human intervention. Yet in practice, fine-tuning on the RL hyperparameters may be required (Zhang et al., 2021). Besides the initial system identification, standard MPC does not require further training time. System identification differs from RL training in that it targets predicting only relevant outputs and possibly states of a system to achieve a low cost. In contrast, the RL target and the potential internal model focus on closed-loop performance. In the RL approach, the possible indirect internal model of the environment is learned only for the particular task described by the MDP. The classical system identification may also use models based on physical principles, enabling the controller to adapt to changing environments or requirements. RL may require a whole new training data set for changes in the cost, model changes, or changes in the distribution of the state space, e.g., an adjusted operating range in the environment.

5.9. Generalization

To some extent, RL policies can generalize out of their training distribution but with limited predictable behavior and guarantees of closed-loop performance (Nasytis et al., 2024). Thus, practitioners often need to ensure that the support of the training distribution covers the state and transition distribution encountered during deployment (Ross et al., 2011). An alternative is to train the RL policy during deployment further (Song, Yang, et al., 2020). If the model used within MPC approximates the real environment well across the entire state space, MPC generalizes well, and safety and performance guarantees can be established. Arguably, the model may generalize well based on available knowledge, starting from first principles and physical or mathematical insight. Knowledge about the model results in more interpretable generalization and is among the main motivations for MPC in general, for learning-based MPC, and for model-based RL. It may not be possible to evaluate whether a physical model approximates the real environment well on the whole state space of the environment. However, the physical explanation may also provide insights into its limitations.

5.10. Performance in practice

In practice, the expected performance difference between MPC and RL depends on the environment's specific characteristics, the availability of computational resources, and the quality of the model or data. Both approaches have their strengths and weaknesses, depending on the particular requirements and constraints of the control problem. Several works compare the two approaches across specific applications; see Table 3. The problems differ widely, from drone racing (Kaufmann et al., 2023) to multi-energy environments involving integer variables (Ceusters et al., 2021). The MPC formulations vary depending on the applications. For high sampling times, fast solvers such as acados (Verschuere et al., 2022) are used. Instead, for combinatorial problems, computationally intensive mixed-integer solvers, such as Gurobi (Gurobi Optimization, LLC, 2023), are required. Most of the authors use the same state space for MPC and RL, despite the RL's ability to cope with arbitrary information inputs, which is exploited only in Kaufmann et al. (2023), Oh (2024). For a known model, most authors report superiority of MPC w.r.t. the closed-loop performance, with less than 4% cost reduction in Brandi et al. (2022), Ernst et al. (2009) and 16% in Wang et al. (2023) when compared to RL. In Mamedov et al. (2024), the authors report that MPC outperforms RL on a flexible robot manipulation task with a rather high-dimensional state space. RL was claimed to be superior to MPC for environments with poor models, e.g., in Ceusters et al. (2021), Hasankhani et al. (2021), yet robust control techniques are not implemented. The authors in Kaufmann et al. (2023) claimed superiority of a particular choice of RL algorithms against MPC in real-world experiments (RWE) of racing drones.

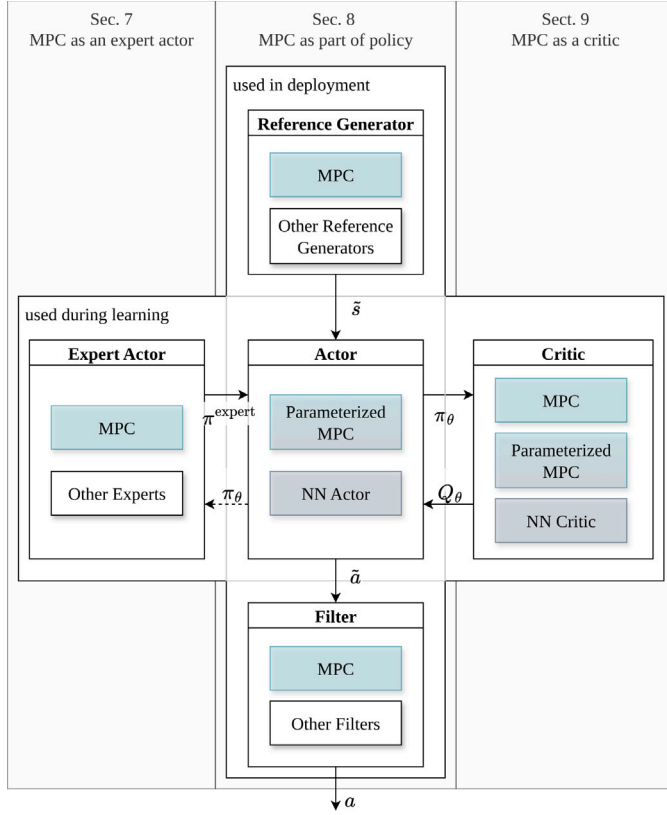


Fig. 2. Modular view on combinations of MPC and RL. The combinations are aligned with the sections of this survey. The horizontal separation corresponds to using MPC within the expert actor, the deployed policy, and the RL critic. This overview highlights the possibility of using several instances of MPC with different roles in various parts of an RL algorithm and a deployed policy. MPC is used with fixed expert parameters within the expert actor, as a reference generator, as a postprocessing filter, or, possibly, in the critic. Learned MPC, i.e., an MPC structure involving learned parameters, can be used within the learned actor or possibly within the learned critic.

6. Combination approaches

As pointed out in the previous Section, the two approaches of MPC and RL exhibit different advantages. Therefore, a combination of both is appealing, which has led to numerous algorithms in the field, often with outstanding similarities. Based on the available literature, we propose a categorization of combination approaches that distinguishes which algorithmic part of RL the MPC framework is applied to. The related general RL building blocks are the actor, the critic, and possibly an expert actor, as shown in Fig. 2. Accordingly, we propose the categories

1. MPC as an expert actor, cf. Section 7,
2. MPC within the deployed policy, cf. Section 8,
3. MPC as part of the critic, cf. Section 9.

In the following, we briefly outline these different categories.

MPC as an expert actor. An MPC with fixed parameters and the desired behavior can be used as an expert to learn policies. Expert behavior can be used either by an IL algorithm that attempts to purely mimic it or to guide the exploration process during RL training (Levine & Koltun, 2013a). MPC used as an expert is elaborated in Section 7.

MPC within the deployed policy. Another common variant of combining MPC and RL is using MPC within the deployed policy. A parameterized MPC can be used as part of the learned actor, as, for instance, proposed

by Gros, Zanon (2022a). Closely related are concepts that use MPC as a posterior filter or as a reference provider within the deployed policy, but are not used during learning. For instance, the authors in Wabersich and Zeilinger (2021) utilize MPC as a safety filter, and Jenelten et al. (2024) employ MPC as a reference provider, as noted in Fig. 2. All concepts that use MPC within the deployed control policy are elaborated in Section 8.

MPC as part of the critic. An MPC, possibly parameterized with learnable parameters, can further be used to evaluate the value function at a given state, i.e., the critic uses an MPC variant. For instance, the critic in Ghezzi et al. (2023) uses an MPC with fixed parameters to compute the value at a given state. This structure is reviewed in Section 9.

Sections 7 to 9 describe combination approaches of MPC and RL conceptually, leaving out a detailed theoretical discussion. Nonetheless, the additional theory in Section 10 highlights some important findings and relevant literature when combining the two approaches.

To the best of the authors' knowledge, in the following review, the most important works that combine MPC and RL can be classified according to the proposed categories in Fig. 2. The works are further compared in terms of their application domain, whether RWEs were performed on embedded hardware, the MPC type and solver the authors used, and the RL algorithm they applied. While the MPC algorithm classification is more distinct, the RL algorithm's classification can be ambiguous and intertwined with the overall algorithm. Besides widespread RL algorithms such as SAC and PPO, several forms of DP, such as policy iteration (PI) or VI, forms of IL such as behavior cloning (BC), and even supervised learning for learning value functions, such as maximum likelihood estimation (MLE), are denoted as RL algorithms in the following tables. Applications include unmanned ground vehicle (UGV) control, temperature control, energy systems, chemical processes, traffic management, autonomous racing with vehicles or drones, and automated driving (AD).

6.1. Architectures of parameterized MPC

Before diving into the specific approaches of how MPC is used as part of RL or IL algorithms, a brief classification of parameterized MPC architectures that utilize NNs is provided. This distinction is crucial for designing combination approaches, as it centrally governs the desired properties and algorithmic choices.

We consider an MPC optimization layer that can be parameterized by a parameter $\phi \in \mathbb{R}^{n_\phi}$ and may depend on the decision variables z of the MPC optimization problem and the current state s through a highly nonlinear function approximator (FA) $\phi = \varphi_\theta(s, z)$, e.g., an artificial neural network (NN) with the (very) high dimensional parameter vector $\theta \in \mathbb{R}^{n_\theta}$. We use the common NNs as an instance for general FAs, but do not exclude other forms of FAs. When fixing the parameters ϕ , the MPC optimization problem is assumed to contain only "optimization-friendly" objective, model, and constraint functions. The optimization algorithm may also involve hyperparameters ψ , such as initialization strategies or integration parameters.

A general form of the parameterized MPC, based on (22), can be written as

$$\min_z L_\phi(z) \quad (25a)$$

$$\text{s.t.} \quad x_0 = s, \quad (25b)$$

$$x_{k+1} = f_\phi^{\text{MPC}}(x_k, u_k), \quad 0 \leq k < N, \quad (25c)$$

$$0 \leq h_\phi^{\text{MPC}}(x_k, u_k), \quad 0 \leq k < N, \quad (25d)$$

$$0 \leq \tilde{h}_\phi^{\text{MPC}}(x_N), \quad (25e)$$

and the parameterized objective is

$$L_\phi(z) := \bar{V}_\phi^{\text{MPC}}(x_N) + \sum_{k=0}^{N-1} l_\phi^{\text{MPC}}(x_k, u_k).$$

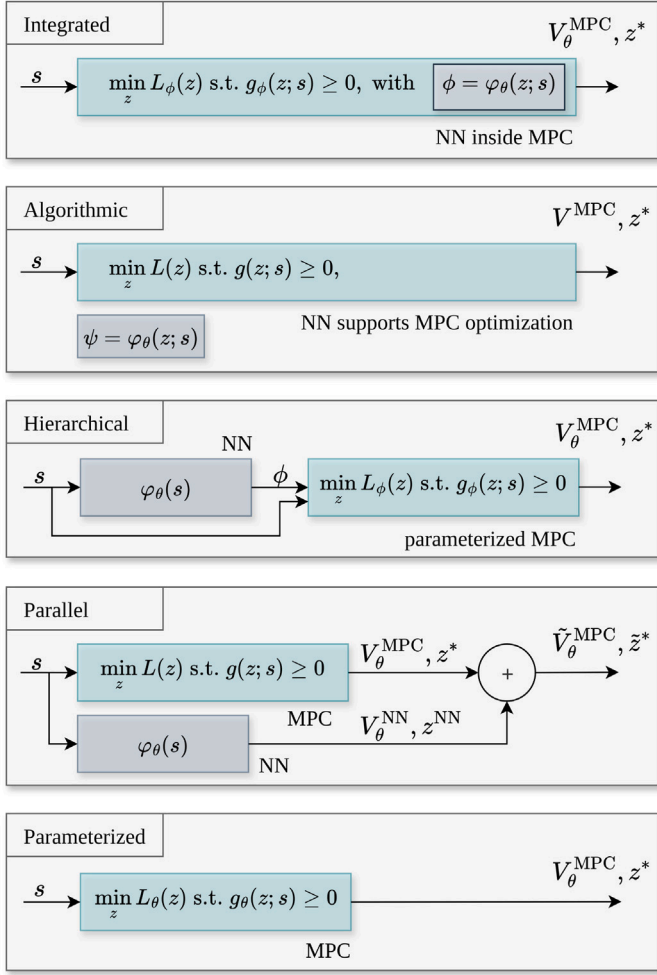


Fig. 3. Parameterized MPC architectures: Proposed architectures of actors (or potentially critics) used in RL utilizing MPCs and NNs/simple parameters. In the integrated architecture, the NN is part of the optimization layer and depends on the decision variables. Suppose an NN can be evaluated separately from the optimization routine but provides the parameters to an MPC optimization layer. In that case, the architecture is referred to as hierarchical. If the MPC problem is solved in parallel to the NN, we refer to a parallel architecture. If the learned parameters do not depend on the current state s , the architecture is referred to as parameterized. When the RL policy supports the algorithm that solves the MPC problem, the architecture is referred to as algorithmic.

For easing the notation, all constraints of (25) are summarized by $g_\phi(z; s) \geq 0$. Thus, the optimization problem (25) can be concisely written as

$$\min_z L_\phi(z) \quad \text{s.t.} \quad g_\phi(z; s) \geq 0. \quad (26)$$

Note that we use the notation $g_\phi(z; s)$ to indicate that z are decision variables of the optimization problem and s are parameters. The following architectures are proposed within this context, starting with the most general one, i.e., the integrated architecture, cf. Fig. 3.

6.1.1. Integrated architecture

In the integrated setting, the parameters depend on both the state s and the optimization variables z via the function $\varphi_\theta(s, z)$, as, for instance, within the algorithms proposed by Hansen et al. (2022, 2024), Lowrey et al. (2019). The authors in Hansen et al. (2022, 2024) solve the optimization problem within a latent state space and use NN encoders and decoders to infer states or rewards and highly nonlinear

NN transition models. Therefore, the NN is part of the optimization problem and cannot be evaluated separately from the optimization algorithm in the inference path. Including the NN in the optimization problem yields a highly nonlinear, possibly nonconvex, problem that may be computationally challenging. Particularly, derivative-based solvers require differentiating the highly nonlinear parameterized function $\varphi_\theta(z; s)$ w.r.t. the decision variables z to obtain $\nabla_z \varphi_\theta(z; s)$. Note that the optimization layer can also be used to provide the value function. Consequently, it may output the optimal MPC objective function V_θ^{MPC} of the MPC optimization problem or its decision variables z^* .

6.1.2. Algorithmic architecture

In the algorithmic architecture, the RL policy φ_θ alters the hyperparameters ψ of the algorithm that solves the MPC optimization problem. Compared to all other proposed architectures, the RL policy does not modify the optimization problem itself. Example algorithms include Hansen et al. (2022, 2024), where an RL policy is used to augment sampled action proposals in a sampling-based MPC approach. Here, the term hyperparameters refers to any information that can guide the optimization process, such as active-set predictions or warm-starting through informed initial guesses.

6.1.3. Hierarchical architecture

In the hierarchical architecture, the neural network provides an input to the MPC. Therefore, the parameter ϕ depends on the current states s via the possibly highly nonlinear function $\phi = \varphi_\theta(s)$. For example, a reference trajectory parameterized by ϕ could be provided, such as in Brito et al. (2021), Reiter et al. (2023), or ϕ could parameterize constraints in the OCP, similar to Jacquet and Alexis (2024). Notably, the function $\varphi_\theta(s)$ can be evaluated before solving the optimization problem. In this case, the potential nonlinearity of $\varphi_\theta(s)$ does not influence the numerical optimization problem structure. Therefore, the optimization problem does not get significantly harder to solve. However, the changing parameters may still change the convergence of the perturbed problem.

6.1.4. Parallel architecture

In the parallel architecture, the MPC problem is usually not parameterized. However, an NN is evaluated in parallel to the MPC optimization problem, and its output is used to correct the optimal solution for decision variables or the value function of the MPC. This architecture holds the advantage of not requiring differentiation of the optimization problem, but the disadvantage of potentially unsafe actions due to perturbation of the MPC actions. An example of this architecture used to approximate the value function can be found in Bhardwaj et al. (2021).

6.1.5. Parameterized architecture

In the parameterized MPC architecture, the parameters ϕ are constant and independent of decision variables z or the state s . Conceptually, a parameterized optimization-friendly MPC problem is formulated that does not require the evaluation of highly nonlinear functions to obtain the parameter ϕ . For instance, such a parameterization is proposed in Gros and Zanon (2020a), Moradimaryamnegari et al. (2022) and Liu et al. (2023).

In this Section, we introduced several architectures for parameterizing MPC with potentially highly nonlinear functions, such as NNs. These architectures may be used in the following MPC and RL combinations, where some parameterized MPC architectures are particularly well-suited to specific combinations.

Table 4

Literature that uses MPC as an expert actor for RL. The table lists the applications and whether real-world experiments (RWE) was performed. Additionally, the IL/RL and MPC algorithms are stated. Note that RL algorithms can be used to perform IL, e.g., [Kang et al. \(2023\)](#). The table further differentiates whether the expert adapts to the approximation error of the learned policy via guided policy search (GPS).

MPC as an Expert Actor					
Ref.	Application	RWE	MPC Formulation (Algorithm, Solver)	IL/RL Algorithm	GPS
Levine and Koltun (2013a)	MuJoCo locomotion	no	NMPC (custom iLQR)	Off-policy PG	yes
Levine and Koltun (2013b)	MuJoCo locomotion	no	NMPC (custom iLQR)	Var. Policy Search ^a	yes
Mordatch and Todorov (2014)	MuJoCo locomotion	no	NMPC (custom iLQR)	BC/ADMM	yes
Levine et al. (2016)	PR2 robot arm/robot manipulation	yes	NMPC (custom iLQR)	BC/Bregman ADMM ^b	yes
Sun et al. (2018)	autonomous driving	no	NMPC (IP, IPOPT)	Dagger ^c	no
Nagabandi et al. (2018)	MuJoCo locomotion	no	NMPC, random sampling (custom)	Dagger, TRPO	yes
Wang and Ba (2019)	MuJoCo locomotion	no	CEM	BC, IL	no
Kaufmann et al. (2020)	drone acrobatics	yes	NMPC (SQP, acado ^d)	Dagger	no
Pinneri et al. (2021)	MuJoCo locomotion	no	CEM	Dagger	no
Sacks and Boots (2022)	robot arm	no	MPPI (custom)	Dagger	no
Dawood et al. (2023)	Kuboki Turtlebot 2	yes	NMPC (SQP, acados)	SAC, TD3	no
Kang et al. (2023)	Unitree Go1 and Aliengo (quadruped)	yes	NMPC (N/A)	PPO	no
Ahn et al. (2023)	random linear system	no	LMPC (N/A)	Dagger	no
Lidec et al. (2023)	robot arm	no	NMPC (FDDP ^e)	BC/ADMM	yes
Mamedov et al. (2024)	flexible robot arm	no	NMPC (SQP, acados)	BC, Dagger, AIRL ^f , GAIL ^g	no
Hoffmann, Clausen, et al. (2024)	autonomous driving	no	LMPC (SQP, acados)	custom IL	no
Schulz et al. (2024)	cart pole, spacecraft	no	NMPC (SQP, acados)	TD3	yes

Notes: ^aNeumann (2011); ^bWang and Banerjee (2014); ^cRoss et al. (2011); ^dHouska et al. (2011); ^eMastalli et al. (2020); ^fFu et al. (2018); ^gHo and Ermon (2016).

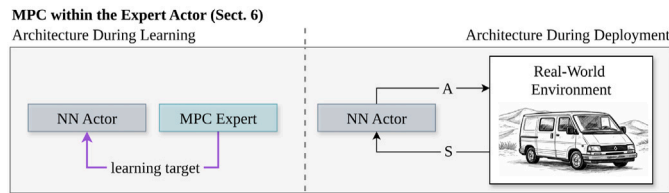


Fig. 4. Combinations: MPC as an expert actor. The plot is split into the learning and deployment phases. Grey boxes indicate Neural Networks (NNs), and green boxes indicate MPCs.

7. MPC as an expert actor

For specific problems, it is possible to design an expert MPC that achieves good closed-loop performance. There are several ways to leverage such an expert, motivating the structure of the following section:

- Imitation learning from MPC**, Section 7.1. In this paradigm, the goal is to replace the expert MPC with an NN to achieve faster computation time using imitation learning.
- Guided Policy Search using MPC**, Section 7.2. In this paradigm, the expert MPC policy improves exploration during RL training, guiding the learned policy towards low-cost regions.

A thorough collection of relevant literature can be found in [Table 4](#) and a sketch in [Fig. 4](#). Besides the explored application, the table identifies whether the algorithm is related to imitation learning (IL) or guided policy search (GPS) and specifies the particular MPC, RL, and IL algorithms on which it is based.

7.1. Imitation learning from MPC

In a problem setting where MPC achieves a sufficient closed-loop performance burdened only by its online computation time, a trained NN may replace the MPC. The NNs' typically fast inference can drastically reduce online computation time by omitting the time-consuming online optimization. In fact, in many real-world applications ([Dawood et al., 2023](#); [Kang et al., 2023](#); [Mamedov et al., 2024](#)), the computation time of the MPC solver is a significant limitation and potentially even unbounded, particularly for more complex structures, e.g., involving nonlinear systems, stochasticity, or discrete decisions, c.f. Section 4.2. Two research directions try to alleviate the problem of large online computation times.

First, for linear systems with linear constraints and convex quadratic costs, the optimal feedback control law is piecewise linear within a polytope of the state space ([Bemporad et al., 1999](#)). Within explicit MPC ([Bemporad et al., 2002](#); [Grancharova & Johansen, 2012](#)), these polytopes and their control laws are computed offline and switched during deployment by determining the active region. For large state spaces or a large number of constraints, explicit MPC becomes quickly intractable. Thus, approximate explicit MPC only approximately stores the control law of the MPC policy ([Johansen & Grancharova, 2003](#)).

A learning-based approach to approximate explicit MPC is using IL to learn the MPC expert. Methods can learn the whole trajectory predictions ([Vaupel et al., 2020](#)) or learn the first applied action ([Hertneck et al., 2018](#)). Early works ([Åkesson & Toivonen, 2006](#); [Parisini & Zoppoli, 1995](#)), and several follow-up studies ([Mor, 2020](#); [Cao & Gopaluni, 2020](#); [Chen et al., 2018](#); [Hertneck et al., 2018](#); [Karg & Lucia, 2020](#); [Pinneri et al., 2021](#)) aim to imitate a complex MPC using a neural network, by training the NN with supervised learning methods. Further aspects of the learned NN policy where analyzed regarding safety ([Cosner et al., 2022](#); [Mamedov et al., 2024](#)), stability ([Schwan et al., 2023](#)) and robustness ([Hertneck et al., 2018](#)). Another important aspect is the verification of the learned NN, with the objective of minimizing the worst-case approximation error relative to the MPC

expert. This can be done in a probabilistic fashion using Hoeffding's inequality, such as in Drgoña et al. (2024), Hertneck et al. (2018), where confidence bounds are derived for an estimate of how often an approximation error threshold might be exceeded. Alternatively, verification techniques, such as those in Schwan et al. (2023), solve an MIQP to bound the worst-case approximation error. For a general overview of NN verification, see Everett (2021).

Often, a surrogate loss function as in standard BC minimizes the squared distances between the predicted action of the NN and the action of the MPC expert. However, it disregards the costs and structure of the underlying OCP while learning the policy. Thus, a natural replacement is to use instead the Q-function of the MPC, cf. (23) as a learning objective (Carius et al., 2020; Ghezzi et al., 2023; Reske et al., 2021). Since the MPC provides a Q-function in addition to the expert policy, cf. Fig. 2, we categorize such methods into a class named MPC as a critic, as further discussed in Section 9.

7.2. Guided policy search using MPC

In the previously mentioned IL methods, the MPC expert does not consider the progress of the learned policy during training. In the guided policy search (GPS) regime, the expert gradually guides the learned policy to better trajectories (Levine et al., 2016; Levine & Koltun, 2013a, 2013b; Mordatch & Todorov, 2014). This is ensured by coupling the trajectory optimization and policy learning by requiring that the MPC expert does not deviate too far from the current predictions of the learned policy (Mordatch & Todorov, 2014). The final learned policies can generalize even when the MPC expert fails to find a solution (Levine et al., 2016). Furthermore, the authors in Mordatch and Todorov (2014) have demonstrated superior performance compared to policies learned from a fixed dataset of expert trajectories.

Another possibility for guiding the exploration process in RL was proposed in Schulz et al. (2024) and is related to the options framework (Sutton et al., 1999). A learned high-level exploration policy decides between using an MPC expert policy or the currently learned low-level RL policy. The usage of the MPC expert is regularized over time to enforce that the learned low-level RL policy works as a stand-alone policy during deployment. A benefit over GPS is that the high-level policy can avoid using the MPC expert for state regions, where the MPC expert guidance is suboptimal.

8. MPC within the Deployed Policy

This section discusses methods that utilize MPC during deployment in real-world environments. A primary distinction is drawn between whether parameterized MPC is trained by RL algorithms or used for pre- or post-processing after RL training. When learning a parameterized MPC, we furthermore distinguish between approaches that aim to align the MPC formulation with the MDP structure, e.g., to learn an internal MPC model that approximates the real environment as closely as possible, and methods that primarily focus on closed-loop optimality. Accordingly, we structure this section as follows

- A. **Aligned-learning**, Section 8.1. In the paradigm of aligned learning, the MPC structure (21) approximates the real-world MDP structure (2), i.e., it uses a transition model, costs, constraints, and a terminal value function that individually approximate the MDP and the optimal value function V^* , respectively.
- B. **Closed-loop learning**, Section 8.2. In the paradigm of closed-loop learning, an MPC is used as an optimization layer within the actor policy and trained within an RL algorithm for closed-loop optimality, which, in general, does not require the individual parts of the MPC to be aligned with the MDP. For instance, the MPC model should not necessarily fit the real system expected or most likely transition to provide closed-loop optimality (Kordabad et al., 2024).
- C. **MPC for pre/postprocessing**, Section 8.3. Using MPC for pre-/postprocessing does not involve MPC during learning, but as part of the deployed policy. This section is further split into reference generation and filtering.
- D. **MPC optimization guided by RL**, Section 8.4. Unlike the paradigms above, where a NN is integrated into the OCP formulation, RL can also be used to guide the solution process of the MPC's OCP. In this case, the MPC layer remains part of the actor during deployment.

The different structures that include MPC in the deployed policy are shown in Fig. 2 as part of the general overview and in more detail in Fig. 5.

8.1. Aligned learning

The classical design procedure of MPC focuses in its first step on finding a mathematical model $f_{\theta}^{\text{MPC}}(x, u)$ that describes the environment sufficiently well (Ljung, 1999) and is typically deterministic. Subsequently, a horizon length N and a terminal value function $\bar{V}_{\theta}^{\text{MPC}}(s)$ are selected to approximate the optimization problem over a finite horizon, as detailed in (21). Both functions are parameterized, and those parameters are jointly noted by θ . The advantage of aligning the MPC with the MDP is evident in its well-interpretable formulation, adaptability to new problems, generalization, potential stability and recursive feasibility guarantees, and the division of the overall design into subtasks. However, the online computation time may be high if complex MPC formulations are used, e.g., stochastic MPC formulations that account for uncertainty.

Obtaining the terminal value function to approximate the infinite horizon value function as in (21) is computationally challenging and, therefore, usually approximated and often obtained through learning algorithms closely related to the RL framework. For simple MDPs, e.g., deterministic MDPs with linear models and a quadratic cost, the terminal value function can be computed exactly and does not require learning or approximation. The costs $J^{\text{MPC}}(x, u)$ are usually given by the application, i.e., the MDP, but may be altered to improve the numerical properties of the optimization problem, such as smoothness and convexity.

Literature that can be categorized as MDP aligned typically uses the integrated or parameterized architecture according to Fig. 3. For instance, the parameters θ could parameterize a NN that is used as the internal MPC model (Nagabandi et al., 2018) or the terminal value function (Lowrey et al., 2019), corresponding to the integrated architecture. Table 5 summarizes literature that can be related to the MDP aligned learning paradigm. Thereby, a distinction is made between whether the model or the terminal value function was learned. Additionally, the table indicates whether the corresponding object was learned during deployment of the continually improving MPC policy, referred to as on-policy learning (ON-P), or via a separate off-policy sampling strategy (OFF-P). An example of an off-policy sampling strategy would involve system identification to identify a model before deploying the MPC.

The following section provides a detailed explanation of the two objectives of learning the model or terminal value function.

8.1.1. Learning the model

The first target of aligned learning is to approximate the stochastic model of the real environment with a model f_{θ}^{MPC} that is used within MPC. The parameterized mathematical model is typically designed via first principles or a more generic model using a function approximator like an NN or Gaussian process (GP). Parameters θ of candidate models are fit to observed transition data $D^{\text{id}} = \{(S_0, A_0, S_1), \dots, (S_{M-1}, A_{M-1}, S_M)\}$ of state transitions and related actions. The evaluation of the approximation requires a validation or

MPC within the Deployed Policy (Sect. 8)

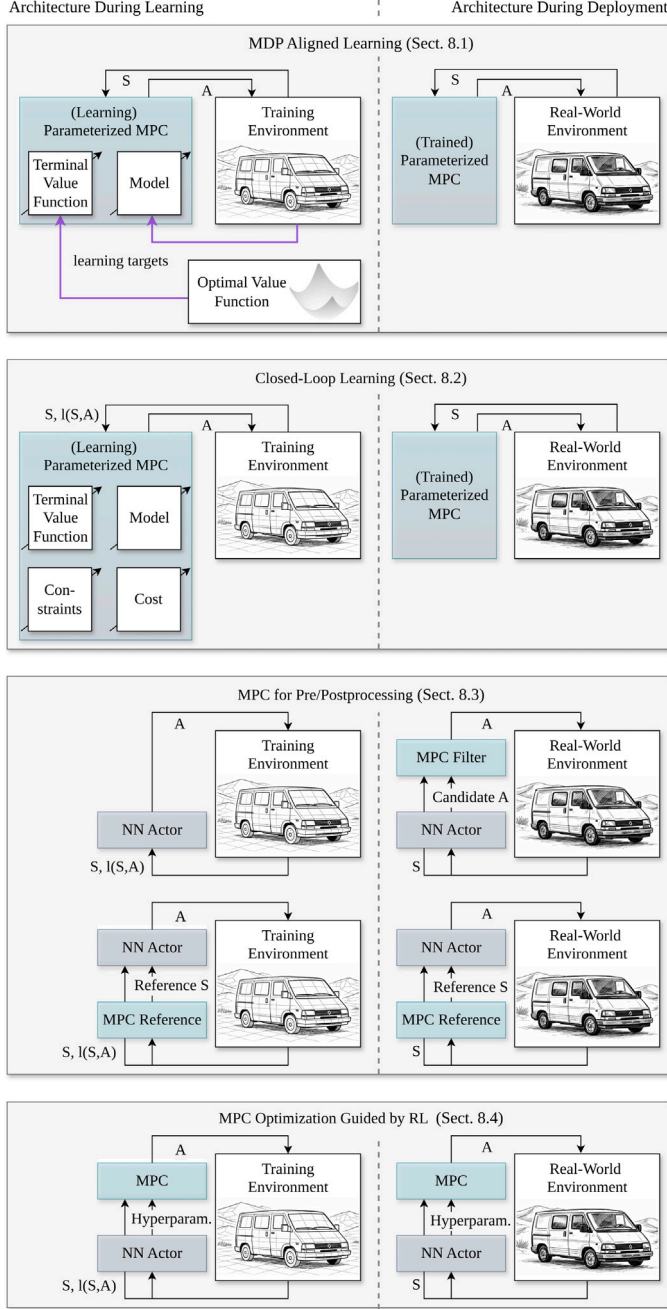


Fig. 5. Combinations: MPC within the deployed policy. The plot is divided into two phases: learning and deployment. Grey boxes indicate Neural Networks (NNs), green boxes are used for MPCs, and grey/green boxes refer to parameterized MPCs that involve learned parameters/NNs during the learning phase.

loss function $\mathcal{L}^{\text{id}}(\cdot)$, which could be, for instance, the least-square loss function. The parameters can be found by minimizing

$$\min_{\theta} \mathbb{E}_{(S, A, S^+) \sim \mathcal{D}^{\text{id}}} \left[\mathcal{L}^{\text{id}}(S^+ - f_{\theta}^{\text{MPC}}(S, A)) \right]. \quad (27)$$

8.1.2. Learning the terminal value function

In order to approximate the MDP on a finite MPC horizon as in (21), a terminal value function $\bar{V}_{\theta}^{\text{MPC}}$ needs to be established. From a system theoretical perspective, the terminal value function is essential in terms

of stability or recursive feasibility, c.f., Section 4. However, when focusing on closed-loop cost, the terminal value function must tractably approximate the true value function V^* of the MDP.

Ultimately, various aspects should be considered when determining an appropriate terminal value function. For instance, the terminal value function $\bar{V}_{\theta}^{\text{MPC}}$ can be learned via a temporal difference update, cf. Lowrey et al. (2019), where a learned terminal value function is incorporated into an MPPI planner. In the following, a simplified version is provided to learn a parameterized terminal value function $\bar{V}_{\theta}^{\text{MPC}}$, potentially a NN, that can be defined via

Terminal value function learning ($\bar{V}_{\theta}^{\text{MPC}} \approx ?^2$)

$$\theta \leftarrow \theta + \frac{\alpha}{B} \sum_{i=1}^B \delta_i \nabla_{\theta} \bar{V}_{\theta}^{\text{MPC}}(S_i),$$

$$\delta_i := l(S_i, A_i) + \bar{V}_{\theta}^{\text{MPC}}(S_i^+) - \bar{V}_{\theta}^{\text{MPC}}(S_i),$$

$$\text{with } S_i \sim \mathcal{D}^{\pi_{\theta}^{\text{MPC}}}, A_i = \mu_{\theta}^{\text{MPC}}(S_i), S_i^+ \sim P(\cdot | S_i, A_i),$$

where $\mathcal{D}^{\pi_{\theta}^{\text{MPC}}}$ is a distribution of states generated by controlling the system via the parameterized MPC exploration policy $\pi_{\theta}^{\text{MPC}}$ and α is the learning rate. Another approach to learning the terminal value function is shown in Reiter et al. (2025), where the authors use either model-free SAC or PPO to obtain a terminal value function for an MPC.

8.2. Closed-loop learning

In the following section, the paradigm of viewing MPC as a parameterized optimization layer as part of an actor policy is explained in more detail. In the closed-loop learning paradigm, the MPC model is not required to minimize a prediction loss $\mathcal{L}^{\text{id}}$ or to provide a terminal value function that approximates the optimal value function as in the MDP aligned setting. Instead, the model, costs, or constraints may be changed to solely improve the closed-loop performance of the MPC policy μ^{MPC} according to Eq. (4). Fig. 5 shows a sketch of the closed-loop optimal learning paradigm.

An advantage of this paradigm is the ability to achieve optimal closed-loop performance in the real environment without the necessity of computationally demanding aligned formulation of Section 8.1. However, by modifying the various parts of the MPC solely to enhance closed-loop performance, explainability, safety, generalization, and adaptability may be compromised, particularly if the model or constraints are allowed to change.

In practice, this paradigm differs depending on whether the MPC controls or the MPC parameterization obtained via a learnable NN are treated as RL actions. The following explains these two practical implementations.

8.2.1. Differentiable MPC

MPC can be used as an optimization layer within the RL policy to achieve good initial performance by leveraging knowledge of the task (Acerbo et al., 2024; Gros, Zanone, 2022a). Ideally, the MPC performance would be only slightly suboptimal and provide safety guarantees for a known model. The hope is that, by only a few RL iterations, the parameters can be modified towards nearly optimal closed-loop performance, particularly in terms of stochasticity.

However, the MPC optimization problem, as part of the learned actor, creates potential challenges for both the forward pass and the training of parameters in the backpropagation of gradients.

Compared to the inference of conventional NNs, such as multi-layer perceptrons, solving an optimization problem is numerically challenging. Thus, the related iterative algorithms in the forward pass may

² The question mark indicates that, to the best of the authors' knowledge, the update scheme does not converge to the optimal value function V^* in general, and its convergence target remains unclear.

Table 5

Literature that uses MDP-aligned learning. Either the value function, the model, or the cost function of an MPC actor is learned. The table lists the applications and whether real-world experiments (RWEs) were performed, and which MPC formulation was used. Moreover, the learning or approximation algorithm used to obtain the model or the terminal value function approximation is stated. The model or terminal value function of the MPC is either updated using the current MPC policy to collect samples, i.e., on-policy (ON-P), or using another policy, referred to as off-policy (OFF-P) sampling. The terminal value function can also be computed by offline DP or online planning (e.g., RRT) based on a given model. This computation does not utilize sampling strategies, thus it is neither related to on-policy nor off-policy learning and indicated by N/A.

MPC as an Actor: MDP Aligned Supervised Learning							
Ref.	Learning of	Architecture/ specific FA	Application	RWE	MPC Formulation (Algorithm, Solver)	Learning/Approx. Algorithm	Sampling
Zhong et al. (2013)	term. val. fun.	integrated/ several FAs	pendulum and acrobat	no	NMPC (iLQR, custom)	DP	OFF-P
Aswani et al. (2013)	model	parameterized	HVAC, quadrotor	yes	NMPC (SQP, SNOPT ^a)	supervised	ON-P
Nagabandi et al. (2018)	model	integrated/ NN	MuJoCo locomotion	no	random sampling (custom)	Dagger-like	ON-P
Lowrey et al. (2019)	term. val. fun.	integrated/ NN	3D humanoid, five-fingered hand	no	MPPI (custom)	supervised	ON-P
Wang and Ba (2019)	model	integrated/ NN	MuJoCo locomotion	no	CEM ^b (N/A)	IL	ON-P
Deits et al. (2019)	term. val. fun.	integrated/ NN	walls pendulum 2D humanoid	no	MIQP-MPC (Gurobi)	supervised	OFF-P
Yang et al. (2020)	model	integrated/ NN	legged robot ^c	yes	CEM (custom)	supervised	ON-P
Lambert et al. (2019)	model	integrated/ NN	Crazyflie quadrotor	yes	random sampling (custom)	supervised	OFF-P
Karnchanachari et al. (2020)	term. val. fun.	integrated/ NN	UGV	yes	NMPC (SQP, acado)	TD	ON-P
Beckenbach et al. (2020)	term. val. fun., stage cost	parameterized	chemical reaction	no	N/A	TD	N/A
Hoeller et al. (2020)	term. val. fun.	integrated/ NN	ballbot (custom)	yes	NMPC (SLQ/iLQR, custom)	TD	ON-P
Hatch and Boots (2021)	term. val. fun.	custom/ roll-out	four-wheeled skid-steered robot	no	MPPI (custom)	DP-related (RRT#)	N/A
Morgan et al. (2021)	model	integrated/ NN	MuJoCo, robotic hand	yes	MPPI (custom)	SAC	ON-P
Dobriborsci et al. (2022)	term. val. fun.	parameterized	mobile robot	yes	NMPC (SLSQP)	DQN	OFF-P
Sikchi et al. (2022)	term. val. fun., model	integrated/ NN	OpenAI Safety Gym, MuJoCo	no	NMPC (custom)	MPPI / custom	OFF-P
Beckenbach and Streif (2022)	term. val. fun.	parameterized	chemical reaction	no	N/A	ADP	N/A
Hansen et al. (2022)	term. val. fun., stage cost, model	integrated/ NN	92 robotic tasks	no	NMPC (custom)	supervised / custom	ON-P
Cai et al. (2023)	term. val. fun., stage cost, model, constraints	parameterized	home energy management	no	NMPC (N/A)	TD3, AC	ON-P
Moreno-Mora et al. (2023)	term. val. fun.	parameterized	spacecraft	no	NMPC (fminunc ^d)	VI	OFF-P
Hansen et al. (2024)	term. val. fun., stage cost, model	integrated/ NN	92 robotic tasks	no	NMPC (custom)	supervised / custom	ON-P
Lin et al. (2024)	term. val. fun.	integrated/ polynomial	mobile robot	no	LQR (N/A)	PI	ON-P
Qu et al. (2024)	term. val. fun.	integrated/ NN	unmanned aerial vehicle	no	MPPI (custom)	SAC	OFF-P
Bar et al. (2024)	model	integrated/ NN	visual navigation	no	CEM	N/A	OFF-P
Reiter et al. (2025)	term. val. fun.	integrated/ NN	autonomous driving	no	NMPC (SQP, acados)	PPO, SAC	OFF-P
Ghezzi et al. (2025)	term. val. fun.	custom/ policy roll-out	drone flight	no	NMPC (SQP, acados)	PPO	OFF-P

Notes: ^aGill et al. (2002); ^bRubinstein and Kroese (2004); ^cKenneally et al. (2016); ^dThe MathWorks Inc (2022).

slow down learning during policy evaluation. Moreover, obtaining a global optimizer outside the class of convex optimization problems is generally intractable. For pure control tasks, local optima are often sufficient to achieve good closed-loop performance. However, local optima introduce a dependency on the initial guess and, therefore, require consistent warm-starting or extending the state by the initial guess in the RL framework.

In backpropagation, gradients must be computed through the optimization solver. These gradients can be calculated using the implicit function theorem, cf. Appendix A and related literature (Amos et al., 2018; Romero et al., 2024; Zanon & Gros, 2021) for further details. However, existing optimization software is required to support such features, as summarized in Section 11.

In this learning paradigm, the integrated, hierarchical, parallel, and parameterized architectures shown in Fig. 3 may be used. Still, to the best of the authors' knowledge, only hierarchical architectures,

e.g. Romero et al. (2024), Zanon et al. (2020), and the parameterized architectures, e.g., Amos et al. (2018), Gros and Zanon (2020a), Moradimaryanegari et al. (2022), have been proposed so far.

The following algorithm based on the ideas of Gros and Zanon (2020a) provides a basic version of closed-loop optimal learning, where the parameters are updated by differentiating the MPC. The temporal difference update (13) is adapted to the parameterized MPC setting to

Q-learning with MPC Q-function ($Q_{\theta}^{\text{MPC}} \approx Q^*$)

$$\begin{aligned} \theta &\leftarrow \theta + \alpha \delta \nabla_{\theta} Q_{\theta}^{\text{MPC}}(S, A), \\ \delta &:= l(S, A) + \gamma V_{\theta}^{\text{MPC}}(S^+) - Q_{\theta}^{\text{MPC}}(S, A), \\ &\text{with } (S, A) \sim \mathcal{D}_{\pi_{\theta}^{\text{MPC}}}, S^+ \sim P(\cdot | S, A). \end{aligned}$$

The state-action distribution $\mathcal{D}$ is generated by controlling the system via the stochastic parameterized MPC exploration policy $\pi_{\theta}^{\text{MPC}}$.

Note that the update scheme only considers a single sample, which is often reasonable in the case that the Q_θ^{MPC} is just a parameterized MPC scheme as described in Fig. 3, without a NN.

Another example of closed-loop optimal learning based on a DDPG actor-critic formulation similar to (19) was introduced in Gros and Zanon (2020a) and is given by

DDPG with MPC actor ($\mu_\theta^{\text{MPC}} \approx \pi^*$)

$$\begin{aligned} w &\leftarrow w + \frac{\alpha_w}{B} \sum_{i=1}^B \delta_i \nabla_w Q_w(S_i, A_i), \\ \theta &\leftarrow \theta + \frac{\alpha_\theta}{B} \sum_{i=1}^B \nabla_\theta \mu_\theta^{\text{MPC}}(S_i) \nabla_a Q_w(S_i, a) \Big|_{a=\mu_\theta^{\text{MPC}}(S_i)}, \\ \delta_i &:= l(S_i, A_i) + \gamma Q_w(S_i^+, A_i^+) - Q_w(S_i, A_i), \\ &\text{with } (S_i, A_i, S^+) \sim \mathcal{D}^{\text{buffer}}, A_i^+ = \mu_\theta^{\text{MPC}}(S_i). \end{aligned}$$

The parameters are updated for the deterministic policy μ_θ^{MPC} and an NN critic Q_w with parameters w . The stochastic policy π_θ^{MPC} is used for exploration to fill the replay buffer $\mathcal{D}^{\text{buffer}}$.

8.2.2. MPC as part of the environment

Passing gradients through the MPC in the actor of the closed-loop optimal learning paradigm can also be omitted by conceptually considering the MPC as part of the environment. In such a setting, a parameterized MPC that controls an environment can be seen as part of an augmented new environment whose actions are the MPC parameters. Accordingly, the RL critic evaluates the values associated with these parameters rather than the actions provided by the MPC, which are, in fact, hidden from the RL framework. Considering the MPC's parameterization as the action to the environment, rather than the actions obtained from the MPC, avoids computing sensitivities but potentially vastly increases the dimensions of the action space and requires gradient information obtained through sampling. Such a setting usually involves the hierarchical (Brito et al., 2021; Reiter et al., 2023) or the parameterized architecture (Zarrouki, Wang, & Betz, 2024) shown in Fig. 3.

Whether MPC is considered as part of the environment in literature, involving its differentiation, is indicated by the differentiating MPC column in Table 6.

8.2.3. Exploration

Table 6 lists closed-loop optimal algorithms that differ in their architecture and whether they include differentiation through the optimization layer. Moreover, Table 6 considers how exploration was performed in related literature, cf. Section 3.2. Four choices of exploration are used, particularly in the case of hierarchical and parameterized architectures:

1. Adding noise to the action proposed by the optimization layer. The downside of adding noise posterior to the optimizer is the potentially unsafe exploration, depending on the chosen exploration noise.
2. Modifying the cost in the optimization layer (25a) with an additive term $d^\top u_0$ where d is a possibly randomly selected vector. Given that only the cost is modified, the actions remain feasible with respect to the model, while the additive term introduces a gradient over the initial control input for exploration (Kordabad & Gros, 2023).
3. Add a perturbation to the parameter ϕ for the hierarchical architecture and θ for the parameterized architecture as shown in Fig. 3, guaranteeing safe actions (Gros & Zanon, 2019).
4. Instead of noise on the parameters or controls, optimistic initialization is an RL technique where Q-values (or estimates) are initialized with higher-than-expected values, encouraging the

MPC as reference generator

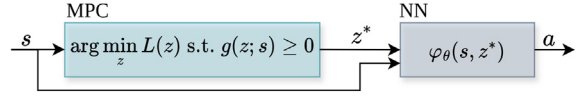


Fig. 6. MPC can be used as a reference generator for an RL policy. The distribution of the input of the policy may depend on the distribution of optimizers obtained from MPC while interacting with the environment.

agent to explore and gradually reduce these optimistic estimates to reflect the true values. For a discussion in the context of deep RL see Rashid et al. (2020).

Closed-loop optimal learning algorithms require the MPC to be part of the RL algorithm. Yet, the following two variants in Section 8.3 use a hierarchical MPC and RL setting, where MPC is added post-training in the deployed controller.

8.3. MPC for pre- and postprocessing

In this section, two concepts are highlighted that utilize MPC within the deployed policy but are not typically used during training. The approaches of Hansen et al. (2022, 2024) are exceptions, since they use RL to obtain a policy that generates initial guesses and which is trained simultaneously as transition models and de/encoders. Since the trained policy is not aware of the filter, its performance cannot be expected to be closed-loop optimal, as in the previous section. Depending on the purpose of the MPC, a distinction is made between an MPC used as a reference generator for preprocessing, as described in Fig. 6 and Section 8.3.1, and an MPC used for postprocessing, cf. Fig. 7 and Section 8.3.2. Relevant literature is compared in Table 7.

8.3.1. Preprocessing: MPC as a reference generator

MPC may be used as a reference generator for an RL policy, such as in Jenelten et al. (2024). This combination approach of MPC and RL is beneficial when the MPC output is a planned trajectory rather than a single action. Since solving the MPC may be slow, the RL policy may be trained with a computationally cheaper reference generator, as proposed in Jenelten et al. (2024). Notably, this setting may often be used in publications without an explicit statement that describes the reference provider.

8.3.2. MPC for postprocessing

Using an MPC layer for postprocessing the output of a trained policy can be used for (i) providing safety with respect to an assumed model and constraints, known as safety filter (Tearle et al., 2021), or (ii) generating a smooth full control trajectory from a coarse solution (Shen & Borrelli, 2023).

As opposed to the previous Section 8.2 of closed-loop optimal learning, the approaches that use MPC as a filter usually do not consider MPC during learning. Thus, the expected behavior may not be closed-loop optimal since the policy is unaware of MPC during training.

Table 7 shows relevant work that uses the MPC for postprocessing and distinguishes the filtering of a single proposed action, e.g., used in the safety-filter framework (Tearle et al., 2021), or a whole trajectory of actions, e.g., Ceder et al. (2024) or filtering of a state and control trajectory as in Grandesso et al. (2023). Filtering action or state trajectories can be interpreted as providing an initial guess for the decision variables of a nonconvex optimization problem to a solver, which then aims to find a reasonable local minimum. In fact, the MPC needs to be approximately aligned with the MDP, and the role of the policy is to assist the MPC optimization solver by finding global/low-cost optimizers.

Table 6

Literature that uses MPC as part of the actor in closed-loop learning. The table lists the applications, whether real-world experiments (RWEs) were performed, and the particular RL and MPC algorithms. The computation of gradients during the back-propagation pass of the RL algorithm may require sensitivities of the MPC solution, as indicated in the column “differentiating MPC”. Alternatively, the sensitivities could be sampled, e.g., by considering the MPC as part of the environment. Similarly, the table compares where the exploration noise of the RL algorithm is added. The noise could be added to the parameters of the MPC optimization problem, to the posterior over the optimal controls provided by the MPC, or via an optimistic initialization of the value functions.

MPC as an actor: closed-loop optimal							
Ref.	Exploration	Diff. MPC	Architecture	Application	RWE	MPC Formulation (Algorithm, Solver)	RL Algorithm
Amos et al. (2018)	actions	yes	parameterized	pendulum, cartpole	no	LMPC (N/A)	IL
Greatwood and Richards (2019)	optimistic	no	hierarchical	drone navigation	yes	LMPC (custom)	TD
Tram et al. (2019)	N/A	no	hierarchical	vehicle intersection	no	LMPC (N/A)	Q-learning
Gros and Zanon (2020a)	actions	yes	parameterized	evaporation process	no	NMPC (SQP, acados)	Q-learning
Brito et al. (2021)	parameters	no	hierarchical	multi-agent unicycle	no	NMPC (N/A, ForcesPro ^a)	PPO
Zanon and Gros (2021)	actions	yes	hierarchical	evaporation process	yes	LMPC (N/A)	Q-learning
Moradimayamnegari et al. (2022)	N/A	yes	parameterized	water tank	yes	NMPC (IP, IPOPT)	SARSA
Brito et al. (2022)	parameters	no	hierarchical	highway traffic	no	NMPC (N/A, ForcesPro)	SAC
Zhang, An et al. (2022)	parameters	no	hierarchical	quadruped robot	no	LMPC (N/A)	
Pfrommer et al. (2022)	actions	no	hierarchical	2D linearized quadrotor	no	LMPC (MOSEK ^b)	PG
Reiter et al. (2023)	parameters	no	hierarchical	autonomous racing	no	NMPC (SQP, acados)	SAC
Liu et al. (2023)	N/A	yes	parameterized	multi-agent games	yes	generalized Nash equilibrium solver (custom)	custom
Acerbo et al. (2024)	N/A	yes	hierarchical	autonomous driving	yes	NMPC (IP, IPOPT)	IL
Romero et al. (2024)	actions	yes	hierarchical	drone racing	yes	NMPC (iLQR, custom)	PPO
Tao et al. (2024)	N/A	yes	parameterized	quadrotor	no	NMPC (SQP, acados)	N/A
Zarrouki, Spanakakis, and Betz (2024)	parameters	no	hierarchical	race car	yes	NMPC, (SQP, acados)	PPO
Zarrouki, Wang, and Betz (2024)	parameters	no	parameterized	race car	yes	NMPC (SQP, acados)	PPO
Wen et al. (2024)	parameters	no	hierarchical	mobile robot	no	LMPC (N/A)	PPO
Rickenbach et al. (2025)	N/A	yes	hierarchical	miniature race cars	yes	NMPC (IP, IPOPT)	IL

Notes: ^aDomahidi and Jerez (2014); ^bApS (2024).

MPC for postprocessing

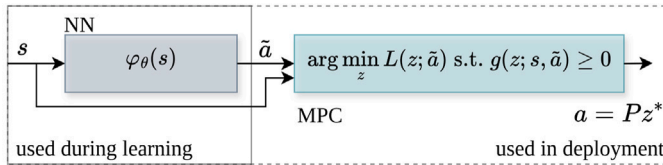


Fig. 7. MPC can be used to smooth or filter reference trajectories. For example, MPC may be used to provide safety guarantees that are hard to achieve by RL policies.

One ought to observe, though, that if an MPC formulation is available to ensure the safety of the action taken in the real environment, typically in the form of a robust MPC scheme, then it is debatable whether training a policy to be filtered by the robust MPC scheme or training the robust MPC scheme directly (Zanon & Gros, 2021) is more effective.

8.4. MPC optimization guided by RL

A conceptually distinct category involves using RL to guide the MPC optimization algorithm. This can, for example, improve online computation time or help escape local minima. Notably, a similar category was found by Bengio et al. (2021) for machine learning algorithms that support combinatorial optimization. Most existing works that guide the MPC optimization algorithm primarily focus on warm-starting the optimization solver (Alboni et al., 2024; Hansen et al., 2022, 2024) to avoid local minima and speed up the solver.

Among those works, a popular model-based RL approach is called TDMPC (Hansen et al., 2022, 2024) where a policy obtained by an online RL algorithm is used to generate initial guesses for warm-starting a sampling-based MPC. Additionally, the transition model, state encoder, and state and reward decoder are trained in a closed loop. Consequently, the algorithm combines guiding the optimization with an initial RL policy guess and learning an aligned latent MPC structure.

Other related categories can be distinguished by the following. Guided policy search (GPS), as discussed in Section 7.2, also leverages policies to warm-start trajectory optimization. However, its objective is to replace the planner entirely with a faster NN. Similarly, MPC post-processing can be interpreted as a form of warm-starting, but with a key semantic difference: post-processing modifies the plan through safety projection, smoothing, or trajectory tracking, rather than improving the numerical optimization itself. Approaches with integrated architectures—for example, those that incorporate a learned terminal value function—can also be viewed as guiding the policy search, as they are iteratively evaluated during the optimization process. However, such methods change the OCP formulation itself rather than help to solve an OCP (see Table 8).

9. MPC as a critic

So far, MPC has been considered as part of the actor or as an expert actor. In the following, we also highlight works that consider MPC as part of the critic, i.e., the MPC is used during training to provide a value function estimate as in the OCPs (22) and (23). Therefore, MPC is used as part of a critic (see Fig. 8).

We distinguish between three variants of MPC as a critic.

A. MPC as an expert critic, Section 9.1. The MPC is parameterized based on expert knowledge of the problem at hand, entering

Table 7

Literature that uses MPC during the deployed algorithm together with the RL policy, but not within the learning phase. Algorithms either use MPC as a reference provider for an RL policy or for postprocessing. Postprocessing is used to ensure safety with respect to a known model and constraints, or to provide an action or state trajectory as an initial guess for the optimization solver.

MPC for Postprocessing					
Ref.	Filtering of	Application	RWE	MPC Formulation (Algorithm, Solver)	RL Algorithm
Li and Bastani (2020)	action	point-mass, kin. vehicle	no	NMPC, robust (N/A)	DDPG
Tearle et al. (2021)	action	miniature race cars	yes	NMPC (SQP, acados)	IL, DAgger
Shen and Borrelli (2023)	traj. roll-out	multi-vehicle motion planning	no	NMPC (IP, IPOPT)	Q-learning
Didier et al. (2023)	action	autonomous driving	no	NMPC (IP, IPOPT)	N/A
Mamedov et al. (2024)	action	flexible robot arm	no	NMPC (SQP, acados)	BC, DAgger, AIRL GAIL, PPO, SAC
Ceder et al. (2024)	action roll-out	mobile robot	no	NMPC (proximal gradient, PANOC ^a)	DDPG
Qu et al. (2024)	traj. roll-out	unmanned aerial vehicle	no	MPPI (custom)	SAC
MPC as Reference Generator					
Jenelten et al. (2024)		quadruped robot	yes	NMPC (TAMOLS ^b)	PPO
Bang et al. (2024)		humanoid robot	no	LMPC (N/A)	PPO
Reiter et al. (2025)	traj. roll-out	vehicle motion planning	no	NMPC (SQP, acados)	PPO, SAC

Notes: ^aStella et al. (2017); ^bJenelten et al. (2022).

Table 8

Literature that uses RL to guide the MPC optimization algorithm. Besides applications and whether RWEs were performed, the changed hyperparameters of the MPC optimization algorithm are indicated.

RL for Guiding the MPC Optimization Algorithm					
Ref.	Changed Hyperparameter	Application	RWE	MPC Formulation (Algorithm, Solver)	RL Algorithm
Hansen et al. (2022)	traj. samples	various robotic tasks	yes	NMPC (MPPI)	DDPG
Grandesso et al. (2023)	initial traj. guess	3-DoF planar manipulator	no	NMPC (IP, IPOPT)	custom AC
Böhn et al. (2023)	MPC recompute flag horizon length	inverted pendulum	no	NMPC (N/A)	PPO
Hansen et al. (2024)	traj samples	various robotic tasks	yes	NMPC (MPPI)	DDPG
Alboni et al. (2024)	initial traj guess	point-mass with obstacles	no	NMPC (IP, IPOPT)	custom AC

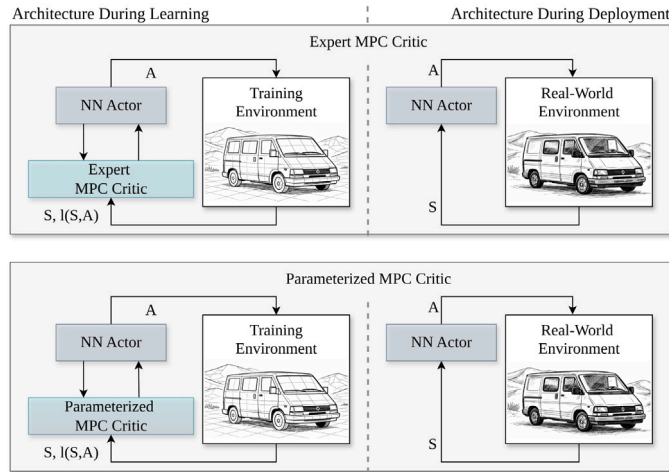
MPC within the Critic (Sect. 9)

Fig. 8. Combinations: MPC as a critic. The plot is divided into two phases: learning and deployment. Grey boxes indicate Neural Networks (NNs), green boxes are used for MPCs, and grey/green boxes refer to parameterized MPCs that involve learned parameters/NNs during the learning phase; see Section 6.1.

through the cost, model, or constraints, which we refer to as expert critic. The motivation here is similar to IL, where the MPC is replaced with a NN for faster computation time and fixed memory footprint, see also Section 7.1.

- B. **MPC as a learnable critic**, Section 9.2. The MPC involves learnable parameters to improve the accuracy of the critic. From

an RL perspective, this approach warm-starts the critic by embedding prior knowledge via the Q-function of the MPC. The critic can, however, be further refined beyond its initial expert configuration.

- C. **MPC as a learnable actor-critic**, Section 9.3. Both the actor and critic incorporate a learnable MPC component to embed additional knowledge on the actor side. The critic parameterization (possibly) differs from that of the actors to allow greater flexibility.

In Table 9, different publications are presented, depending on whether MPC parameters are tuned and which critic type described in the following three sections was used.

9.1. MPC as an expert critic

An MPC scheme, as previously discussed, can readily deliver a value function, as, for example, shown in Ghezzi et al. (2023), Grandesso et al. (2023), Reske et al. (2021). The value function can be used to derive policy gradient equations for training an actor. An important assumption here is that the value function Q^{MPC} approximates Q^* as closely as possible. In fact, the MPC may usually just provide a value function of a desirable good suboptimal policy. Approximating Q^* by the fixed MPC Q-function was recently proposed in Ghezzi et al. (2023). Very related is the line of work in Carius et al. (2020) that, inspired by the Hamiltonian–Jacobi–Bellman equations, uses a first-order approximation of a Q-value function to criticize the actions generated by a learned policy.

Consider the evaluation of the critic, which involves solving the MPC optimization problem to obtain the corresponding action-value function Q^{MPC} . Then, for a deterministic, parameterized policy μ_θ , the learning objective related to (14) is

$$J_{\text{MPC}}^\mu(\theta) = \frac{1}{1-\gamma} \mathbb{E}_{S \sim \rho^{\mu_\theta}} \left[Q^{\text{MPC}}(S, \mu_\theta(S)) \right], \quad (28)$$

with the discounted visitation frequency ρ^{μ_θ} as defined in Section 3.1.3. Note that the policy parameter θ is updated to minimize the objective (28) instead of the behavior cloning objective in Section 7 or the MDP objective in (14).

An update scheme using the Q-function from MPC to criticize the learned policy μ_θ as in Ghezzi et al. (2023) is defined by

DDPG with an MPC expert critic ($\mu_\theta \approx \mu^{\text{MPC}}$)

$$\theta \leftarrow \theta + \frac{\alpha_\theta}{B} \sum_{i=1}^B \nabla_{\theta} \mu_\theta(S_i) \nabla_a Q^{\text{MPC}}(S_i, a) \Big|_{a=\mu_\theta(S_i)},$$

with $S_i \sim \mathcal{D}$,

which is related to the DPG in (16) but with a fixed expert critic. Here, $\mathcal{D}$ could be either a fixed dataset of states or generated iteratively by a mixture of μ_θ and μ^{MPC} like in Ross et al. (2011).

Similar to Section 7, using an MPC as an expert critic leads to imitating and even surpassing the performance of the MPC expert policy. The primary advantage of using the MPC as an expert critic when compared to standard IL that relies on the mismatch between the expert and learned controls is its ability to guide a function approximator with limited expressive power by emphasizing which actions are more important to fit by using the Q-function. It can further introduce the constraint satisfactions to the objective using slacked constraints (Ghezzi et al., 2023).

It is important to emphasize that the MPC expert critic remains fixed and, therefore, does not approximate the action-value function Q^{μ_θ} of the learned policy during training. Indeed, policy gradient methods formally require the critic to be of the policy, i.e., to deliver the policy action-value function Q^{μ_θ} . In contrast, an MPC as an expert critic instead delivers the action-value function Q^{MPC} based on expert knowledge of the environment, intending to approximate the optimal action-value function Q^* as accurately as possible. If the actor has enough flexibility to clone the MPC policy perfectly, then using MPC as an expert critic would ultimately result in the actor matching the MPC performance but not improving it further.

Consequently, as discussed in the following sections, further improvements can be achieved by allowing the MPC critic to adapt to the currently learned policy μ_θ .

9.2. MPC as a learnable critic

In the context of policy gradient methods, a parameterized MPC of any architecture shown in Fig. 3 can be used to approximate the policy action-value function Q^π and be updated using data to capture it as accurately as possible. For instance, the parallel architecture depicted in Fig. 3 is utilized in Bhardwaj et al. (2021). Using the MPC sensitivities, the MPC parameters can be learned using value-based methods from (13). As discussed before, a parametric MPC can fully capture Q^π under the condition that it has a rich parameterization (Adhau et al., 2023; Cai et al., 2023; Esfahani et al., 2023; Seel et al., 2022). This approach can be thought of as introducing expert knowledge into the policy-gradient pipeline, using a critic that harmoniously combines model-based knowledge and learning to approximate Q^π as accurately and quickly as possible. The primary rationale for this combination is that MPC is typically capable of providing a correct structure for the action-value function Q^π prior to any training, thereby providing a good starting point for the policy gradient method. Then, the classic learning of Q^π allows the MPC to serve as a better critic of the policy and to remain a good critic as the policy is modified by the policy-gradient method. It is worth mentioning here that MPC as a learnable critic can be readily combined with a more classic NN, e.g., as a summation of their contribution to the action-value approximation (Bharadhwaj et al., 2020). In that context, the MPC scheme can deliver a broadly correct approximation, while the NN can provide fine corrections. Unlike the approach of Section 9.1, MPC as a learnable critic aligns well with the actor-critic framework.

9.3. MPC as a learnable actor-critic

Using MPC as an actor and as a learnable critic naturally offers the opportunity to combine them into an actor-critic setup using MPC for both. In this setting, MPCs can be used both as an actor, delivering a parameterized policy π_θ^{MPC} to be trained, and as a critic, delivering value functions that are by construction close to the value function of the MPC policy, i.e., $V_\theta^{\text{MPC}} \approx V^{\pi_\theta^{\text{MPC}}}$, $Q_\theta^{\text{MPC}} \approx Q^{\pi_\theta^{\text{MPC}}}$, as opposed to random initializations of NNs.

During the learning, the MPC scheme operating as an actor typically needs to differ from the MPC scheme operating as a critic because π is not a minimizer of Q^π when the policy is not optimal. In practice, two different MPC schemes can be used and trained in parallel: one as a critic, maintaining a good approximation of $Q^{\pi_\theta^{\text{MPC}}}$ associated with π_θ^{MPC} given by (23), and one to support π_θ^{MPC} itself, given by (22). It can be, however, computationally expensive to use two MPC schemes in parallel, as the optimal solution of both must be produced at every training step of the policy, rather than the solution of only the MPC supporting the policy π_θ^{MPC} .

Along that line, Anand et al. (2023) proposed a critic formulation based on the value function obtained from an MPC-based actor. The formulation employed V_θ^{MPC} along with the compatible function approximation suggested in Silver et al. (2014) to build a local approximation of the critic by using a first-order Taylor expansion, simplifying the actor-critic formulation with a single MPC scheme. This approach is effective if the MPC policy is sufficiently close to the optimal policy.

10. Theoretical considerations for combining MPC and RL

MDPs establish a fundamental bridge between RL and MPC, as RL provides tools to solve MDPs. In contrast, MPC formulations can provide approximations, as discussed in the context of aligned learning in Section 8.1 and closed-loop optimal learning in Section 8.2. This section explores the theoretical foundations of combining MPC and RL within the framework of MPC-based MDP approximations. A central theoretical contribution by Gros and Zanon (2020a) provides a theoretical link between economic nonlinear model predictive control (ENMPC) and RL by connecting the MDP value functions and policy discussed in Section 2 with those generated by derivative-based MPC formulations detailed in Section 4.2.2. This connection provides theoretical justification for approaches that incorporate MPC as either an actor or critic, as described in Section 8 and Section 9, respectively. We begin by presenting this key theoretical result, followed by an outline of its subsequent developments and a summary in Table 10.

Following the definition in Section 3.1.3, let $p_{f^{\text{MPC}}}(s_0 \rightarrow s^+, k, \pi_\theta)$ denote the probability of reaching state s^+ at step k when starting from the initial state s_0 and following the policy π , under model dynamics f^{MPC} . A key assumption in Gros and Zanon (2020a), Kordabad et al. (2024) is that the optimal value function remains bounded under the optimal policy and model of the dynamics:

Assumption 10.1 (Kordabad et al. (2024, Assumption 1)). The set S is non-empty for a given $\bar{N} \in \mathbb{N}$, with

$$S := \left\{ s \in \mathcal{X} \mid \left| \mathbb{E}_{S \sim p_{f^{\text{MPC}}}(s_0 \rightarrow s^+, k, \pi^*)} [V^*(S)] \right| < \infty, \forall k \leq \bar{N} \right\}. \quad (29)$$

The authors of Gros and Zanon (2020a) demonstrate that discounted MPC can capture the optimal value functions and policy of a discounted MDP through modifications of the stage and terminal costs, even when the model f^{MPC} differs from the true state-transition function. This result was later extended in Kordabad et al. (2024) to undiscounted MPC by the following Theorem, establishing a central link to classical MPC stability theory:

Theorem 10.1 (Kordabad et al. (2024, Theorem 1)). Suppose that Assumption 10.1 holds for $\bar{N} \geq N$. Then, there exists a terminal cost $\bar{V}_\theta^{\text{MPC}}$ and a stage cost l_θ^{MPC} such that the following identities hold, for all γ , $N \in \mathbb{N}$ and $s \in S$:

Table 9

Literature that uses MPC as a critic for RL. The MPC parameters are either learned within the RL algorithm or fixed. Further, a distinction is made, whether the critic is a fixed expert, see Section 9.1, a more flexible learned critic as described in Section 9.2, or is part of both the actor and the critic, Section 9.3.

MPC as a Critic						
Ref.	Learned MPC Parameters	Critic Type/ Par. MPC Arch.	Application	RWE	MPC Formulation (Algorithm, Solver)	RL Algorithm
Reske et al. (2021)	no	expert/N/A	quadruped robot	yes	NMPC (DDP, N/A)	IL
Bhardwaj et al. (2021)	no	learned / parallel	several robotic in-hand manipulation	no	MPPI	TD
Ghezzi et al. (2023)	no	expert/N/A	cart pole	no	NMPC (SQP, acados)	IL
Anand et al. (2023)	yes	Actor-Critic / parameterized	cart pole	no	NMPC (IP, IPOPT)	DPG
Grandesso et al. (2023)	no	expert/N/A	3-DoF planar manipulator	no	NMPC (IP, IPOPT)	custom AC
Carius et al. (2020)	no	expert/N/A	quadruped robot	yes	NMPC (DDP, N/A)	IL
Alboni et al. (2024)	no	expert/N/A	Dubin's car parking	no	NMPC (DDP, IP, IPOPT)	custom AC

1. $\pi_{\theta}^{\text{MPC}}(s) = \pi^*(s)$,
2. $V_{\theta}^{\text{MPC}}(s) = V^*(s)$,
3. $Q_{\theta}^{\text{MPC}}(s, a) = Q^*(s, a)$, for the inputs $a \in \mathcal{U}$ such that $\left| \mathbb{E}_{S^+ = f^{\text{MPC}}(s, A), A \sim \pi_{\theta}^{\text{MPC}}(\cdot|s)} [V^*(S^+)|s, A] \right| < \infty$.

Proof. See Kordabad et al. (2024).

Given the conditions in Theorem 10.1, we classify MPC formulations that satisfy condition (1) as closed-loop optimal, where the learning process that aims at satisfying condition (1) using MPC within the actor is closed-loop optimal learning, see 8.2. When both conditions (1) and (2) are satisfied, the MPC is MDP complete since it captures both the optimal policy and optimal action-value function of an MDP. At the top of this theoretical hierarchy lies the MDP aligned property, which demands that the MPC model exactly matches the state-transition kernel of the MDP. However, this property is rarely satisfied in practice, as MPC formulations typically employ deterministic models, while MDP state transitions are inherently stochastic. In the following, we survey literature that provides theoretical contributions on the combination of MPC and RL.

10.1. Extensions of MPC-MDP equivalence

Theorem 10.1 was originally developed in the context of ENMPC. Subsequent research has extended the result to several MPC variants, including real-time iteration MPC (Zanon et al., 2020), mixed-integer MPC (Gros & Zanon, 2020), scenario-based MPC (Kordabad, Esfahani, et al., 2021), or any model-based policy (Anand et al., 2025).

Instead of using OCP formulations that rely on one-step prediction rollouts of the environment, Sawant and Gros (2022) explores the connection between MDPs and QP formulations that arise for tracking problems with linear dynamics or as subproblems in OCP solvers. By parameterizing the QP directly rather than the prediction model, this approach offers additional flexibility that can enhance the performance of the MPC policy. Building on this idea, Sawant et al. (2023) incorporates formulations from subspace predictive control (SPC) (Favoreel et al., 1999), showing that the past input/output sequences can serve as a surrogate state from which future predictions are built. While this approach effectively handles systems without state estimators, its application is limited to problems permitting autoregressive linear maps from inputs and outputs to future trajectories.

For systems requiring state estimation, an estimation layer such as moving horizon estimation (MHE) can be integrated to provide state feedback to the MPC. Research by Esfahani et al. (2023, 2021) demonstrated that jointly optimizing the parameters of both the MHE and MPC leads to significantly better performance compared to optimizing MPC parameters alone. The previously discussed challenges of differentiation through optimization layers and computational complexity in the forward pass extend naturally to the MHE layer.

The use of MPC and safety filters to ensure the safety of learned policies has emerged as a prominent research direction. One common

approach is to project control actions from trained policies onto a safe set by minimizing the distance to feasible actions that satisfy the model dynamics and safety constraints. The authors of Wabersich et al. (2023), Wabersich and Zeilinger (2021) exemplify this strategy by introducing a predictive safety filter that post-processes actions from trained RL policies. However, this projection step may lead to suboptimal actions when the agent is unaware of the filter, as discussed in Gros, Zanon, and Bemporad (2020). An approach to learning safe and stable policies by construction via robust MPC within the policy is discussed in Zanon and Gros (2021) and further developed in Gros, Zanon (2022b), Kordabad, Bahari and Wisniewski, and Gros (2022).

10.2. Approximating discounted MDP with undiscounted MPC

Research exploring the relationship between MPCs and MDPs has focused on identifying conditions under which an infinite-horizon discounted MDP:

$$V^*(s) := \min_{\pi} V^{\pi}(s) := \mathbb{E}_{\tau^{\pi}} \left[\sum_{k=0}^{\infty} \gamma^k \ell(s_k^{\pi}, \pi(s_k^{\pi})) \right]$$

is equivalent to a finite-horizon undiscounted OCP typically used in MPC:

$$\hat{V}_N^*(s) := \min_{\pi} \hat{V}_N^{\pi}(s) := \mathbb{E}_{\hat{\tau}^{\pi}} \left[\hat{T}(\hat{s}_N^{\pi}) + \sum_{k=0}^{N-1} \hat{L}(\hat{s}_k^{\pi}, \pi(\hat{s}_k^{\pi})) \right]$$

where $\tau^{\pi} := p_P(s_0 \rightarrow s^+, k, \pi^*)$ and $\hat{\tau}^{\pi} := p_{f^{\text{MPC}}}(s_0 \rightarrow s^+, k, \pi^*)$ denote the likelihood of trajectories under the true MDP dynamics $s_{0,1,\dots}^{\pi}$ and the MPC model $\hat{s}_{0,1,\dots}^{\pi}$, respectively. Here, $\hat{T}$, $\hat{L}$, $\hat{V}_N^*$, and $\hat{V}_N^{\pi}$ represent the terminal cost, stage cost, optimal value function, and policy value function associated with model f^{MPC} . The equivalency draws from dissipativity theory (Gros, Zanon, 2022a), a key concept in ENMPC stability analysis. Using dissipativity arguments, Kordabad, Bahari, and Gros (2021) investigates undiscounted Q-Learning of tracking MPC schemes that are locally equivalent to dissipative ENMPC, focusing on learning storage functions that maintain ENMPC dissipativity. The authors of Zanon et al. (2022) prove that under weak stability conditions, the optimal policy of undiscounted and discounted MDPs coincide — enabling stable undiscounted MPCs to yield the optimal policy of stable discounted MDPs. Recent advances by Kordabad et al. (2024) extend these results to unknown dynamics, demonstrating that undiscounted MPCs can capture optimal policies of discounted MDPs, as discussed around Theorem 10.1. Complementary work in Kordabad and Gros (2023) introduces methods to align undiscounted MPC cost function minimizers with those of discounted MDPs. These contributions share a common thread: by enforcing parameter updates that yield dissipative ENMPC formulations, stable ENMPC policies can be designed constructively, rather than relying solely on dissipativity as a verification criterion.

Table 10

Literature that discusses theoretical aspects of the connection between MPC and RL.

Ref.	Contribution
MPC as a Model of the MDP	
Görges (2017)	Compares explicit MPC for linear input and state-constrained systems to function approximation by NN in the context of RL
Zanon et al. (2019)	Economic nonlinear model predictive control (ENMPC) as function approximation
Gros and Zanon (2020a)	Fundamental principles for ENMPC to approximate MDP and its connection to RL
Zanon et al. (2020)	Extends (Gros & Zanon, 2020a) to Real-Time Iteration NMPC
Gros and Zanon (2020)	Extends (Gros & Zanon, 2020a) to Mixed-Integer MPC
Kordabad, Esfahani, et al. (2021)	Extends (Gros & Zanon, 2020a) to scenario-based MPC
Esfahani et al. (2021)	Use combined parameterized MHE and parameterized MPC as a function approximation in RL
Sawant and Gros (2022)	Extends (Gros & Zanon, 2020a) to generic parameterized QP layers instead of MPC
Sawant et al. (2023)	Extends to subspace predictive control (SPC) using input–output data as surrogate state
Seel et al. (2022)	Extends the cost to convex artificial neural network (NN) formulations to facilitate a more complex function approximation
Anand et al. (2023)	Proposes deterministic policy gradient to train MPC via a parameter-perturbed variant as the critic.
Esfahani et al. (2023)	Use combined parameterized MHE and parameterized MPC (extension of Esfahani et al. (2021))
Post-processing and Safety	
Gros, Zanon, and Bemporad (2020)	Discusses the impact of safe policy projections, cf. Wabersich and Zeilinger (2021), on learning and proposes corrections in the context of Q-learning
Zanon and Gros (2021)	Addresses safety in RL via robust MPC and safe parameter updates
Wabersich and Zeilinger (2021)	Address safety via MPC-based predictive safety filters that modify unsafe actions from RL policies
Wabersich et al. (2023)	Address safety via MPC-based predictive safety filters that modify unsafe actions from RL policies
Gros, Zanon (2022b)	Studies safety and stability of MPC schemes subject to parameter updates by an RL policy
Kordabad, Bahari and Wisniewski, and Gros (2022)	Use Wasserstein Distributionally Robust MPC as a tool to generate safe RL policies
Approximation Theory	
Kordabad, Bahari, and Gros (2021)	Proposes Q-learning to modify the storage function in ENMPC such that it meets dissipativity criteria.
Kordabad, Nejatbakhsh Esfahani and Gros (2021)	Address the problem that MPC-based policies with hard constraints can lead to biased gradient estimates in actor–critic methods.
Gros, Zanon (2022a)	Extends dissipativity theory of ENMPC to a wider class of probabilistic dynamics to connect to undiscounted MDPs.
Zanon et al. (2022)	Connects discounted and undiscounted MDPs with known dynamics to generate stability-constrained optimal policies through MPC.
Kordabad and Gros (2023)	Proposes a cost modification for discounted ENMPC such that stage-cost minimizer coincides with the undiscounted case.
Seel et al. (2023)	Proposes a combination of policy gradient and Q-learning to exploit gradient information and capture cost function properties not affecting the policy.
Kordabad et al. (2024)	Extends (Zanon et al., 2022) to mismatching dynamics and extends (Gros & Zanon, 2020a) to capture a discounted MDP via undiscounted MPC.

10.3. State-transition model for closed-loop optimality

Traditional MPC applications select prediction models by minimizing the identification loss (27), along with properties such as convexity or smoothness. However, models achieving good mean-error predictions or maximum-likelihood models may not necessarily guarantee closed-loop optimality. Recent research by Anand et al. (2024), Reinhardt et al. (2024) establishes formal requirements for prediction models in (23) to achieve closed-loop optimality and capture optimal value functions. They demonstrate that for a model f^{MPC} to deliver both the optimal policy π^* and optimal action-value function Q^* , it must satisfy

$$\mathbb{E}_{S^+ \sim P(\cdot|s, a)} [V^*(S^+) | s, a] - V^*(f^{\text{MPC}}(s, a)) = V_0, \quad (30)$$

where V_0 is a constant. While (30) does not directly yield a system identification procedure, it provides insights into models minimizing (27).

Such models prove optimal for deterministic MDPs and LQR problems. Local optimality can be established for tracking problems, set-point stabilization, and dissipative economic problems by bounding the conditional covariance of the state transition dynamics P given in (1) and the curvature of the (locally smooth) optimal value function V^* . However, models minimizing (27) may not yield optimal policies for problems with non-smooth cost functions or dynamics, non-dissipative MDPs, or strong disturbances.

The authors of Nikishin et al. (2022) approach the problem of model design by formulating an equivalent expression to (30) in terms of the Bellman equation, introducing the optimal model design (OMD) method for simultaneous learning of Q-functions and models. This connects to broader developments in RL, such as the learned hidden state dynamic models in Schrittwieser et al. (2020), which focus on capturing only the state information essential for optimal actions and rewards. While Anand et al. (2024, 2025), Reinhardt et al. (2024) focus

on the theoretical properties of models in MPC contexts, Nikishin et al. (2022) provides algorithmic tools for their construction.

11. Software tools and implementation aspects

This section highlights challenges in practical implementations and points to available software solutions for combining MPC and RL.

11.1. Integrating software from machine learning and numerical optimization

The architectural combinations of NNs and MPCs described in Section 6 and illustrated in Fig. 3 significantly influence the choice of optimization software, particularly for derivative-based MPC solvers. When objective or constraint functions are highly nonlinear, the resulting optimization problems become nonconvex and computationally challenging. Furthermore, the selected software must be compatible with machine learning libraries to enable seamless integration.

To address these challenges, recent open-source tools have emerged. The package `l4casadi` (Salzmann et al., 2023) enables the integration of NNs, i.e., $\varphi_\theta(s, z)$ in Fig. 3, within the automatic differentiation tool `CasADi` (Andersson & Rawlings, 2018). Similarly, `l4acados` (Lahr et al., 2024) bridges learned models from PyTorch to acados, focusing on residual models based on GPs in an integrated approach with $\phi = \varphi_\theta(z; s)$. While these tools address derivative-based optimization, sampling-based MPC solvers offer an alternative approach, as they are less sensitive to nonlinearities and require only forward simulation.

11.2. Implementation aspects

Besides the difficulty of solving an optimization problem involving NNs with derivative-based solvers, several additional issues arise when combining MPC and RL, which should be considered in the implementation.

1. *Differentiation*: The MPC solver differentiation requires the computation of sensitivities - a process that is theoretically straightforward but demands existing solvers to provide an efficient implementation of the parametric sensitivity computations. Various software packages allow for the differentiation of optimization problems. The package `PyPose` (Wang et al., 2023) provides differentiable optimization tailored for robot learning. The software package `Theseus` (Pineda et al., 2022) is dedicated to differentiable nonlinear optimization. The authors in Frey et al. (2025) and Drghona et al. (2023) propose a software implementation of differentiable optimization tailored to nonlinear MPC.
2. *Solver States*: MPC solvers often introduce additional solver states that must be incorporated into an augmented MDP. Nonlinear optimization processes start from initial guesses and may converge to different local minima, making these initial guesses part of the state space. This becomes particularly relevant in the RTI scheme (Diehl, 2002), cf. Section 4, where the solver tracks optima across iterations rather than achieving full convergence. These additional solver states increase the complexity of the MDP.
3. *Computational Efficiency*: Some RL algorithms, such as SAC, require random sampling from replay buffers during learning. Computing gradients for each sample necessitates multiple solutions to optimization problems, resulting in significant computational overhead.
4. *Hardware Utilization*: While machine learning frameworks benefit from parallelizable architectures, MPC solvers typically target CPU-based embedded applications. Efforts to leverage GPUs or parallel CPU implementations represent an emerging research direction, with recent developments including GPU-based sampling MPC (Vlahov et al., 2024) and multi-core CPU implementations for derivative-based MPC in acados (Verschuere et al., 2022).

Complementary to this survey, early-stage software development enables rapid prototyping of many of the algorithms discussed. The software, publicly available as `Learning for Predictive Control (leap-c)`,³ enables seamless integration of acados into PyTorch as a general differentiable layer, using an efficient multi-threaded implementation to compute batched inputs effectively (Frey et al., 2025). The implementation supports differentiation not only with respect to control and state trajectories but also the Q-function of MPC, enabling the use of critic-based schemes as discussed in Section 9. The current focus of `leap-c` is on closed-loop learning, as described in Section 8.2. Further software projects exist that focus on providing valuable educational examples, albeit with a lower degree of integration with fast OCP solvers.⁴

12. Conclusion and discussion

Model predictive control (MPC) and reinforcement learning (RL) each require extensive background knowledge, as detailed in Sections 3 and 4, and exhibit complementary properties as highlighted in Section 5. The latter raises the question of how to effectively combine these approaches to leverage their respective strengths while mitigating their limitations. In Section 6, various possible combinations within the framework of solving Markov decision processes (MDPs) through actor-critic RL are demonstrated. In the following, we conclude and discuss these combinations, as well as the related architectures of how artificial neural networks (NN) are integrated into MPC, their advantages, and alternative classifications. Finally, we provide an outlook for promising future research.

Combining MPC and RL. Within this framework, MPC can serve as an algorithmic part in multiple roles. One role is as a supervisory expert actor in IL or GPS frameworks as discussed in Section 7. This allows for the inclusion of domain knowledge and other features to develop computationally efficient policies for online deployment.

Another role for MPC is as a part of the policy — the most prevalent approach in the literature, as shown in Section 8. The categorization becomes more intricate in this case. We distinguish between two main directions: MDP aligned approaches that aim at modifying the MPC to reflect the MDP structure in terms of cost and state transitions, and closed-loop optimal approaches that focus on the closed-loop optimality of MPC. In both cases, the MPC enters the learning pipeline via an optimization layer that may be expensive to evaluate in the forward inference path or to differentiate in the backpropagation path to provide the respective gradients, cf. Appendix A. A challenge concerning the implementation is that the computation of gradients is often not provided by optimization solvers, despite recent implementations (Drghona et al., 2023; Frey et al., 2025; Pineda et al., 2022; Wang et al., 2023) discussed in Section 11. It is possible to avoid the gradient computation of the MPC solution by integrating it into the environment (cf. Section 8.2) such that its parameterization becomes part of the action space. The critic then evaluates the parameterization of the MPC as opposed to the actions provided by the MPC output. Considering MPC as part of the environment and evaluating the critic on the MPC parameterization can be viewed as an example of the second practical architecture we emphasize: architectures that use the MPC solver within the forward path but do not require its differentiation. Other architectures that avoid differentiation include using an expert MPC (Section 7), using a fixed MPC critic (Section 9), and on-policy learning of the model and the terminal value function in the MDP aligned structure (Section 8.1). These can be seen as variants that do not require differentiation but use MPC as part of the forward path. Still, the repeated solutions of optimization problems make learning

³ <https://github.com/leap-c/leap-c>

⁴ <https://github.com/FilippoAiraldi/mpc-reinforcement-learning>

iterations significantly more expensive, and implementation aspects and tailored software are required, as outlined in Section 11.

The last role of MPC in the RL framework is that of a (learnable) critic, as discussed in Section 9, which is not as prominent in the literature but may have the advantage of building explainable value function estimates and significantly improving the sample efficiency and avoidance of local minima of RL training. Again, repeated MPC evaluations increase the computational cost compared to NN layers.

Beyond these categories, several variants exist in the literature where MPC is not used during RL training but where MPC is solely used in the final deployed policy. These include off-policy learning of the model and some algorithms that learn a terminal value function in the MDP aligned structure (Section 8.1) as well as pre- and post-processing components (Section 8.3). In the latter, MPC can function as a safety filter to enhance trained policies with its inherent constraint satisfaction capabilities (Hewing, Kabzan, & Zeilinger, 2020) and as a reference provider (Section 8.3) - a role that may extend to the learning phase as well.

Implementation architectures. Another critical design decision is the combination architecture of NNs and MPCs presented in Section 6 for which we identified the integrated, algorithmic, hierarchical, parallel, or parameterized approaches outlined in Fig. 3. NNs are usually highly nonlinear functions that deserve a distinction from moderately nonlinear functions used as part of derivative-based MPC. In Section 6, we contrast between using NNs outside the MPC, i.e., they do not depend on decision variables of the optimization problem, using NNs inside the MPC optimization problem, or avoiding NNs and using simpler linear or quadratic functions.

Advantages and disadvantages of combinations. As outlined in the previous chapters, combining MPC and RL can offer several advantages over the separate use of either, such as sample efficiency, safety, and closed-loop optimality. Based on the introduced classification and architectures, several qualitative statements can be made regarding the advantages and disadvantages.

First, assuming a correct model and a conservative safe set, using MPC within the actor, in the integrated, hierarchical, or parameterized structure, with exploration of the MPC parameters, can be considered safe. Instead, using MPC as an expert within the actor in the parallel structure, or as the critic, is generally not safe.

Secondly, only specific variants converge to the closed-loop optimal solution. The aligned formulation typically does not plan over policies but rather open-loop rollouts, and is therefore usually suboptimal. A closed-loop formulation is capable of converging to the optimal policy even though the terminal value function or model is wrong.

Regarding the explainability of a policy, the aligned formulation provides insights, as the model aims to capture the true MDP dynamics. Closed-loop optimal formulations, where MPC parameters or a residual term are determined by a trained NN, provide less insight. For instance, in certain states, the MPC parameters may be altered more extensively than in others, indicating some MPC misalignment. However, usually the model used in the MPC is a coarse approximation, and the parameters set by the NN can be hard to read (Romero et al., 2024).

Most variants that use MPC are data-efficient, depending on how well the model captures the true MDP dynamics. Differentiating the MPC layer to update the parameters in hierarchical MPC RL architectures can improve the sample efficiency. However, a rigorous study of the significance of first-order backpropagation through the MPC layer compared to sampling the parameters is not available.

Alternative classification. With the proposed classification, we incorporate several relevant conceptual and algorithmic distinctions:

- Does the MPC formulation model the MDP? (aligned vs. closed-loop learning, Section 8)

- Is MPC used to evaluate the cost, to provide the next action, or to provide expert trajectories? (MPC in the actor vs. MPC in the critic, Fig. 2)
- Is MPC used during learning or deployment? (Fig. 2)
- How are highly nonlinear neural networks combined with the optimization algorithms? (Architectures, Section 6.1)

The chosen classification of MPC and RL synthesis approaches provides a structured answer to these questions, extending previous related survey literature (Hewing, Wabersich, et al., 2020; Mesbah et al., 2022; Moerland et al., 2023).

With a focus on safe learning, the authors in Hewing, Wabersich, et al. (2020) present early work that does not consider concepts, such as closed-loop learning, MPC used to evaluate the value function, or MPC experts.

In general, related overviews do not focus specifically on variants for RL and MPC synthesis (Hewing, Wabersich, et al., 2020; Mesbah et al., 2022). The authors of Hewing, Wabersich, et al. (2020), Mesbah et al. (2022) pivot around Bayesian optimization, specifically in closed-loop learning. Due to its different focus, the authors in Mesbah et al. (2022) also do not provide a categorization when MPC is used as part of the critic, for preprocessing, or safety filtering of trained policies, or refining initial guesses provided by RL policies. We propose a framework specifically targeted for RL, excluding Bayesian optimization, and make the key distinction between closed-loop learning and aligned learning, which renders “model learning” and “terminal value function learning” mentioned in Mesbah et al. (2022) in our novel category “aligned learning”. The categorization in Mesbah et al. (2022) can be mapped to our broader proposed framework.

The authors of Moerland et al. (2023) survey how models and planning are used in RL and consider planning procedures to be much more general. For instance, they include discrete planning and do not specifically mention MPC. While in principle optimization-based MPC falls under the “differential planning” category of Moerland et al. (2023), no details are explicitly provided about MPC. The authors of Moerland et al. (2023) draw a primary distinction between model-based RL, where a model and a global solution are learned, e.g., as in Dyna (Sutton, 1991), and model-based RL, with a known model to obtain the global solution in the form of a value function or policy, e.g., as in dynamic programming (DP). We find this distinction less useful in the context of MPC, as the classification would specify a learned terminal value function as a global approximation.

12.1. Future research directions

Several promising research directions emerge for advancing the combination of MPC and RL:

Expanding to broader decision-making frameworks. A natural extension involves generalizing from MPC to encompass all forms of repeated planning, including combinatorial optimization, stochastic multi-stage programming, and mixed-integer linear programming (mixed-integer linear program (LP) (MILP)). Many domains employ MPC-like approaches without explicitly labeling them as such, presenting opportunities for cross-fertilization of ideas.

Computational efficiency for large-scale problems. While our current approaches demonstrate computational feasibility, scaling to high-dimensional offline RL with massive datasets remains challenging. GPU-based MPC implementations and more efficient offline learning algorithms could accelerate training and enable broader applications.

Risk-aware learning. Current MPC formulations excel at constraint handling but could better represent risk through more sophisticated metrics. Moving beyond robust and stochastic MPC towards constrained MDPs with probabilistic constraints — such as mission completion probabilities — would enhance safety-oriented applications.

Multi-agent systems. Despite existing work, multi-agent RL with MPC contains significant unresolved questions regarding coordination, communication, and distributed decision-making. Research activity in this direction would involve modeling the solution of the MDP of a multi-agent RL problem as a distributed MPC, which is a significant challenge.

Application domains. The practical impact spans multiple sectors. Energy systems and process industries offer immediate opportunities given the established presence of MPC. Healthcare applications in patient navigation and personalized medicine could benefit from repeated planning approaches. Logistics and supply chain optimization naturally align with these methods, while automotive applications may focus more on safety guarantees than pure optimality. The theoretical foundations established in this survey, combined with emerging software tools and increasing computational resources, position the field for significant advances. As the community develops more sophisticated integration methods and demonstrates success across diverse domains, the synthesis of MPC and RL will likely transition from a research subject to a standard tool in the control engineering toolkit.

Outlook. As indicated in this survey, the expectations for the superiority of MPC and RL combinations are high, but challenges remain. Theoretical work already provided a solid foundation, cf. Section 10. Nonetheless, practical work that demonstrates significant improvements across various domains is still required. Most importantly, the practical work requires numerical solvers to align with machine learning frameworks and tools that allow fast MPC solvers to be embedded in learning frameworks, see Section 11. With the continued development of such tools, the outlook for the MPC-RL combination is promising. In future work, we hope the community will investigate whether the control paradigm shifts from MDP aligned learning to closed-loop optimal learning and whether the sampling complexity and other challenges of RL can be improved by MPC optimization modules.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Differentiating through a parameterized MPC layer

In the following, we discuss how to compute gradients with respect to parameters θ within the architectures introduced in Fig. 3. The availability of gradient information is a crucial prerequisite for the learning approaches discussed in this paper. We restrict the discussion to the parameterized MPC formulation of Fig. 3, where the corresponding solution map is defined by

$$z^*(s, \theta) := \underset{z}{\operatorname{argmin}} L(z; \theta) \text{ s.t. } g(z; s, \theta) \geq 0. \quad (\text{A.1})$$

Assuming that the objective is twice, the constraints once, continuously differentiable, the implicit function theorem (IFT) guarantees the existence of the solution sensitivities $\frac{\partial z^*}{\partial \theta}(\bar{s}, \bar{\theta})$ at $(\bar{s}, \bar{\theta})$ if the linear independence constraint qualification, second-order sufficient conditions, and strict complementarity are satisfied at the solution $z^*(\bar{\theta})$. Under these conditions, the IFT furthermore implies that the solution sensitivity can be computed as

$$\begin{bmatrix} \frac{\partial z^*}{\partial \theta}(\bar{s}, \bar{\theta}) \\ \frac{\partial \mu_A^*}{\partial \theta}(\bar{s}, \bar{\theta}) \end{bmatrix} = - \left(\frac{\partial r}{\partial (z, \mu_A)} \right)^{-1} \frac{\partial r}{\partial \theta} \quad (\text{A.2})$$

where μ_A are the dual variables associated with the active constraints at the solution $z^*(\bar{s}, \bar{\theta})$ and where the partial derivatives of the residual map $r(z, \mu_A; s, \theta)$ are evaluated at $z = z^*(\bar{\theta}, \bar{s})$, $\mu_A = \mu_A^*(\bar{s}, \bar{\theta})$, $s = \bar{s}$, and $\theta = \bar{\theta}$. The residual map is defined as

$$r(z, \mu_A; s, \theta) = \begin{bmatrix} \nabla_z L(z; \theta) + \nabla_z g_A(z; s, \theta) \mu_A \\ g_A(z, \mu_A; s, \theta) \end{bmatrix} \quad (\text{A.3})$$

where $g_A(z; s, \theta)$ denotes all equalities as well as all inequalities that are active at $z^*(\bar{s}, \bar{\theta})$. A formal proof of the above statement is given, for instance, in Still (2018).

At an active set change, the solution map is not differentiable. Within the stochastic optimization framework, this nondifferentiability of the solution map for some values of θ and s is, in general, not problematic, as the gradient needs to be well-defined only almost everywhere.

A standard approximation replaces the residual map in (A.3) with a smoothed approximation of the KKT conditions associated with (A.1), as is done within an interior point solver, which leads to a natural smoothing of the solution map, alleviating the problem of nondifferentiability at active set changes (Gros & Zanon, 2020a).

Appendix B. Gradients in the hierarchical architecture

The following discussion is restricted to the hierarchical NN-MPC architecture of Fig. 3. At each decision step, the NN predicts the parameter ϕ , which is then processed by the MPC-optimization layer to generate a control $u_0^*(\phi)$. We distinguish the learning approaches based on the actor's feedback. The value function, i.e., the critic, is evaluated for the predicted parameter ϕ or the resulting control $u_0^*(\phi)$. Note that this distinction is first and foremost important for the optimization properties of the NN component or when safe exploration is required during training.

B.1. Parameter critic

Assuming that the critic gives feedback based on the predicted parameter ϕ , the optimization layer can be considered part of the environment (Reiter et al., 2023). Thus, an altered version of the original MDP can be defined by introducing a modified MDP transition model

$$P^{\text{MPC}}(s^+ | s, a) := P(s^+ | s, u_0^*(a)). \quad (\text{B.1})$$

One of the benefits of this approach is that implementations of RL methods can be used directly without further adjustments, as the MPC optimization is performed only when generating new samples by forward simulating the system from a given state to the next using (B.1).

B.2. Control critic

Assuming that the critic gives feedback based on the control of the MPC optimization layer u_0^* , there are multiple ways to obtain a gradient. One can avoid differentiating through the MPC optimization layer

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\psi \sim \pi_{\theta}(s), s \sim D} [Q(s, u_0^*(\psi)) \nabla_{\theta} \log \pi_{\theta}(\psi)],$$

by extending the stochastic policy gradient (Gros & Zanon, 2021). As there is no gradient information, using such a policy gradient can be seen as black box optimization with respect to the MPC optimization layer. This can be especially problematic for high-dimensional parameter spaces, as discussed in Silver et al. (2014).

Alternatively, one can differentiate through the MPC (Gros & Zanon, 2020a) extending the deterministic policy gradient (16) to derive a policy gradient with

$$\nabla_{\theta} J^{\pi}(\theta) = \mathbb{E}_{s \sim D} \left[\nabla_{\theta} \mu_{\theta}(s) \nabla_{\psi} Q^{\mu}(s, u_0^*(\psi)) \Big|_{\psi = \mu_{\theta}(s)} \right].$$

Data availability

No data was used for the research described in the article.

References

- Moraes, C. G. d. S., Waltrich, G., Jones, C., & Maddalena, E. (Eds.), (2020). A Neural Network Architecture to Learn Explicit MPC Controllers from Data. *Ifac PapersOnline*.
- Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., & Riedmiller, M. (2018). Maximum a Posteriori Policy Optimisation. In *International conference on learning representations*.
- Acerbo, F. S., Swevers, J., Tuytelaars, T., & Son, T. D. (2024). Driving from vision through differentiable optimal control. In *2024 IEEE/RSJ international conference on intelligent robots and systems* (pp. 2824–2831).
- Achiam, J., Held, D., Tamar, A., & Abbeel, P. (2017). Constrained Policy Optimization. In *Proceedings of the 34th international conference on machine learning* (pp. 22–31). PMLR, ISSN: 2640-3498.
- Adhau, S., Reinhardt, D., Skogestad, S., & Gros, S. (2023). Fast Reinforcement Learning Based MPC based on NLP Sensitivities. *IFAC-PapersOnLine*, 56(2), 11841–11846.
- Ahn, K., Mhammedi, Z., Mania, H., Hong, Z.-W., & Jadbabaie, A. (2023). Model Predictive Control via On-Policy Imitation Learning. In *Proceedings of the 5th annual learning for dynamics and control conference* (pp. 1493–1505). PMLR, ISSN: 2640-3498.
- Åkesson, B. M., & Toivonen, H. T. (2006). A neural network model predictive controller. *Journal of Process Control*, 16(9), 937–946.
- Alboni, E., Grandesso, G., Papini, G. P. R., Carpentier, J., & Del Prete, A. (2024). CACTO-SL: Using Sobolev learning to improve continuous actor-critic with trajectory optimization. In *Proceedings of the 6th annual learning for dynamics & control conference* (pp. 1452–1463). PMLR, ISSN: 2640-3498.
- Allan, D. A., Bates, C. N., Risbeck, M. J., & Rawlings, J. B. (2017). On the inherent robustness of optimal and suboptimal nonlinear MPC. *Systems & Control Letters*, 106, 68–78.
- Altman, E. (1999). *Constrained Markov decision processes* (1st ed.). New York: Routledge.
- Amos, B., Jimenez, I., Sacks, J., Boots, B., & Kolter, J. Z. (2018). Differentiable MPC for End-to-end Planning and Control. In *Advances in neural information processing systems: Vol. 31*, Curran Associates, Inc..
- Anand, A. S., Reinhardt, D., Sawant, S., Gravadahl, J. T., & Gros, S. (2023). A Painless Deterministic Policy Gradient Method for Learning-based MPC. In *European control conference (ECC)* (pp. 1–7).
- Anand, A. S., Sawant, S., Reinhardt, D., & Gros, S. (2024). Data-Driven Predictive Control and MPC: Do we achieve optimality?
- Anand, A. S., Sawant, S., Reinhardt, D., & Gros, S. (2025). All AI Models are Wrong, but Some are Optimal. arXiv:2501.06086 [cs].
- Anderson, B. D. O., & Moore, J. B. (1990). *Optimal control - linear quadratic methods*. Dover.
- Andersson, J. A. E., & Rawlings, J. B. (2018). Sensitivity Analysis for Nonlinear Programming in CasADi*. *IFAC-PapersOnLine*, 51(20), 331–336.
- Angeli, D., Amrit, R., & Rawlings, J. B. (2012). On Average Performance and Stability of Economic Model Predictive Control. *IEEE Transactions on Automatic Control*, 57(7), 1615–1626.
- ApS, M. (2024). The MOSEK optimization toolbox for MATLAB manual. Version 10.1.. [Online]. Available. URL <http://docs.mosek.com/latest/toolbox/index.html>.
- Arnström, D. (2023). Real-Time Certified MPC : Reliable Active-Set QP Solvers. Publisher: Linköping University Electronic Press.
- Aswani, A., Gonzalez, H., Sastry, S. S., & Tomlin, C. (2013). Provably safe and robust learning-based model predictive control. *Automatica*, 49(5), 1216–1226.
- Bakker, B. (2001). Reinforcement Learning with Long Short-Term Memory. In *Advances in neural information processing systems: Vol. 14*, MIT Press.
- Ball, P. J., & Roberts, S. J. (2021). OffCon3: What is state of the art anyway?.
- Bang, S. H., Jové, C. A., & Sents, L. (2024). RL-augmented MPC Framework for Agile and Robust Bipedal Footstep Locomotion Planning and Control. arXiv:2407.17683 [cs].
- Bar, A., Zhou, G., Tran, D., Darrell, T., & LeCun, Y. (2024). Navigation world models. arXiv:2412.03572.
- Baumgärtner, K., Messerer, F., & Diehl, M. (2023). A Unified Local Convergence Analysis of Differential Dynamic Programming, Direct Single Shooting, and Direct Multiple Shooting. In *Proceedings of the European control conference (ECC)*.
- Beckenbach, L., Osinenko, P., & Streif, S. (2020). A Q-learning predictive control scheme with guaranteed stability. *European Journal of Control*, 56, 167–178.
- Beckenbach, L., & Streif, S. (2022). Approximate infinite-horizon predictive control. In *IEEE 61st conference on decision and control (CDC)* (pp. 3711–3717). ISSN: 2576-2370.
- Bellman, R. (1966). Dynamic programming. *American Association for the Advancement of Science*, 153(3731), 34–37.
- Bemporad, A., Borrelli, F., & Morari, M. (1999). The explicit solution of constrained LP-Based receding horizon control. In *Proceedings of the IEEE conference on decision and control (CDC)*. Sydney, Australia.
- Bemporad, A., Borrelli, F., & Morari, M. (2002). Model predictive control based on linear programming - the explicit solution. *IEEE Transactions on Automatic Control*, 47(12), 1974–1985, Conference Name: IEEE Transactions on Automatic Control.
- Bemporad, A., Morari, M., Dua, V., & Pistikopoulos, E. N. (2000). The explicit solution of model predictive control via multiparametric quadratic programming. In *Proceedings of the 2000 American control conference. ACC (IEEE cat. no. 00CH36334): Vol. 2*, (pp. 872–876). IEEE.
- Ben-Tal, A., Goryashko, A., Guslitzer, E., & Nemirovski, A. (2004). Adjustable robust solutions of uncertain linear programs. *Mathematical Programming*, 99(2), 351–376.
- Bengio, Y., Lodi, A., & Prouvost, A. (2021). Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2), 405–421.
- Bertsekas, D. P. (2005). Dynamic Programming and Suboptimal Control: A Survey from {ADP} to {MPC}*. *European Journal of Control*, 11(4), 310–334.
- Bertsekas, D. (2019). *Reinforcement learning and optimal control* (1st ed.). Belmont, Massachusetts: Athena Scientific.
- Bertsekas, D. (2022a). *Lessons from alphazero for optimal, model predictive, and adaptive control*. Athena Scientific.
- Bertsekas, D. (2022b). Newton's method for reinforcement learning and model predictive control. *Results in Control and Optimization*, 7, Article 100121.
- Bertsekas, D. P., & Tsitsiklis, J. N. (1996). Neuro-dynamic programming. In *Optimization and neural computation series*, Belmont, Mass: Athena Scientific.
- Bharadhwaj, H., Xie, K., & Shkurti, F. (2020). Model-predictive control via cross-entropy and gradient-based optimization. In *Learning for dynamics and control* (pp. 277–286). PMLR.
- Bhardwaj, M., Choudhury, S., & Boots, B. (2021). Blending MPC & Value Function Approximation for Efficient Reinforcement Learning. In *9th international conference on learning representations, ICLR*.
- Binder, T., Blank, L., Bock, H. G., Bulirsch, R., Dahmen, W., Diehl, M., Kronseder, T., Marquardt, W., Schlöder, J. P., & Stryk, O. v. (2001). Introduction to Model Based Optimization of Chemical Processes on Moving Horizons. In M. Grötschel, S. O. Krumke, & J. Rambau (Eds.), *Online optimization of large scale systems: state of the art* (pp. 295–340). Springer.
- Bock, H. G. (1983). Recent Advances in Parameter Identification Techniques for ODE. In *Numerical treatment of inverse problems in differential and integral equations* (pp. 95–121). Birk\hau\ser.
- Bock, H. G., Diehl, M., Kostina, E. A., & Schlöder, J. P. (2007). Constrained Optimal Feedback Control of Systems Governed by Large Differential Algebraic Equations. In *Real-time and online PDE-constrained optimization* (pp. 3–22). SIAM.
- Bock, H. G., & Plitt, K. J. (1984). A Multiple Shooting Algorithm for Direct Solution of Optimal Control Problems. In *Proceedings of the IFAC world congress* (pp. 242–247). Pergamon Press.
- Bøhn, E., Gros, S., Moe, S., & Johansen, T. A. (2023). Optimization of the model predictive control meta-parameters through reinforcement learning. *Engineering Applications of Artificial Intelligence*, 123, Article 106211.
- Bou, A., Bettini, M., Dittert, S., Kumar, V., Sodhani, S., Yang, X., Fabritius, G. D., & Moens, V. (2023). TorchRL: A data-driven decision-making library for PyTorch. arXiv:2306.00577 [cs].
- Brandi, S., Fiorentini, M., & Capozzoli, A. (2022). Comparison of online and offline deep reinforcement learning with model predictive control for thermal energy management. *Automation in Construction*, 135, Article 104128.
- Brito, B., Agarwal, A., & Alonso-Mora, J. (2022). Learning Interaction-Aware Guidance for Trajectory Optimization in Dense Traffic Scenarios. *IEEE Transactions on Intelligent Transportation Systems*, 23(10), 18808–18821, Conference Name: IEEE Transactions on Intelligent Transportation Systems.
- Brito, B., Everett, M., How, J. P., & Alonso-Mora, J. (2021). Where to go Next: Learning a Subgoal Recommendation Policy for Navigation in Dynamic Environments. *IEEE Robotics and Automation Letters*, 6(3), 4616–4623.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., & Askell, A. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Brunke, L., Greeff, M., Hall, A. W., Yuan, Z., Zhou, S., Panerati, J., & Schoellig, A. P. (2022). Safe Learning in Robotics: From Learning-Based Control to Safe Reinforcement Learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(1), 411–444.
- Byravan, A., Hasenclever, L., Trochim, P., Mirza, M., Ialongo, A. D., Tassa, J. T., Abdolmaleki, A., Heess, N., Merel, J., & Riedmiller, M. (2022). Evaluating Model-Based Planning and Planner Amortization for Continuous Control. In *10th international conference on learning representations, ICLR*.
- Cai, W., Sawant, S., Reinhardt, D., Rastegarpour, S., & Gros, S. (2023). A Learning-Based Model Predictive Control Strategy for Home Energy Management Systems. *IEEE Access*, 11, 145264–145280.
- Calafiore, G. C., & Campi, M. C. (2006). The Scenario Approach to Robust Control Design. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*.
- Calvo-Fullana, M., Paternain, S., Chamon, L. F. O., & Ribeiro, A. (2024). State Augmented Constrained Reinforcement Learning: Overcoming the Limitations of Learning With Rewards. *IEEE Transactions on Automatic Control*, 69(7), 4275–4290, Conference Name: IEEE Transactions on Automatic Control.
- Cao, Y., & Gopaluni, R. B. (2020). Deep Neural Network Approximation of Nonlinear Model Predictive Control. *IFAC-PapersOnLine*, 53(2), 11319–11324.
- Carius, J., Farshidian, F., & Hutter, M. (2020). MPC-Net: A First Principles Guided Policy Search. *IEEE Robotics and Automation Letters*, 5(2), 2897–2904.
- Ceder, K., Zhang, Z., Burman, A., Kuangaliyev, I., Mattsson, K., Nyman, G., Petersén, A., Wisell, L., & Åkesson, K. (2024). Bird's-Eye-View Trajectory Planning of Multiple Robots using Continuous Deep Reinforcement Learning and Model Predictive Control. In *2024 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 8002–8008). ISSN: 2153-0866.

- Ceusters, G., Rodríguez, R. C., García, A. B., Franke, R., Deconinck, G., Helsen, L., Nowé, A., Messagie, M., & Camargo, L. R. (2021). Model-predictive control and reinforcement learning in multi-energy system case studies. *Applied Energy*, 303, Article 117634.
- Chen, H., & Allgöwer, F. (1998). A Quasi-Infinite Horizon Nonlinear Model Predictive Control Scheme with Guaranteed Stability. *Automatica*, 34(10), 1205–1218.
- Chen, S., Saulnier, K., Atanasov, N., Lee, D. D., Kumar, V., Pappas, G. J., & Morari, M. (2018). Approximating Explicit Model Predictive Control Using Constrained Neural Networks. In *2018 annual American control conference (ACC)* (pp. 1520–1527).
- Cheng, R., Orosz, G., Murray, R. M., & Burdick, J. W. (2019). End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks. In *Proceedings of the AAAI conference on artificial intelligence: Vol. 33*, (pp. 3387–3395). Issue: 01.
- Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. *Advances in Neural Information Processing Systems*, 31.
- Cosner, R. K., Yue, Y., & Ames, A. D. (2022). End-to-End Imitation Learning with Safety Guarantees using Control Barrier Functions. In *2022 IEEE 61st conference on decision and control (CDC)* (pp. 5316–5322). Cancun, Mexico: IEEE.
- Coulson, J., Lygeros, J., & Dörfler, F. (2019a). Regularized and Distributionally Robust Data-Enabled Predictive Control. In *IEEE 58th conference on decision and control (CDC)* (pp. 2696–2701). ISSN: 2576-2370.
- Coulson, J., Lygeros, J., & Dörfler, F. (2019b). Data-Enabled Predictive Control: In the Shallows of the DeepPC. In *18th European control conference (ECC)* (pp. 307–312).
- Cplex, I. I. (2009). V12. 1: User's Manual for CPLEX. *International Business Machines Corporation*, 46(53), 157.
- Dantzig, G. B. (1956). *The simplex method*. Santa Monica, CA: RAND Corporation.
- Dawood, M., Dengler, N., de Heuvel, J., & Bennewitz, M. (2023). Handling Sparse Rewards in Reinforcement Learning Using Model Predictive Control. In *IEEE international conference on robotics and automation (ICRA)* (pp. 879–885).
- Deits, R., Koolen, T., & Tedrake, R. (2019). LVIS: Learning from Value Function Intervals for Contact-Aware Robot Controllers. In *International conference on robotics and automation (ICRA)* (pp. 7762–7768). Montreal, QC, Canada: IEEE Press.
- Di Natale, L., Lian, Y., Maddalena, E. T., Shi, J., & Jones, C. N. (2022). Lessons Learned from Data-Driven Building Control Experiments: Contrasting Gaussian Process-based MPC, Bilevel DeepPC, and Deep Reinforcement Learning. In *IEEE 61st conference on decision and control (CDC)* (pp. 1111–1117). ISSN: 2576-2370.
- Didier, A., Jacobs, R. C., Sieber, J., Wabersich, K. P., & Zeilinger, M. N. (2023). Approximate Predictive Control Barrier Functions using Neural Networks: A Computationally Cheap and Permissive Safety Filter. In *European control conference (ECC)* (pp. 1–7).
- Didier, A., Zanelli, A., Wabersich, K. P., & Zeilinger, M. N. (2024). Predictive stability filters for nonlinear dynamical systems affected by disturbances. *IFAC-PapersOnLine*, 58(18), 200–207, 8th IFAC Conference on Nonlinear Model Predictive Control NMPC 2024.
- Diehl, M. (2002). Real-Time Optimization for Large Scale Nonlinear Processes. In *Fortschritt-berichte VDI reihe 8, meß-, steuerungs- und regelungstechnik: Vol. 920*, Düsseldorf: VDI Verlag.
- Diehl, M. (2007). Formulation of Closed Loop Min-Max MPC as a Quadratically Constrained Quadratic Program. *IEEE Transactions on Automatic Control*, 52(2), 339–343.
- Diehl, M., Ferreau, H. J., & Haverbeke, N. (2009). Efficient Numerical Methods for Nonlinear MPC and Moving Horizon Estimation. In L. Magni, M. D. Raimondo, & F. Allgöwer (Eds.), *Lecture notes in control and information sciences: Vol. 384, Nonlinear model predictive control* (pp. 391–417). Springer.
- Diehl, M., Findeisen, R., Allgöwer, F., Bock, H. G., & Schlöder, J. P. (2005). Nominal Stability of the Real-Time Iteration Scheme for Nonlinear Model Predictive Control. *IEEE Proc.-Control Theory Appl.*, 152(3), 296–308, Publisher: IEE.
- Diehl, M., Magni, L., & Nicolao, G. D. (2003). Online NMPC of a looping kite using approximate infinite horizon closed loop costing. In *Proceedings of the IFAC conference on control systems design*. Bratislava, Slovak Republic: IFAC.
- Diehl, M., Magni, L., & Nicolao, G. D. (2004). Efficient NMPC of unstable periodic systems using approximate infinite horizon closed loop costing. *Annual Reviews in Control*, 28(1), 37–45.
- Dobriborsci, D., Osinenko, P., & Aumer, W. (2022). An experimental study of two predictive reinforcement learning methods and comparison with model-predictive control. *IFAC-PapersOnLine*, 55(10), 1545–1550.
- Domahidi, A., & Jerez, J. (2014). FORCES Professional. Published: Embotech AG, url=https://embotech.com/FORCES-Pro.
- Drgona, J., Tuor, A., Koch, J., Shapiro, M., Jacob, B., & Vrabie, D. (2023). NeuroMANCER: Neural Modules with Adaptive Nonlinear Constraints and Efficient Regularizations. [Online]. Available. URL https://github.com/pnnl/neuromancer.
- Drgoňa, J., Tuor, A., & Vrabie, D. (2024). Learning Constrained Parametric Differentiable Predictive Control Policies With Guarantees. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 1–12.
- Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Paduraru, C., Goyal, S., & Hester, T. (2021). Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9), 2419–2468.
- Englert, T., Völz, A., Mesmer, F., Rhein, S., & Graichen, K. (2019). A software framework for embedded nonlinear model predictive control using a gradient-based augmented Lagrangian approach (GRAMPC). *Optimization and Engineering*, 20(3), 769–809.
- Ernst, D., Geurts, P., & Wehenkel, L. (2003). Iteratively Extending Time Horizon Reinforcement Learning. In N. Lavrac, D. Gamberger, H. Blockeel, & L. c. Todorovski (Eds.), *Lecture notes in computer science, Machine learning: ECML 2003* (pp. 96–107). Berlin, Heidelberg: Springer.
- Ernst, D., Glavic, M., Capitanescu, F., & Wehenkel, L. (2009). Reinforcement Learning Versus Model Predictive Control: A Comparison on a Power System Problem. *IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics)*, 39(2), 517–529, Conference Name: IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics).
- Esfahani, H. N., Bahari Kordabad, A., Cai, W., & Gros, S. (2023). Learning-based state estimation and control using MHE and MPC schemes with imperfect models. *European Journal of Control*, 73, Article 100880.
- Esfahani, H. N., Kordabad, A. B., & Gros, S. (2021). Reinforcement learning based on MPC/MHE for unmodeled and partially observable dynamics. In *American control conference (ACC)* (pp. 2121–2126). IEEE.
- Everett, M. (2021). Neural Network Verification in Control. In *2021 60th IEEE conference on decision and control (CDC)* (pp. 6326–6340). Austin, TX, USA: IEEE.
- Eysenbach, B., & Levine, S. (2022). Maximum Entropy RL (Provably) Solves Some Robust RL Problems. In *10th international conference on learning representations, ICLR*.
- Favoreel, W., Moor, B. D., & Gevers, M. (1999). SPC: Subspace Predictive Control. *IFAC Proceedings Volumes*, 32(2), 4004–4009.
- Feng, X., Di Cairano, S., & Quirynen, R. (2020). Inexact adjoint-based SQP algorithm for real-time stochastic nonlinear MPC. *IFAC-PapersOnLine*, 53(2), 6529–6535, Publisher: Elsevier.
- Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning* (pp. 1126–1135). PMLR.
- Fleming, W. H. (1971). Stochastic Control for Small Noise Intensities. *SIAM Journal on Control*, 9(3).
- Frank, J., Mannor, S., & Precup, D. (2008). Reinforcement learning in the presence of rare events. In *Proceedings of the 25th international conference on machine learning* (pp. 336–343).
- Frey, J., Baumgärtner, K., Frison, G., Reinhardt, D., Hoffmann, J., Fichtner, L., Gros, S., & Diehl, M. (2025). Differentiable nonlinear model predictive control. arXiv:2505.01353.
- Frey, J., Cairano, S. D., & Quirynen, R. (2020). Active-Set based Inexact Interior Point QP Solver for Model Predictive Control. In *Proceedings of the IFAC world congress*.
- Frey, J., Gao, Y., Messerer, F., Lahr, A., Zeilinger, M. N., & Diehl, M. (2024). Efficient Zero-Order Robust Optimization for Real-Time Model Predictive Control with acados. In *Proceedings of the European control conference (ECC)*.
- Frison, G., & Diehl, M. (2020). HPIPM: a high-performance quadratic programming framework for model predictive control. In *21st IFAC world congress: IFAC-PapersOnLine*, In *21st IFAC world congress: Vol. 53(2)*.6563–6569,
- Frison, G., Kouzoupis, D., Diehl, M., & Jørgensen, J. B. (2017). A high-performance Riccati based solver for tree-structured quadratic programs. 50, In *IFAC* (pp. 14399–14405). Issue: 1.
- Fu, J., Luo, K., & Levine, S. (2018). Learning Robust Rewards with Adversarial Inverse Reinforcement Learning.
- Fujimoto, S., van Hoof, H., & Meger, D. (2018). Addressing function approximation error in actor-critic methods. In J. G. Dy, & A. Krause (Eds.), *Proceedings of machine learning research: 80, Proceedings of the 35th international conference on machine learning, ICLR 2018* (pp. 1582–1591). PMLR.
- Gauss, C. F. (1809). *Theoria motus corporum coelestium in sectionibus conicis solem ambientium: Vol. 7*, Perthes et Besser.
- Ghezzi, A., Hoffman, J., Frey, J., Boedecker, J., & Diehl, M. (2023). Imitation Learning from Nonlinear MPC via the Exact Q-Loss and its Gauss-Newton Approximation. In *62nd IEEE conference on decision and control (CDC)* (pp. 4766–4771). ISSN: 2576-2370.
- Ghezzi, A., Reiter, R., Baumgärtner, K., Bemporad, A., & Diehl, M. (2025). A numerically efficient method to enhance model predictive control performance with a reinforcement learning policy. arXiv:2504.02710.
- Gill, P. E., Murray, W., & Saunders, M. A. (2002). SNOPT: An SQP Algorithm for Large-Scale Constrained Optimization. *SIAM Journal on Optimization*, 12(4), 979–1006.
- Goldfain, B., Drews, P., You, C., Barulic, M., Velez, O., Tsiotras, P., & Reh, J. M. (2019). AutoRally: An Open Platform for Aggressive Autonomous Driving. *IEEE Control Systems Magazine*, 39(1), 26–55, Conference Name: IEEE Control Systems Magazine.
- Görges, D. (2017). Relations between Model Predictive Control and Reinforcement Learning. *IFAC-PapersOnLine*, 50(1), 4920–4928.
- Goulart, P. J., Kerrigan, E. C., & Maciejowski, J. M. (2006). Optimization over state feedback policies for robust control with constraints. *Automatica*, 42, 523–533.
- Goulart, P. J., Kerrigan, E. C., & Ralph, D. (2008). Efficient robust optimization for robust control with constraints. *Mathematical Programming*, 114(1), 115–147, Publisher: Springer.

- Grancharova, A., & Johansen, T. A. (2012). Explicit Nonlinear Model Predictive Control: Theory and Applications. *Lecture notes in control and information sciences: Vol. 429*, Berlin, Heidelberg: Springer.
- Grandesso, G., Alboni, E., Papini, G. P. R., Wensing, P. M., & Del Prete, A. (2023). CACTO: Continuous Actor-Critic With Trajectory Optimization—Towards Global Optimality. *IEEE Robotics and Automation Letters*, 8(6), 3318–3325.
- Greatwood, C., & Richards, A. G. (2019). Reinforcement learning and model predictive control for robust embedded quadrotor guidance and control. *Autonomous Robots*, 43(7), 1681–1693.
- Gros, S., & Zanon, M. (2019). Towards Safe Reinforcement Learning Using NMPC and Policy Gradients: Part I - Stochastic case. [arXiv:1906.04057](https://arxiv.org/abs/1906.04057) [cs].
- Gros, S., & Zanon, M. (2020). Data-Driven Economic NMPC Using Reinforcement Learning. *IEEE Transactions on Automatic Control*, 65(2), 636–648.
- Gros, S., & Zanon, M. (2020). Reinforcement Learning for mixed-integer problems based on MPC. *IFAC-PapersOnLine*, 53(2), 5219–5224.
- Gros, S., & Zanon, M. (2021). Reinforcement Learning based on MPC and the Stochastic Policy Gradient Method. In *American control conference (ACC)* (pp. 1947–1952). ISSN: 2378-5861.
- Gros, S., & Zanon, M. (2022a). Economic MPC of Markov Decision Processes: Dissipativity in undiscounted infinite-horizon optimal control. *Automatica*, 146, Article 110602.
- Gros, S., & Zanon, M. (2022b). Learning for MPC with stability & safety guarantees. *Automatica*, 146, Article 110598.
- Gros, S., Zanon, M., & Bemporad, A. (2020). Safe reinforcement learning via projection on a safe set: How to achieve optimality? *IFAC-PapersOnLine*, 53(2), 8076–8081, Publisher: Elsevier.
- Gros, S., Zanon, M., Quirynen, R., Bemporad, A., & Diehl, M. (2016). From Linear to Nonlinear MPC: bridging the gap via the Real-Time Iteration. *International Journal of Control*.
- Grudzien, J., De Witt, C. A. S., & Foerster, J. (2022). Mirror learning: A unifying framework of policy optimisation. In *International conference on machine learning* (pp. 7825–7844). PMLR.
- Grüne, L. (2012). NMPC without terminal constraints. *IFAC Proceedings Volumes*, 45(17), 1–13, Publisher: Elsevier.
- Grüne, L. (2013). Economic receding horizon control without terminal constraints. *Automatica*, 49(3), 725–734.
- Grüne, L., & Pannek, J. (2017). *Nonlinear model predictive control. theory and algorithms* (2nd ed.). Springer.
- Grüne, L., & Rantzer, A. (2008). On the infinite horizon performance of receding horizon controllers. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*, 53(9), Publisher: University of Bayreuth.
- Gurobi Optimization, LLC (2023). Gurobi Optimizer Reference Manual. [Online]. Available. URL <https://www.gurobi.com>.
- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In J. G. Dy, & A. Krause (Eds.), *Proceedings of machine learning research: Vol. 80, Proceedings of the 35th international conference on machine learning, ICML (pp. 1856–1865)*. PMLR.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., & Levine, S. (2019). Soft Actor-Critic Algorithms and Applications. [arXiv:1812.05905](https://arxiv.org/abs/1812.05905) [cs].
- Hadjiyiannis, M. J., Goulart, P. J., & Kuhn, D. (2011). An Efficient Method to Estimate the Suboptimality of Affine Controllers. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*, 56(12).
- Hansen, N. A., Su, H., & Wang, X. (2022). Temporal difference learning for model predictive control. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, & S. Sabato (Eds.), *Proceedings of machine learning research: Vol. 162, Proceedings of the 39th international conference on machine learning (pp. 8387–8406)*. PMLR.
- Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, robust world models for continuous control. In *International conference on learning representations*.
- Hasankhani, A., Tang, Y., VanZwieten, J., & Sultan, C. (2021). Comparison of Deep Reinforcement Learning and Model Predictive Control for Real-Time Depth Optimization of a Lifting Surface Controlled Ocean Current Turbine. In *IEEE conference on control technology and applications (CCTA)* (pp. 301–308). ISSN: 2768-0770.
- Hatch, N., & Boots, B. (2021). The Value of Planning for Infinite-Horizon Model Predictive Control. In *2021 IEEE international conference on robotics and automation (ICRA)* (pp. 7372–7378). ISSN: 2577-087X.
- Hertneck, M., Köhler, J., Trimpe, S., & Allgöwer, F. (2018). Learning an Approximate Model Predictive Controller With Guarantees. *IEEE Control Systems Letters*, 2(3), 543–548, Conference Name: IEEE Control Systems Letters.
- Hewing, L., Kabzan, J., & Zeilinger, M. N. (2020). Cautious Model Predictive Control Using Gaussian Process Regression. *IEEE Transactions on Control Systems Technology*, 28(6), 2736–2743.
- Hewing, L., Wabersich, K. P., Menner, M., & Zeilinger, M. N. (2020). Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3(1), 269–296.
- Ho, J., & Ermon, S. (2016). Generative Adversarial Imitation Learning. In *Advances in neural information processing systems: Vol. 29*, Curran Associates, Inc..
- Hoeller, D., Farshidian, F., & Hutter, M. (2020). Deep Value Model Predictive Control. In *Proceedings of the conference on robot learning (pp. 990–1004)*. PMLR, ISSN: 2640-3498.
- Hoffmann, J., Clausen, D. F., Brosseit, J., Bernhard, J., Esterle, K., Werling, M., Karg, M., & Bödecker, J. J. (2024). PlanNetX: Learning an efficient neural network planner from MPC for longitudinal control. In *Proceedings of the 6th annual learning for dynamics & control conference* (pp. 1214–1227). PMLR, ISSN: 2640-3498.
- Hoffmann, P., Gorelik, K., & Ivanov, V. (2024). Comparison of Reinforcement Learning and Model Predictive Control for Automated Generation of Optimal Control for Dynamic Systems within a Design Space Exploration Framework. *International Journal of Automotive Engineering*, 15(1), 19–26.
- Houska, B. (2011). *Robust optimization of dynamic systems* (Ph.D. thesis), KU Leuven.
- Houska, B., Ferreau, H. J., & Diehl, M. (2011). ACADO toolkit—An open-source framework for automatic control and dynamic optimization. *Optimal Control Applications and Methods*, 32(3), 298–312, eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/oca.939>.
- Imran, M., Izzo, R., Tortorelli, A., & Liberati, F. (2023). Comparison of Traffic Control with Model Predictive Control and Deep Reinforcement Learning. In *9th international conference on control, decision and information technologies (coDIT)* (pp. 989–994). ISSN: 2576-3555.
- Jacquet, M., & Alexis, K. (2024). N-MPC for Deep Neural Network-Based Collision Avoidance exploiting Depth Images. [arXiv:2402.13038](https://arxiv.org/abs/2402.13038) [cs].
- Jenelten, F., Grandia, R., Farshidian, F., & Hutter, M. (2022). TAMOLS: Terrain-Aware Motion Optimization for Legged Systems. *IEEE Transactions on Robotics*, 38(6), 3395–3413, [arXiv:2206.14049](https://arxiv.org/abs/2206.14049) [cs].
- Jenelten, F., He, J., Farshidian, F., & Hutter, M. (2024). DTC: Deep Tracking Control. *Science Robotics*, 9(86), eadh5401, Publisher: American Association for the Advancement of Science.
- Jerez, J. L., Goulart, P. J., Richter, S., Constantinides, G. A., Kerrigan, E. C., & Morari, M. (2014). Embedded online optimization for model predictive control at megahertz rates. *IEEE Transactions on Automatic Control*, 59(12), 3238–3251, Publisher: IEEE.
- Johansen, T., & Grancharova, A. (2003). Approximate explicit constrained linear model predictive control via orthogonal search tree. *IEEE Transactions on Automatic Control*, 48, 810–815.
- Kalweit, G., Huegle, M., Werling, M., & Boedecker, J. (2020). Deep Inverse Q-learning with Constraints. In *Advances in neural information processing systems: Vol. 33*, (pp. 14291–14302). Curran Associates, Inc..
- Kang, D., Cheng, J., Zamora, M., Zarghashi, F., & Coros, S. (2023). RL + Model-Based Control: Using On-Demand Optimal Control to Learn Versatile Legged Locomotion. *IEEE Robotics and Automation Letters*, 8(10), 6619–6626.
- Kang, J., Raghunathan, A. U., & Di Cairano, S. (2015). Decomposition via ADMM for scenario-based model predictive control. In *American control conference (ACC)* (pp. 1246–1251). IEEE.
- Kantas, N., Maciejowski, J., & Lecchini-Visintini, A. (2009). Sequential Monte Carlo for model predictive control. *Nonlinear Model Predictive Control: Towards New Challenging Applications*, 263–273, Publisher: Springer.
- Kappen, H. J. (2005). Linear Theory for Control of Nonlinear Stochastic Systems. *Physical Review Letters*, 95(20), Article 200201, Publisher: American Physical Society.
- Karapetyan, A., Balta, E. C., Iannelli, A., & Lygeros, J. (2023). On the Finite-Time Behavior of Suboptimal Linear Model Predictive Control. In *Proceedings of the IEEE conference on decision and control (CDC)*.
- Karg, B., & Lucia, S. (2020). Efficient Representation and Approximation of Model Predictive Control Laws via Deep Learning. *IEEE Transactions on Cybernetics*, 50(9), 3866–3878.
- Karnchanachari, N., Valls, M. I., Hoeller, D., & Hutter, M. (2020). Practical Reinforcement Learning For MPC: Learning from sparse objectives in under an hour on a real robot. In *Proceedings of the 2nd conference on learning for dynamics and control* (pp. 211–224). PMLR, ISSN: 2640-3498.
- Kaufmann, E., Bauersfeld, L., Loquercio, A., Müller, M., Koltun, V., & Scaramuzza, D. (2023). Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976), 982–987, Number: 7976 Publisher: Nature Publishing Group.
- Kaufmann, E., Loquercio, A., Ranftl, R., Müller, M., Koltun, V., & Scaramuzza, D. (2020). Deep Drone Acrobatics. In *Robotics: science and systems XVI. Robotics: Science and Systems Foundation*.
- Kennelly, G., De, A., & Koditschek, D. E. (2016). Design Principles for a Family of Direct-Drive Legged Robots. *IEEE Robotics and Automation Letters*, 1(2), 900–907.
- Kerrigan, E. C. (2000). *Robust constraint satisfaction: Invariant sets and predictive control*. University of Cambridge, UK.
- Klintberg, E., Dahl, J., Fredriksson, J., & Gros, S. (2016). An improved dual Newton strategy for scenario-tree MPC. In *CDC* (pp. 3675–3681).
- Kobilarov, M. (2012). Cross-entropy motion planning. *The International Journal of Robotics Research*, 31(7), 855–871, Publisher: SAGE Publications Sage UK: London, England.
- Kordabad, A. B., Esfahani, H. N., Lekkas, A. M., & Gros, S. (2021). Reinforcement Learning based on Scenario-tree MPC for ASVs. In *2021 American control conference (ACC)* (pp. 1985–1990). ISSN: 2378-5861.
- Kordabad, A. B., & Gros, S. (2021). Verification of Dissipativity and Evaluation of Storage Function in Economic Nonlinear MPC using Q-Learning. *IFAC-PapersOnLine*, 54(6), 308–313.
- Kordabad, A. B., & Gros, S. (2023). Bias Correction of Discounted Optimal Steady-State using Cost Modification. In *2023 European control conference (ECC)* (pp. 1–6).

- Kordabad, A. B., Nejatbakhsh Esfahani, H., & Gros, S. (2021). Bias Correction in Deterministic Policy Gradient Using Robust MPC. In *2021 European control conference (ECC)* (pp. 1086–1091).
- Kordabad, A. B., Wisniewski, R., & Gros, S. (2022). Safe Reinforcement Learning Using Wasserstein Distributionally Robust MPC and Chance Constraint. *IEEE Access*, 10, 130058–130067, Conference Name: IEEE Access.
- Kordabad, A. B., Zanon, M., & Gros, S. (2024). Equivalence of Optimality Criteria for Markov Decision Process and Model Predictive Control. *IEEE Transactions on Automatic Control*, 69(2), 1149–1156, Conference Name: IEEE Transactions on Automatic Control.
- Kouvaritakis, B., & Cannon, M. (2015). *Model predictive control: Classical, robust and stochastic* (1st ed.). Cham Heidelberg New York Dordrecht London: Springer.
- Kouzoupis, D. (2019). *Structure-exploiting numerical methods for tree-sparse optimal control problems* (Ph.D. thesis), University of Freiburg.
- Kouzoupis, D., Frison, G., Zanelli, A., & Diehl, M. (2018). Recent advances in quadratic programming algorithms for nonlinear model predictive control. *Vietnam Journal of Mathematics*, 46(4), 863–882.
- Kouzoupis, D., Klintberg, E., Frison, G., Gros, S., & Diehl, M. (2019). A dual Newton strategy for tree-sparse quadratic programs and its implementation in the open-source software treeQP. *International Journal of Robust and Nonlinear Control*.
- Krinner, M., Romero, A., Bauersfeld, L., Zeilinger, M., Carron, A., & Scaramuzza, D. (2024). MPCC+: Model Predictive Contouring Control for Time-Optimal Flight with Safety Constraints. In *Robotics: science and systems XX*. Robotics: Science and Systems Foundation.
- Kurzhanski, A. B., & Valyi, P. (1997). *Ellipsoidal calculus for estimation and control*. Birkhäuser Boston.
- Lahr, A., Näf, J., Wabersich, K. P., Frey, J., Siehl, P., Carron, A., Diehl, M., & Zeilinger, M. N. (2024). L4cadados: Learning-based models for acados, applied to Gaussian process-based predictive control. arXiv:2411.19258. tex.pubstate: prepublished.
- Lambert, N., Drew, D. S., Yaconelli, J., Levine, S., Calandra, R., & Pister, K. S. J. (2019). Low-Level Control of a Quadrotor With Deep Model-Based Reinforcement Learning. *IEEE Robotics and Automation Letters*, 4, 4224–4230.
- Lambrechts, G., Bolland, A., & Ernst, D. (2022). Recurrent networks, hidden states and beliefs in partially observable environments. *Transactions on Machine Learning Research*.
- Langson, W., I. Chrysoschoos, S. R., & Mayne, D. Q. (2004). Robust model predictive control using tubes. *Automatica*, 40(1), 125–133.
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (1998). Efficient BackProp. In G. B. Orr, & K.-R. Müller (Eds.), *Neural networks: tricks of the trade* (pp. 9–50). Berlin, Heidelberg: Springer.
- Leeman, A. P., Köhler, J., Messerer, F., Lahr, A., Diehl, M., & Zeilinger, M. N. (2024). Fast System Level Synthesis: Robust Model Predictive Control using Riccati Recursions. In *Proceedings of the IFAC conference on nonlinear model predictive control (NMPC)*.
- Levine, S., Finn, C., Darrell, T., & Abbeel, P. (2016). End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39), 1–40, ISBN: 1533-7928.
- Levine, S., & Koltun, V. (2013a). Guided Policy Search. In *Proceedings of the 30th international conference on machine learning* (pp. 1–9). PMLR, ISSN: 1938-7228.
- Levine, S., & Koltun, V. (2013b). Variational Policy Search via Trajectory Optimization. In *Advances in neural information processing systems: Vol. 26*, Curran Associates, Inc..
- Li, S., & Bastani, O. (2020). Robust Model Predictive Shielding for Safe Reinforcement Learning with Stochastic Dynamics. In *2020 IEEE international conference on robotics and automation (ICRA)* (pp. 7166–7172). ISSN: 2577-087X.
- Li, Y., Karapetyan, A., Lygeros, J., Johansson, K. H., & M° artensson, J. (2023). Performance Bounds of Model Predictive Control for Unconstrained and Constrained Linear Quadratic Problems and Beyond. In *Proceedings of the IFAC World Congress*.
- Li, W., & Todorov, E. (2004). Iterative Linear Quadratic Regulator Design for Nonlinear Biological Movement Systems. In *Proceedings of the 1st international conference on informatics in control, automation and robotics*.
- Lidec, Q. L., Jallet, W., Laptev, I., Schmid, C., & Carpentier, J. (2023). Enforcing the consensus between Trajectory Optimization and Policy Learning for precise robot control. In *IEEE international conference on robotics and automation (ICRA)* (pp. 946–952).
- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. In Y. Bengio, & Y. LeCun (Eds.), *4th international conference on learning representations, ICLR*.
- Limon, D., et al. (2009). Input-to-State Stability: A Unifying Framework for Robust Model Predictive Control. In F. A. L. Magni (Ed.), vol. 384, *Nonlinear model predictive control. lecture notes in control and information sciences*. Heidelberg: Springer.
- Lin, Y., Hu, Y., Qu, G., Li, T., & Wierman, A. (2022). Bounded-Regret MPC via Perturbation Analysis: Prediction Error, Constraints, and Nonlinearity. *NeurIPS*.
- Lin, Y., McPhee, J., & Azad, N. L. (2021). Comparison of Deep Reinforcement Learning and Model Predictive Control for Adaptive Cruise Control. *IEEE Transactions on Intelligent Vehicles*, 6(2), 221–231, Conference Name: IEEE Transactions on Intelligent Vehicles.
- Lin, M., Sun, Z., Xia, Y., & Zhang, J. (2024). Reinforcement Learning-Based Model Predictive Control for Discrete-Time Systems. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3), 3312–3324, Conference Name: IEEE Transactions on Neural Networks and Learning Systems.
- Liu, X., Peters, L., & Alonso-Mora, J. (2023). Learning to Play Trajectory Games Against Opponents With Unknown Objectives. *IEEE Robotics and Automation Letters*, 8(7), 4139–4146, Conference Name: IEEE Robotics and Automation Letters.
- Ljung, L. (1999). *System identification: Theory for the user*. Prentice Hall PTR.
- Lowrey, K., Rajeswaran, A., Kakade, S., Todorov, E., & Mordatch, I. (2019). Plan Online, Learn Offline: Efficient Learning and Exploration via Model-Based Control. In *7th international conference on learning representations*.
- Magni, L., & Sepulchre, R. (1997). Stability margins of nonlinear receding-horizon control via inverse optimality. *Systems & Control Letters*, 32, 241–245.
- Mamedov, S., Reiter, R., Azad, S. M. B., Viljoen, R., Boedecker, J., Diehl, M., & Swevers, J. (2024). Safe Imitation Learning of Nonlinear Model Predictive Control for Flexible Robots. In *2024 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 3613–3619). ISSN: 2153-0866.
- Marti-Saumell, J., Solà, J., Mastalli, C., & Santamaria-Navarro, A. (2020). Squash-box feasibility driven differential dynamic programming. In *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 7637–7644). IEEE.
- Mastalli, C., Budhiraja, R., Merkt, W., Saurel, G., Hammoud, B., Naveau, M., Carpentier, J., Vijayakumar, S., & Mansard, N. (2020). Crocodyl: An Efficient and Versatile Framework for Multi-Contact Optimal Control. In *ICRA 2020 IEEE international conference on robotics and automation*. Paris / Virtual, France.
- Mayne, D. (1966). A Second-order Gradient Method for Determining Optimal Trajectories of Non-linear Discrete-time Systems. *International Journal of Control*, 3(1), 85–96.
- Mayne, D. (2013). An apologia for stabilising terminal conditions in model predictive control. *International Journal of Control*, 11.
- Mayne, D., Kerrigan, E., Wyk, E. J. v., & Falugi, P. (2011). Tube-based robust nonlinear model predictive control. *International Journal of Robust and Nonlinear Control*, 21, 1341–1353.
- Mayne, D. Q., Rawlings, J. B., Rao, C. V., & Scokaert, P. O. M. (2000). Constrained model predictive control: Stability and optimality. *Automatica*, 26(6), 789–814.
- McAllister, R. D., & Rawlings, J. B. (2022). Inherent Stochastic Robustness of Model Predictive Control to Large and Infrequent Disturbances. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*, 67(10).
- McAllister, R. D., & Rawlings, J. B. (2024). On the Inherent Distributional Robustness of Stochastic and Nominal Model Predictive Control. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*, 69(2).
- Mesbah, A. (2016). Stochastic Model Predictive Control: An Overview and Perspectives for Future Research. *IEEE Control Systems Magazine*, 36(6), 30–44, Conference Name: IEEE Control Systems Magazine.
- Mesbah, A., Wabersich, K. P., Schoellig, A. P., Zeilinger, M. N., Lucia, S., Badgwell, T. A., & Paulson, J. A. (2022). Fusion of Machine Learning and MPC under Uncertainty: What Advances Are on the Horizon? In *American control conference (ACC)* (pp. 342–357). ISSN: 2378-5861.
- Messerer, F., Baumgärtner, K., Lucia, S., & Diehl, M. (2024). Fourth-order suboptimality of nominal model predictive control in the presence of uncertainty. ArXiv.
- Messerer, F., & Diehl, M. (2021). An Efficient Algorithm for Tube-based Robust Nonlinear Optimal Control with Optimal Linear Feedback. In *Proceedings of the IEEE conference on decision and control (CDC)*.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M. A., Fiedler, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nat.*, 518(7540), 529–533.
- Moerland, T. M., Broekens, J., Plaat, A., & Jonker, C. M. (2023). Model-based reinforcement learning: A survey. *Found. Trends Mach. Learn.*, 16(1), 1–118.
- Moos, J., Hansel, K., Abdulsamad, H., Stark, S., Clever, D., & Peters, J. (2022). Robust Reinforcement Learning: A Review of Foundations and Recent Advances. *Machine Learning and Knowledge Extraction*, 4(1), 276–315, Number: 1 Publisher: Multidisciplinary Digital Publishing Institute.
- Moradimaryamnegari, H., Frego, M., & Peer, A. (2022). Model Predictive Control-Based Reinforcement Learning Using Expected Sarsa. *IEEE Access*, 10, 81177–81191, Conference Name: IEEE Access.
- Morari, M., & Lee, J. H. (1999). Model predictive control: past, present and future. *Computers & Chemical Engineering*, 23(4), 667–682.
- Morcego, B., Yin, W., Boersma, S., van Henten, E., Puig, V., & Sun, C. (2023). Reinforcement Learning versus Model Predictive Control on greenhouse climate control. *Computers and Electronics in Agriculture*, 215, Article 108372.
- Mordatch, I., & Todorov, E. (2014). *Combining the benefits of function approximation and trajectory optimization: Vol. 10*.
- Moreno-Mora, F., Beckenbach, L., & Streif, S. (2023). Predictive Control with Learning-Based Terminal Costs Using Approximate Value Iteration. *IFAC-PapersOnLine*, 56(2), 3874–3879.
- Morgan, A. S., Nandha, D., Chalvatzaki, G., D'Eramo, C., Dollar, A. M., & Peters, J. (2021). Model Predictive Actor-Critic: Accelerating Robot Skill Acquisition with Deep Reinforcement Learning. In *2021 IEEE international conference on robotics and automation (ICRA)* (pp. 6672–6678). IEEE Press.

- Morimoto, J., & Doya, K. (2000). Robust Reinforcement Learning. In *Advances in neural information processing systems: Vol. 13*, MIT Press.
- Müller, J., & Cayci, S. (2024). Essentially Sharp Estimates on the Entropy Regularization Error in Discounted Markov Decision Processes.
- Nagabandi, A., Kahn, G., Fearing, R. S., & Levine, S. (2018). Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE international conference on robotics and automation (ICRA)* (pp. 7559–7566). IEEE.
- Nagy, Z. K., & Braatz, R. D. (2004). Open-loop and closed-loop robust optimal control of batch processes using distributional and worst-case analysis. *Journal of Process Control*, 14, 411–422.
- Nasvytis, L., Sandbrink, K., Foerster, J., Franzmeyer, T., & Schroeder de Witt, C. (2024). Rethinking Out-of-Distribution Detection for Reinforcement Learning: Advancing Methods for Evaluation and Detection. In *Proceedings of the 23rd international conference on autonomous agents and multiagent systems* (pp. 1445–1453).
- Neumann, G. (2011). Variational inference for policy search in changing situations. In *Proceedings of the 28th international conference on machine learning* (pp. 817–824). Bellevue, Washington, USA: Omni Press.
- Nicolao, G. D., Magni, L., & Scattolini, R. (1996a). Stabilizing nonlinear receding horizon control via a nonquadratic terminal state penalty. In *Symposium on control, optimization and supervision, CESA'96 IMACS multicongress* (pp. 185–187). Lille.
- Nicolao, G. D., Magni, L., & Scattolini, R. (1996b). On the Robustness of Receding-Horizon Control with Terminal Constraints. *Institute of Electrical and Electronics Engineers. Transactions on Automatic Control*, 41(3).
- Nicolao, G. D., Magni, L., & Scattolini, R. (1998). Stabilizing Receding-Horizon control of nonlinear time varying systems. *IEEE Transactions on Automatic Control*, AC-43(7), 1030–1036.
- Nikishin, E., Abachi, R., Agarwal, R., & Bacon, P.-L. (2022). Control-oriented model-based reinforcement learning with implicit differentiation. In *Proceedings of the AAAI conference on artificial intelligence: Vol. 36*, (pp. 7886–7894). Issue: 7.
- Nocedal, J., & Wright, S. J. (2006). Numerical Optimization, *Springer series in operations research and financial engineering*, (2nd ed.). Springer.
- Norouzi, A., Heidarifar, H., Borhan, H., Shabbakhti, M., & Koch, C. R. (2023). Integrating Machine Learning and Model Predictive Control for automotive applications: A review and future directions. *Engineering Applications of Artificial Intelligence*, 120, Article 105878.
- Nurkanović, A. (2023). *Numerical methods for optimal control of nonsmooth dynamical systems* (Ph.D. thesis), University of Freiburg.
- Oh, T. H. (2024). Quantitative comparison of reinforcement learning and data-driven model predictive control for chemical and biological processes. *Computers & Chemical Engineering*, 181, Article 108558.
- Pannocchia, G., Rawlings, J., & Wright, S. (2011). Conditions under which suboptimal nonlinear MPC is inherently robust. *Systems & Control Letters*, 60(9), 747–755.
- Parisini, T., & Zoppoli, R. (1995). A receding-horizon regulator for nonlinear systems and a neural approximation. *Automatica*, 31(10), 1443–1451.
- Patil, S., Kahn, G., Laskey, M., Schulman, J., Goldberg, K., & Abbeel, P. (2015). Scaling up Gaussian Belief Space Planning Through Covariance-Free Trajectory Optimization and Automatic Differentiation. In H. L. Akin, N. M. Amato, V. Isler, & A. F. van der Stappen (Eds.), *Algorithmic foundations of robotics XI: selected contributions of the eleventh international workshop on the algorithmic foundations of robotics* (pp. 515–533). Cham: Springer International Publishing.
- Patrinos, P., & Bemporad, A. (2013). An accelerated dual gradient-projection algorithm for embedded linear model predictive control. *IEEE Transactions on Automatic Control*, 59(1), 18–33, Publisher: IEEE.
- Pfrommer, S., Gautam, T., Zhou, A., & Sojoudi, S. (2022). Safe Reinforcement Learning with Chance-constrained Model Predictive Control. In *Proceedings of the 4th annual learning for dynamics and control conference* (pp. 291–303). PMLR, ISSN: 2640-3498.
- Piche, A., Thomas, V., Ibrahim, C., Bengio, Y., & Pal, C. (2018). Probabilistic Planning with Sequential Monte Carlo methods.
- Pineda, L., Fan, T., Monge, M., Venkataraman, S., Sodhi, P., Chen, R. T. Q., Ortiz, J., DeTone, D., Wang, A., Anderson, S., Dong, J., Amos, B., & Mukadam, M. (2022). Theseus: A library for differentiable nonlinear optimization. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems: Vol.35*, (pp. 3801–3818). Curran Associates, Inc..
- Pinneri, C., Sawant, S., Blaes, S., & Martius, G. (2021). Extracting Strong Policies for Robotics Tasks from Zero-Order Trajectory Optimizers. In *International conference on learning representations*.
- Piovesan, J. L., & Tanner, H. G. (2009). Randomized model predictive control for robot navigation. In *2009 IEEE international conference on robotics and automation* (pp. 94–99). IEEE.
- Powell, W. B. (2007). *Approximate dynamic programming: Solving the curses of dimensionality: Vol. 703*, John Wiley & Sons.
- Puterman, M. L. (2005). Markov decision processes: discrete stochastic dynamic programming. *Wiley series in probability and statistics*, Hoboken, NJ: Wiley-Interscience, OCLC: 254152847.
- Qu, Y., Chu, H., Gao, S., Guan, J., Yan, H., Xiao, L., Li, S. E., & Duan, J. (2024). RL-Driven MPPI: Accelerating Online Control Laws Calculation With Offline Policy. *IEEE Transactions on Intelligent Vehicles*, 9(2), 3605–3616.
- Raković, S. V. (2019). Robust Model Predictive Control. In J. Baillieul, & T. Samad (Eds.), *Encyclopedia of systems and control* (pp. 1–11). London: Springer London.
- Rakovic, S., Kouvaritakis, B., Cannon, M., Panos, C., & Findeisen, R. (2012). Parameterized Tube Model Predictive Control. *IEEE Transactions on Automatic Control*, 57(11), 2746–2761.
- Rao, A. V. (2009). A survey of numerical methods for optimal control. *Advances in the Astronautical Sciences*, 135(1), 497–528, Publisher: Univelt, Inc..
- Rao, C. V., Wright, S. J., & Rawlings, J. B. (1998). Application of Interior-Point Methods to Model Predictive Control. *Journal of Optimization Theory and Applications*, 99, 723–757.
- Rashid, T., Peng, B., Böhrer, W., & Whiteson, S. (2020). Optimistic Exploration even with a Pessimistic Initialisation. In *8th international conference on learning representations, ICLR*.
- Rawlings, J. B., Mayne, D. Q., & Diehl, M. M. (2017). *Model predictive control: Theory, computation, and design* (2nd ed.). Santa Barbara, California: Nob Hill Publishing.
- Rawlings, J. B., & Risbeck, M. J. (2017). Model predictive control with discrete actuators: Theory and application. *Automatica*, 78.
- Ray, A., Achiam, J., & Amodei, D. (2019). Benchmarking Safe Exploration in Deep Reinforcement Learning. URL <https://cdn.openai.com/safexp-short.pdf>.
- Recht, B. (2019). A Tour of Reinforcement Learning: The View from Continuous Control. *Annual Review of Control, Robotics, and Autonomous Systems*, 2(Volume 2), 253–279, Publisher: Annual Reviews.
- Reinhardt, D., Anand, A. S., Sawant, S., & Gros, S. (2024). Economic model predictive control as a solution to markov decision processes.
- Reiter, R., Ghezzi, A., Baumgärtner, K., Hoffmann, J., McAllister, R. D., & Diehl, M. (2025). AC4MPC: Actor-critic reinforcement learning for guiding model predictive control. *IEEE Transactions on Control Systems Technology*, 1–16.
- Reiter, R., Hoffmann, J., Boedecker, J., & Diehl, M. (2023). A Hierarchical Approach for Strategic Motion Planning in Autonomous Racing. In *European control conference (ECC)* (pp. 1–8).
- Reiter, R., Kirchengast, M., Watzenig, D., & Diehl, M. (2021). Mixed-integer optimization-based planning for autonomous racing with obstacles and rewards. In *7th IFAC conference on nonlinear model predictive control NMPC 2021: IFAC-PapersOnLine*, In *7th IFAC conference on nonlinear model predictive control NMPC 2021: Vol. 54*(6):99–106,
- Reske, A., Carius, J., Ma, Y., Farshidian, F., & Hutter, M. (2021). Imitation Learning from MPC for Quadrupedal Multi-Gait Control. In *2021 IEEE international conference on robotics and automation (ICRA)* (pp. 5014–5020). Xi'an, China: IEEE.
- Richter, S., Jones, C. N., & Morari, M. (2011). Computational complexity certification for real-time MPC with input constraints based on the fast gradient method. *IEEE Transactions on Automatic Control*, 57(6), 1391–1403, Publisher: IEEE.
- Rickenbach, R., Lahoud, A. A., Schaffernicht, E., Zeilinger, M. N., & Stork, J. A. (2025). ZipMPC: Compressed context-dependent MPC cost via imitation learning. arXiv:2507.13088.
- Romero, A., Song, Y., & Scaramuzza, D. (2024). Actor-Critic Model Predictive Control. In *2024 IEEE international conference on robotics and automation (ICRA)* (pp. 14777–14784). Yokohama, Japan: IEEE.
- Ross, S., Gordon, G., & Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In G. Gordon, D. Dunson, & M. Dudík (Eds.), *Proceedings of machine learning research: Vol. 15, Proceedings of the fourteenth international conference on artificial intelligence and statistics* (pp. 627–635). Fort Lauderdale, FL, USA: PMLR.
- Rubinstein, R. Y. (1997). Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1), 89–112, Publisher: Elsevier.
- Rubinstein, R. Y., & Kroese, D. P. (2004). The Cross-Entropy Method. In M. Jordan, J. Kleinberg, B. Schölkopf, F. P. Kelly, & I. Witten (Eds.), *Information science and statistics*, New York, NY: Springer.
- Sacks, J., & Boots, B. (2022). Learning to Optimize in Model Predictive Control. In *International conference on robotics and automation (ICRA)* (pp. 10549–10556).
- Salzmann, T., Arrizabalaga, J., Andersson, J., Pavone, M., & Ryll, M. (2023). Learning for CasADi: Data-driven models in numerical optimization. arXiv:2312.05873 [eess.SY].
- Sawant, S. V., & Gros, S. N. (2022). Bridging the gap between QP-based and MPC-based Reinforcement Learning. *IFAC-PapersOnLine*.
- Sawant, S., Reinhardt, D., Kordabad, A. B., & Gros, S. (2023). Model-Free Data-Driven Predictive Control Using Reinforcement Learning. In *62nd IEEE conference on decision and control (CDC)* (pp. 4046–4052).
- Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., & Graepel, T. (2020). Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839), 604–609, ISBN: 1476-4687 Publisher: Nature Publishing Group.
- Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015). Trust region policy optimization. CoRR. arXiv:1502.05477.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation.. In *4th international conference on learning representations, ICLR*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. CoRR. arXiv:1707.06347.
- Schulz, F., Hoffmann, J., Zhang, Y., & Boedecker, J. (2024). Learning When to Trust the Expert for Guided Exploration in RL. In *ICML 2024 workshop: foundations of reinforcement learning and control – connections and perspectives*.

- Schwan, R., Jones, C. N., & Kuhn, D. (2023). Stability Verification of Neural Network Controllers Using Mixed-Integer Programming. *IEEE Transactions on Automatic Control*, 1–16. Conference Name: IEEE Transactions on Automatic Control.
- Schwenzer, M., Ay, M., Bergs, T., & Abel, D. (2021). Review on model predictive control: an engineering perspective. *International Journal of Advanced Manufacturing Technology*, 117, 1327–1349.
- SciPy v1.15.0 Manual (2026). SciPy v1.15.0 Manual. [Online]. Available. URL <https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html>.
- Scokaert, P. O. M., & Mayne, D. Q. (1998). Min-max feedback model predictive control for constrained linear systems. *IEEE Transactions on Automatic Control*, 43, 1136–1142.
- Scokaert, P. O. M., Mayne, D. Q., & Rawlings, J. (1999). Suboptimal Model Predictive Control (Feasibility Implies Stability). *IEEE Transactions on Automatic Control*, 44(3), 648–654.
- Scokaert, P., Rawlings, J., & Meadows, E. (1997). Discrete-time Stability with Perturbations: Application to Model Predictive Control. *Automatica*, 33(3), 463–470.
- Seel, K., Gros, S., & Gravdahl, J. T. (2023). Combining Q-learning and Deterministic Policy Gradient for Learning-Based MPC. In *2023 62nd IEEE conference on decision and control (CDC)* (pp. 610–617). ISSN: 2576-2370.
- Seel, K., Kordabad, A. B., Gros, S., & Gravdahl, J. T. (2022). Convex Neural Network-Based Cost Modifications for Learning Model Predictive Control. *IEEE Open Journal of Control Systems*, 1, 366–379. Conference Name: IEEE Open Journal of Control Systems.
- Shapiro, A., Dentcheva, D., & Ruszczyński, A. (2009). *Lectures on stochastic programming: Modelling and theory*. SIAM.
- Shen, X., & Borrelli, F. (2023). Reinforcement Learning and Distributed Model Predictive Control for Conflict Resolution in Highly Constrained Spaces. In *IEEE intelligent vehicles symposium (IV)* (pp. 1–6). ISSN: 2642-7214.
- Shi, J., Li, K., Piao, C., Gao, J., & Chen, L. (2023). Model-Based Predictive Control and Reinforcement Learning for Planning Vehicle-Parking Trajectories for Vertical Parking Spaces. *Sensors*, 23(16), 7124. Number: 16 Publisher: Multidisciplinary Digital Publishing Institute.
- Sideris, A., & Bodrow, J. (2005). An efficient sequential linear quadratic algorithm for solving unconstrained nonlinear optimal control problems. *IEEE Transactions on Automatic Control*, 50(12), 2043–2047.
- Sikchi, H., Zhou, W., & Held, D. (2022). Learning off-policy with online planning. In A. Faust, D. Hsu, & G. Neumann (Eds.), *Proceedings of machine learning research: Vol. 164, Proceedings of the 5th conference on robot learning* (pp. 1622–1633). PMLR.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., & Hassabis, D. (2017). Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. *cs*.
- Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. A. (2014). Deterministic policy gradient algorithms. In *JMLR workshop and conference proceedings: Vol. 32, Proceedings of the 31th international conference on machine learning, ICML 2014, Beijing, China, 21–26 June 2014* (pp. 387–395). JMLR.org.
- Simpson, L., Ghezzi, A., Aspiron, J., & Diehl, M. (2023). An Efficient Method for the Joint Estimation of System Parameters and Noise Covariances for Linear Time-Variant Systems. In *62nd IEEE conference on decision and control (CDC)* (pp. 4524–4529).
- Singh, B., Kumar, R., & Singh, V. (2022). Reinforcement learning in robotic applications: a comprehensive survey. *Artificial Intelligence Review*, 55.
- Smith, L., Kostrikov, I., & Levine, S. (2023). Demonstrating a walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *Robotics: Science and Systems (RSS) Demo*, 2(3), 4.
- Song, Y., Naji, S., Kaufmann, E., Loquercio, A., & Scaramuzza, D. (2020). Flightmare: A Flexible Quadrotor Simulator. *Conference on Robot Learning (CoRL)*.
- Song, Y., Romero, A., Müller, M., Koltun, V., & Scaramuzza, D. (2023). Reaching the limit in autonomous racing: Optimal control versus reinforcement learning. *Science Robotics*, 8(82), eadg1462. Publisher: American Association for the Advancement of Science.
- Song, X., Yang, Y., Choromanski, K., Caluwaerts, K., Gao, W., Finn, C., & Tan, J. (2020). Rapidly adaptable legged robots via evolutionary meta-learning. In *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 3769–3776). IEEE.
- Sootla, A., Cowen-Rivers, A. I., Jafferjee, T., Wang, Z., Mguni, D. H., Wang, J., & Ammar, H. (2022). Saute RL: Almost Surely Safe Reinforcement Learning Using State Augmentation. In *Proceedings of the 39th international conference on machine learning* (pp. 20423–20443). PMLR, ISSN: 2640-3498.
- Steinbach, M. C. (2002). Tree-Sparse Convex Programs. *Mathematical Methods of Operations Research*, 56(3), 347–376.
- Stella, L., Themelis, A., Sotasakis, P., & Patrinos, P. (2017). A simple and efficient algorithm for nonlinear model predictive control. In *56th IEEE conference on decision and control (CDC)* (pp. 1939–1944).
- Still, G. (2018). Lectures on parametric optimization: An introduction. *Optimization Online*, 2.
- Sun, L., Peng, C., Zhan, W., & Tomizuka, M. (2018). *A fast integrated planning and control framework for autonomous driving via imitation learning*. American Society of Mechanical Engineers Digital Collection.
- Sutton, R. S. (1991). Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bull.*, 2(4), 160–163.
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: an introduction, In *Adaptive computation and machine learning series*, (2nd ed.). Cambridge, Massachusetts: The MIT Press.
- Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2), 181–211. Publisher: Elsevier.
- Tallec, C., Blier, L., & Ollivier, Y. (2019). Making deep q-learning methods robust to time discretization. In *International conference on machine learning* (pp. 6096–6104). PMLR.
- Tao, R., Cheng, S., Wang, X., Wang, S., & Hovakimyan, N. (2024). DiffTune-MPC: Closed-Loop Learning for Model Predictive Control. *IEEE Robotics and Automation Letters*. Publisher: IEEE.
- Tassa, Y., Mansard, N., & Todorov, E. (2014). Control-Limited Differential Dynamic Programming. In *IEEE international conference on robotics and automation*.
- Tearle, B., Wabersich, K. P., Carron, A., & Zeilinger, M. N. (2021). A Predictive Safety Filter for Learning-Based Racing Control. *IEEE Robotics and Automation Letters*, 6(4), 7635–7642.
- The MathWorks Inc (2022). *MATLAB version: 9.13.0 (R2022b)*. Natick, Massachusetts, United States: The MathWorks Inc, [Online]. Available. URL <https://www.mathworks.com>.
- Theodorou, E. A. (2015). Nonlinear Stochastic Control and Information Theoretic Dualities: Connections, Interdependencies and Thermodynamic Interpretations. *Entropy*, 17(5), 3352–3375. Number: 5 Publisher: Multidisciplinary Digital Publishing Institute.
- Theodorou, E. A., & Todorov, E. (2012). Relative entropy and free energy dualities: Connections to Path Integral and KL control. In *IEEE 51st IEEE conference on decision and control (CDC)* (pp. 1466–1473). ISSN: 0743-1546.
- Tilbury, C. R., Christianos, F., & Albrecht, S. V. (2023). Revisiting the Gumbel-Softmax in MADDPG. *arXiv:2302.11793 [cs]*.
- Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems* (pp. 5026–5033).
- Todorov, E., & Li, W. (2005). A generalized iterative LQG method for locally-optimal feedback control of constrained nonlinear stochastic systems. In *Proceedings of the American control conference (ACC)*.
- Tram, T., Batkovic, I., Ali, M., & Sjöberg, J. (2019). Learning When to Drive in Intersections by Combining Reinforcement Learning and Model Predictive Control. In *2019 IEEE intelligent transportation systems conference (ITSC)* (pp. 3263–3268).
- Van Parys, B. P. G., Kuhn, D., Goulart, P. J., & Morari, M. (2016). Distributionally Robust Control of Constrained Stochastic Systems. *IEEE Transactions on Automatic Control*, 61(2), 430–442.
- Vanroye, L., Sathya, A., De Schutter, J., & Decré, W. (2023). Fatrop: A fast constrained optimal control problem solver for robot trajectory optimization and control. In *2023 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 10036–10043). IEEE.
- Vaupel, Y., Hamacher, N. C., Caspari, A., Mhamdi, A., Kevrekidis, I. G., & Mitsos, A. (2020). Accelerating nonlinear model predictive control through machine learning. *Journal of Process Control*, 92, 261–270.
- Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., Duijken, N. v., Zanelli, A., Novoselnik, B., Albin, T., Quirynen, R., & Diehl, M. (2022). Acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1), 147–183.
- Villanueva, M. E., Müller, M. A., & Houska, B. (2024). Configuration-Constrained Tube MPC. *Automatica*, 163.
- Villanueva, M. E., Quirynen, R., Diehl, M., Chachuat, B., & Houska, B. (2017). Robust MPC via min-max differential inequalities. *Automatica*, 77, 311–321.
- Vlahov, B., Gibson, J., Gandhi, M., & Theodorou, E. A. (2024). MPPI-Generic: A CUDA Library for Stochastic Optimization.
- Wabersich, K. P., Taylor, A. J., Choi, J. J., Sreenath, K., Tomlin, C. J., Ames, A. D., & Zeilinger, M. N. (2023). Data-Driven Safety Filters: Hamilton-Jacobi Reachability, Control Barrier Functions, and Predictive Methods for Uncertain Systems. *IEEE Control Systems Magazine*, 43(5), 137–177. Conference Name: IEEE Control Systems Magazine.
- Wabersich, K. P., & Zeilinger, M. N. (2018). Linear Model Predictive Safety Certification for Learning-Based Control. In *2018 IEEE conference on decision and control (CDC)* (pp. 7130–7135). ISSN: 2576-2370.
- Wabersich, K. P., & Zeilinger, M. N. (2021). A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129, Article 109597.
- Wächter, A., & Biegler, L. T. (2006). On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1), 25–57.
- Wang, T., & Ba, J. (2019). Exploring Model-based Planning with Policy Networks.
- Wang, H., & Banerjee, A. (2014). Bregman Alternating Direction Method of Multipliers. In *Advances in neural information processing systems: Vol. 27*, Curran Associates, Inc..
- Wang, T., Bao, X., Clavera, I., Hoang, J., Wen, Y., Langlois, E., Zhang, S., Zhang, G., Abbeel, P., & Ba, J. (2019). Benchmarking Model-Based Reinforcement Learning. *arXiv:1907.02057 [cs]*.

- Wang, C., Gao, D., Xu, K., Geng, J., Hu, Y., Qiu, Y., Li, B., Yang, F., Moon, B., Pandey, A., Aryan, Xu, J., Wu, T., He, H., Huang, D., Ren, Z., Zhao, S., Fu, T., Reddy, P., Scherer, S. (2023). PyPose: A library for robot learning with physics-based optimization. In *IEEE/CVF conference on computer vision and pattern recognition*.
- Wang, D., Zheng, W., Wang, Z., Wang, Y., Pang, X., & Wang, W. (2023). Comparison of reinforcement learning and model predictive control for building energy system optimization. *Applied Thermal Engineering*, 228, Article 120430.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3), 279–292.
- Wen, Z., Dong, M., & Chen, X. (2024). Collision-Free Robot Navigation in Crowded Environments using Learning based Convex Model Predictive Control. In *2024 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 5452–5459). ISSN: 2153-0866.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3), 229–256.
- Williams, G., Aldrich, A., & Theodorou, E. A. (2017). Model Predictive Path Integral Control: From Theory to Parallel Computation. *Journal of Guidance, Control, and Dynamics*, 40(2), 344–357.
- Wurman, P. R., Barrett, S., Kawamoto, K., MacGlashan, J., Subramanian, K., Walsh, T. J., Capobianco, R., Devlic, A., Eckert, F., Fuchs, F., Gilpin, L., Khandelwal, P., Kompella, V., Lin, H., MacAlpine, P., Oller, D., Seno, T., Sherstan, C., Thomure, M. D., Kitano, H. (2022). Outracing champion Gran Turismo drivers with deep reinforcement learning. *Nature*, 602, 223–228.
- Yang, Y., Caluwaerts, K., Iscen, A., Zhang, T., Tan, J., & Sindhvani, V. (2020). Data Efficient Reinforcement Learning for Legged Robots. In *Proceedings of the conference on robot learning* (pp. 1–10). PMLR, ISSN: 2640-3498.
- Yoon, H.-J., Tao, C., Kim, H., Hovakimyan, N., & Voulgaris, P. (2022). Sampling Complexity of Path Integral Methods for Trajectory Optimization. In *2022 American control conference (ACC)* (pp. 3482–3487).
- Zanelli, A., Frey, J., Messerer, F., & Diehl, M. (2021). Zero-Order Robust Nonlinear Model Predictive Control with Ellipsoidal Uncertainty Sets. *Proceedings of the IFAC Conference on Nonlinear Model Predictive Control (NMPC)*.
- Zanon, M., & Gros, S. (2021). Safe Reinforcement Learning Using Robust MPC. *IEEE Transactions on Automatic Control*, 66(8), 3638–3652, Conference Name: IEEE Transactions on Automatic Control.
- Zanon, M., Gros, S., & Bemporad, A. (2019). Practical reinforcement learning of stabilizing economic MPC. In *18th European control conference (ECC)* (pp. 2258–2263). IEEE.
- Zanon, M., Gros, S., & Palladino, M. (2022). Stability-constrained Markov Decision Processes using MPC. *Automatica*, 143, Article 110399.
- Zanon, M., Kungurtsev, V., & Gros, S. (2020). Reinforcement Learning Based on Real-Time Iteration NMPC. *IFAC-PapersOnLine*, 53(2), 5213–5218.
- Zarrouki, B., Spanakakis, M., & Betz, J. (2024). A Safe Reinforcement Learning driven Weights-varying Model Predictive Control for Autonomous Vehicle Motion Control: 35th IEEE Intelligent Vehicles Symposium, IV 2024. In *IEEE intelligent vehicles symposium, proceedings* (pp. 1401–1408).
- Zarrouki, B., Wang, C., & Betz, J. (2024). Adaptive Stochastic Nonlinear Model Predictive Control with Look-ahead Deep Reinforcement Learning for Autonomous Vehicle Motion Control. In *2024 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 12726–12733). ISSN: 2153-0866.
- Zhang, Z., An, H., Wei, Q., & Ma, H. (2022). Learning-Based Model Predictive Control for Quadruped Locomotion on Slippery Ground. In *4th international conference on control and robotics (ICCR)* (pp. 47–52).
- Zhang, B., Rajan, R., Pineda, L., Lambert, N., Biedenkapp, A., Chua, K., Hutter, F., & Calandra, R. (2021). On the importance of hyperparameter optimization for model-based reinforcement learning. In *International conference on artificial intelligence and statistics* (pp. 4015–4023). PMLR.
- Zhang, H., Seal, S., Wu, D., Bouffard, F., & Boulet, B. (2022). Building Energy Management With Reinforcement Learning and Model Predictive Control: A Survey. *IEEE Access*, 10, 27853–27862.
- Zhang, B., Zhang, Y., Frison, L., Brox, T., & Bödecker, J. (2024). Constrained Reinforcement Learning with Smoothed Log Barrier Function. *arXiv:2403.14508 [cs]*.
- Zhong, M., Johnson, M., Tassa, Y., Erez, T., & Todorov, E. (2013). Value function approximation and model predictive control. In *IEEE symposium on adaptive dynamic programming and reinforcement learning (ADPRL)* (pp. 100–107). ISSN: 2325-1867.
- Ziebart, B. D., Maas, A. L., Bagnell, J. A., & Dey, A. K. (2008). Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI conference on artificial intelligence* (pp. 1433–1438). AAAI Press.